

AI for Everyone

From rules to deep learning

Onur Önder

AI for Everyone

Onur Önder

© 2026 Onur Önder. All rights reserved. Marketing and sales rights belong to Fittechs Yazılım Anonim Şirketi. No part of this book may be reproduced, published or transmitted in any form without the written permission of the author.

Published by the author.

Print edition ISBN: [ISBN]

Interactive digital edition ISBN: 978-625-00-5299-0

First edition: [month, year]

Cover and interior design: Onur Önder

Copy editing: Onur Önder

Printing and binding: [printer name, address, certificate no.]

Sales and contact: Fittechs Yazılım Anonim Şirketi, Gayrettepe Mah. Yıldız Posta Cad. No: 8/34, İstanbul, Türkiye. Email: support@fittechs.com. Web: onuronder.com

The interactive digital edition of this book, with all 45 live demos, is at book.onuronder.com.

Acknowledgments

This book was born on a screen first. Without the people who read the digital edition early, who poked at the demos and who said plainly where they got stuck, these pages would have stayed far more abstract. My thanks go to my first readers, [names] above all.

I owe the text to the eyes that read it line by line and were not shy to say “this part is not clear”: [names]. Every error that remains is mine.

The ideas this book rests on are not mine. From Turing to Hinton, from the people who wrote rules to the people who computed probabilities, I am indebted to the researchers who wrote this story and to everyone who shared what they learned in the open. The bibliography names only a few of them.

Finally, to my family, who put up with someone saying “one more chapter” for months: [names]. This book is yours too.

Onur Önder

Preface

This book begins with a question: Can a machine think? The question is not new. Long before computers, four hundred years ago, it kept philosophers busy. What is new is that the machines that can try to answer it are now sitting on the table.

I am going to tell a two-century story, from the first calculating gears to today's chatbots. Its heroes are ideas: rules, search, probability, learning, neurons, attention. Each one was born where the one before it hit a wall. So the chapters follow the order of history: each one shows where the last idea ran out and where the next began.

In the digital edition of this book every idea is tried with your own hands: you run a Turing machine step by step, turn the inputs of a neuron, pull a picture out of noise. Paper cannot do that. So I have worked every experiment out on the page, with numbers and drawings, and then handed it over to you. The "Try it yourself" exercises are solved with pencil and paper, and the link to the live demo sits under every figure.

I wrote the book at two depths. The main text is for someone with no background at all. The "Technical depth" boxes are for anyone who wants the formulas and the terminology behind the idea. You can read the book while skipping the boxes; nothing will be missing. If curiosity strikes later, go back to them.

I have tried to explain what everyone who talks about artificial intelligence needs to know, and to leave no reader behind. I hope that when you finish, you will have given your own answer to the question we started with.

Onur Önder

[place, date]

How to Read This Book

The book has eight chapters. Each chapter covers one era and the main idea of that era, and the chapters were written to be read in order. Still, every chapter stands on its own; you can also start wherever your curiosity takes you.

Two depths. The main text is written in plain language and is complete on its own. At the end of some sections there are boxes headed **Technical depth**. They retell the same idea with its formula, its terms and its math. The boxes can be skipped; the main text does not depend on them. It is the paper version of the dial on the cover, which switches one idea between two depths.

Margin notes. The short notes beside the text give the link between a topic and today, or a detail that is easy to miss at first glance.

Figures. Each of the 45 live demos in the digital edition has become a figure block here. Every block follows the same order:

- *Setup*: what you see in the figure.
- *Step by step* (or *Self-test* for the figures that quiz you): the experiment, carried out on paper with real numbers.
- *What is happening?*: a one-paragraph explanation.
- *Try it yourself*: one to three questions to solve with pencil and paper. Answers are at the back of the book.

Figures that show a process work like a film strip: follow the frames from left to right, row by row.

Live demos. The QR code under each figure opens the live version of that experiment on your phone. No sign-in or purchase is needed; only that demo opens. The whole book, with all of its demos, is in the digital edition. A list of all demos and their links is at the back of the book.

Test yourself. Every chapter ends with a short quiz. The answer key is at the back of the book. Some figures are tests as well (“Self-

test”); their answers and the reasoning are in the same place.

What to keep from this chapter. The bullet list at the end of each chapter sums up what should stay with you from that chapter. Once you have finished the book, reading only these lists is a good way to review.

Contents

Chapter 1 • Minds and Machines	13
1.1 Can a machine think?	14
1.2 What is intelligence?	15
1.3 Is thinking = computing?	18
1.4 From Babbage to Turing	22
1.5 ENIAC and the birth of the modern computer	26
1.6 Narrow AI or General AI?	29
1.7 Why now? Moore's law	31
1.8 Test yourself	35
Chapter 2 • The Age of Rules	37
2.1 The age of rules	38
2.2 Representing knowledge with symbols	39
2.3 If... then...: rules and expert systems	42
2.4 Search and heuristic shortcuts	46
2.5 Handling uncertainty: probability and Markov	49
2.6 Neats and scruffies	53
2.7 Test yourself	56
Chapter 3 • How Machines Learn	59
3.1 How do machines learn?	60
3.2 Features and labels	61
3.3 Three kinds of learning	64
3.4 Classification and regression	67
3.5 Clustering and anomalies	69
3.6 How a model improves: loss and gradient descent	73
3.7 Overfitting and ensembles	76
3.8 Test yourself	79
Chapter 4 • The Artificial Brain	83
4.1 The artificial brain: deep learning	84
4.2 A single artificial neuron	85

4.3 Layers and the forward pass	88
4.4 Backpropagation: learning from error	91
4.5 Seeing images: convolutional networks (CNN)	94
4.6 Understanding sequences: recurrent networks (RNN)	98
4.7 The art of forgery: GANs	101
4.8 Test yourself	105
 Chapter 5 • Today's AI	 109
5.1 The generative era: from recognizing to creating	110
5.2 How a machine sees words: tokens	111
5.3 Turning meaning into numbers: embeddings	115
5.4 Attention: the heart of the Transformer	118
5.5 A language model: predict the next word	121
5.6 How a model is raised: the training pipeline	125
5.7 Making images: diffusion	129
5.8 Limits: hallucination, context and cost	132
5.9 Test yourself	136
 Chapter 6 • Using and Building AI	 139
6.1 Using and building AI	140
6.2 Prompt engineering: asking the right question	141
6.3 RAG: give the model your own data	145
6.4 Agents: think, use tools, observe	149
6.5 The architecture of an AI application	153
6.6 AI in the real world	158
6.7 Test yourself	161
 Chapter 7 • AI and Society	 165
7.1 AI and society	166
7.2 Bias: from data to decisions	167
7.3 Black box or white box?	170
7.4 Deepfakes and disinformation	174
7.5 Regulation: classifying risk	176
7.6 Alignment: what you said, or what you meant?	180

7.7 Test yourself	183
Chapter 8 · Philosophy and the Future	185
8.1 Philosophy and the future	186
8.2 The Turing test: can you tell?	187
8.3 The Chinese Room: understanding or processing?	190
8.4 From narrow AI to superintelligence	193
8.5 The singularity: myth or serious possibility?	196
8.6 Responsibility and rights	200
8.7 Test yourself	203
Answer Key	207
Glossary	229
Bibliography and Further Reading	239
Live Demos	243
Index	247

1

CHAPTER 1

Minds and Machines

1.1 Can a machine think?

Once upon a time... except this story isn't over, and you are in it. It began long before computers, with a question: What is thinking? If some kind of gear turns inside our heads, couldn't a machine turn the same gear? People have argued about this for centuries.

This first chapter goes back to the very beginning. What is intelligence? Can “thinking” be written down as a step-by-step recipe? The path runs from Babbage's gears to the birth of the modern computer, and by the end it will be clear why artificial intelligence (AI for short) is possible at all.

Margin note. AI isn't new: “can machines think?” has occupied philosophers since the 1600s. What's new is the hardware that can test it.

TECHNICAL DEPTH

AI was born at the intersection of computability theory and the philosophy of mind. The core claim: if mental processes can be described by formal rules, then in principle a machine can carry them out too.

This chapter starts from defining intelligence, lays the foundation stones of computation (binary representation, algorithms, the Turing machine, the stored-program architecture) and connects to the modern debate via the weak/strong AI distinction. That gives the later learning and neural-network ideas solid ground to stand on.

Start with the first question. What is this “intelligence” you want to hand to a machine?

1.2 What is intelligence?

It looks like an easy question, but “what is intelligence?” has no single answer. Is the puzzle-solver smart, or the person who instantly finds their way in a new city, or the friend who reads your sadness in one glance? Perhaps all of them; yet each is a very different skill.

The psychologist Howard Gardner made the same point: intelligence doesn’t fit in one number (IQ); it comes in several kinds. Explore them below. The interesting part is that today’s AI is a master of some and still a toddler at others.

***Margin note.** A calculator is a million times faster than you at arithmetic but can’t laugh at a joke. Being “smart” isn’t a single axis.*

Figure 1.1 · Explore multiple intelligences

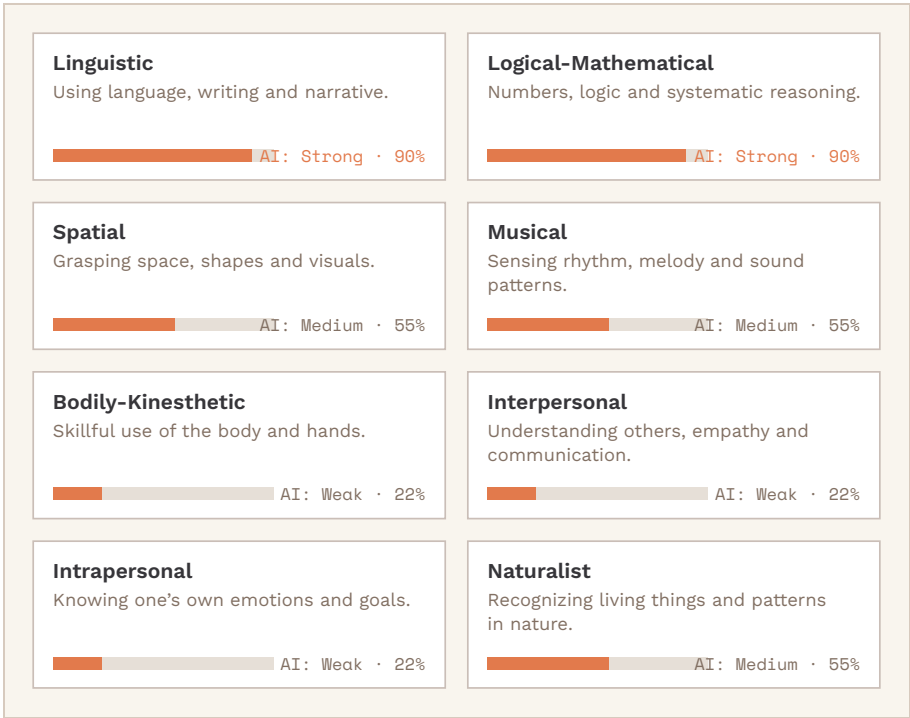


Figure 1.1

Setup. The figure shows eight cards, one for each kind of intelligence on Gardner’s list. The bar under each card shows where today’s AI stands in that area. A full bar means “strong,” a half bar “medium,” a short stub “weak.”

Step by step. All eight cards, in the order of the figure:

Intelligence type	What it means	AI today	Bar
Linguistic	Using language, writing and narrative.	Strong	90%
Logical-Mathematical	Numbers, logic and systematic reasoning.	Strong	90%
Spatial	Grasping space, shapes and visuals.	Medium	55%
Musical	Sensing rhythm, melody and sound patterns.	Medium	55%

Bodily-Kinesthetic	Skillful use of the body and hands.	Weak	22%
Interpersonal	Understanding others, empathy and communication.	Weak	22%
Intrapersonal	Knowing one's own emotions and goals.	Weak	22%
Naturalist	Recognizing living things and patterns in nature.	Medium	55%

Tally the last column and you get two “strong,” three “medium,” three “weak.” What do the two strong ones have in common? Both work with symbols: letters, numbers, rules. A computer runs on that same material; the binary code in the next section is a string of symbols too.

The three weak ones ask for a different material: a body, the face of the person across from you, your own inner world. None of these turns into symbols easily. The middle three, space, music and nature, sit between the two. Part of sound and image can be turned into numbers; part of it still resists.

The order of the cards is not a ranking, either. They only count separate abilities; none of them is “more intelligence” than another. The same holds for AI: being strong in language does not mean being strong in the other seven areas.

What is happening? For Gardner, intelligence isn’t one number but eight distinct families of ability; each card in the figure is one of them. And today’s AI is great with language and logic, yet far behind a small child at anything involving the body or emotions.

Try it yourself. 1) Close the book and list the eight kinds of intelligence from memory. How many did you recall, and which ones slipped your mind? 2) Think about yesterday: from morning to night, which three kinds did you use most? According to the table, in how many of those is today’s AI strong? 3) What do the three “weak” kinds in the table have in common?

Live demo:



book.onuronder.com/d/en/40ee8a0d04

TECHNICAL DEPTH

There is no agreed single definition of intelligence; working definitions cluster around “goal-directed adaptation, abstraction, learning and problem-solving capacity.” The AI field uses a pragmatic one: systems that perform tasks requiring human intelligence.

Gardner’s Theory of Multiple Intelligences (1983) splits intelligence into relatively independent domains: linguistic, logical-mathematical, spatial, musical, bodily-kinesthetic, interpersonal, intrapersonal and naturalistic. The theory is contested in psychometrics, but it usefully shows that intelligence is multi-dimensional: today’s AI is wildly uneven across these dimensions.

Each card is one of the eight domains in Gardner’s theory: AI is strong in language and logic, weak in the bodily and social/intrapersonal domains.

If intelligence comes in pieces, can at least one of those pieces be written down as a step-by-step recipe? A kitchen is a good place to find out.

1.3 Is thinking = computing?

Step into the kitchen for a moment. You hold a cake recipe: crack the eggs, whisk, add the flour, into the oven. Someone following that recipe to the letter will produce a cake without “understanding” baking at all. The idea beneath all of AI is the same: maybe “thinking,” too, is applying small, mechanical steps in order.

If so, a machine can do those steps too. That takes two things:

- A simple alphabet the machine can use (binary: 0 and 1)
- A clear list of steps (the recipe itself: an algorithm)

See how binary works for yourself below.

Margin note. The word “algorithm” comes from the name of the 9th-century Persian mathematician al-Khwarizmi. Binary itself was formalized by Leibniz in 1703.

Figure 1.2 • Crack the binary code

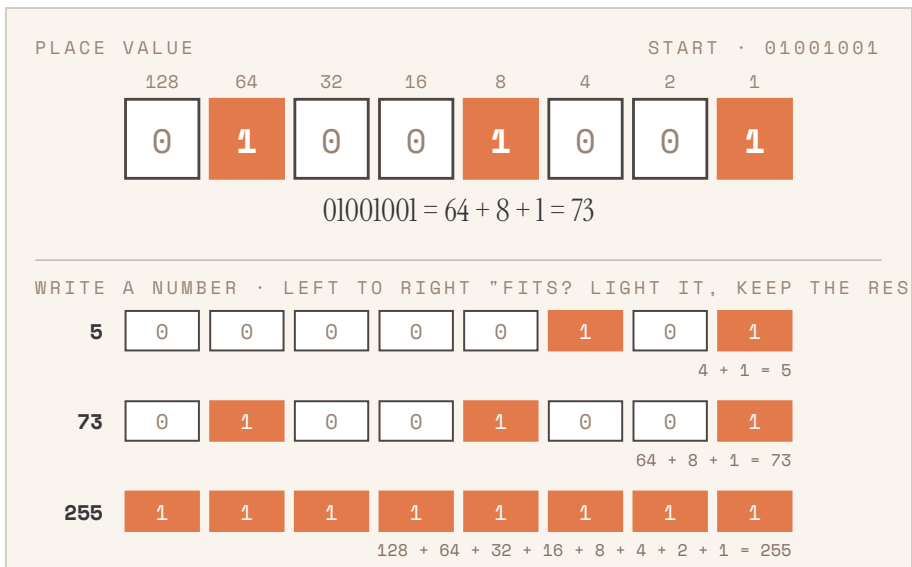


Figure 1.2

Setup. The figure shows eight boxes in a row. Above each box is its place value; from left to right, 128, 64, 32, 16, 8, 4, 2 and 1. A lit box means 1, a dark box means 0. The top row shows the opening state, 01001001. The number underneath is the sum of the place values of the lit boxes. Under it, three worked rows show 5, 73 and 255 written the same way.

Step by step. First, the place-value chart:

Box	1	2	3	4	5	6	7	8
Place value	128	64	32	16	8	4	2	1
Start	0	1	0	0	1	0	0	1

1. Decode the opening number. The lit boxes are 2, 5 and 8; their values are 64, 8 and 1. Add them: $64 + 8 + 1 = 73$. So 01001001 = 73.
2. Now go the other way and write the number 5 into the boxes. Start from the left and ask each box: “Does your value fit into the number?” 128, 64, 32, 16 and 8 do not fit; they all stay dark. 4 fits: light it, and 1 is left. 2 does not fit. 1 fits: light it, and 0 is left. The result is 00000101.
3. Write 73 the same way. 128 does not fit. 64 fits, 9 left. 32 and 16 do not fit. 8 fits, 1 left. 4 and 2 do not fit. 1 fits, 0 left. The result is 01001001, the same pattern as the opening state.
4. Finally, 255: light every box. $128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$, that is, 11111111. This is the largest number eight boxes can say. One more, 256, needs a ninth box.

Did you spot the rule? Going from left to right, “if it fits, light it and carry on with the remainder” is all it takes. Every number has one pattern and one only; two different patterns never give the same number. Eight boxes hold 256 different numbers, from 0 to 255; computer people call that a byte.

What is happening? Eight boxes, eight bulbs: off means 0, on means 1. But who handed out the values? Start from the right: the right-most box holds the smallest value, 1. Moving left, each box carries double its right-hand neighbor: 1, 2, 4, 8, 16, 32, 64, and 128 on the far left. Why always double? Because each new box must be able to say one more than all the boxes to its right combined; otherwise some numbers could never be written. Now add up the values of the lit boxes: there is your number. A computer’s whole world is this game of on and off.

Try it yourself. 1) Write the number 10 with eight boxes. 2) Which number is the pattern 10110000? 3) If only the leftmost box is lit, what is the number? If you added a ninth box, what would the largest number be?

Live demo:



book.onuronder.com/d/en/4642d3916a

TECHNICAL DEPTH

This view is called computationalism: the mind is an information-processing system, and cognition is computation in the form of symbol manipulation. Its roots reach back to Hobbes (“reasoning is reckoning”); its modern form developed with Putnam and Fodor.

In practice, two building blocks are needed. Binary representation: all information is encoded in base-2 (bits); Boolean algebra (Boole, 1854) and Shannon’s 1937 thesis showed that logic can be realized with electrical switches. Algorithm: a finite sequence of well-defined steps (the term comes from the 9th-century mathematician al-Khwarizmi). Together they make “thinking” mechanically executable.

Binary 01001001 = the sum of its place values = $64 + 8 + 1 = 73$. Each bit represents 128, 64, 32, ... from left to right. 8 bits (1 byte) encode any number from 0 to 255.

General rule: $\text{value} = \sum b_i \cdot 2^i$, with $i = 0 \dots 7$ counted from right to left; b_i is the 0 or 1 of that bit. n bits encode 2^n distinct numbers; for 8 bits, $2^8 = 256$.

The alphabet and the recipe are ready. Someone still has to carry the recipe out, and the first candidates, back in the 1800s, were made of gears.

1.4 From Babbage to Turing

The dream of a “computer” was first dreamed in the 1800s. Charles Babbage drew a giant calculating machine of gears and levers: the Analytical Engine. Money and years ran out; he died without ever seeing it built. But his friend Ada Lovelace wrote the first “recipe” for that imaginary machine. So she counts as history’s first programmer.

About a century later, Alan Turing carried the dream a step further. Picture a single little box that reads a tape, writes on it and slides left or right. In principle, it can perform any calculation. It is called a Turing machine. Follow a real one as it “thinks,” frame by frame, in Figure 1.3.

***Margin note.** In 1948 Turing designed a chess algorithm called “Turochamp.” No machine of the era could run it, so Turing worked out the moves by hand on paper.*

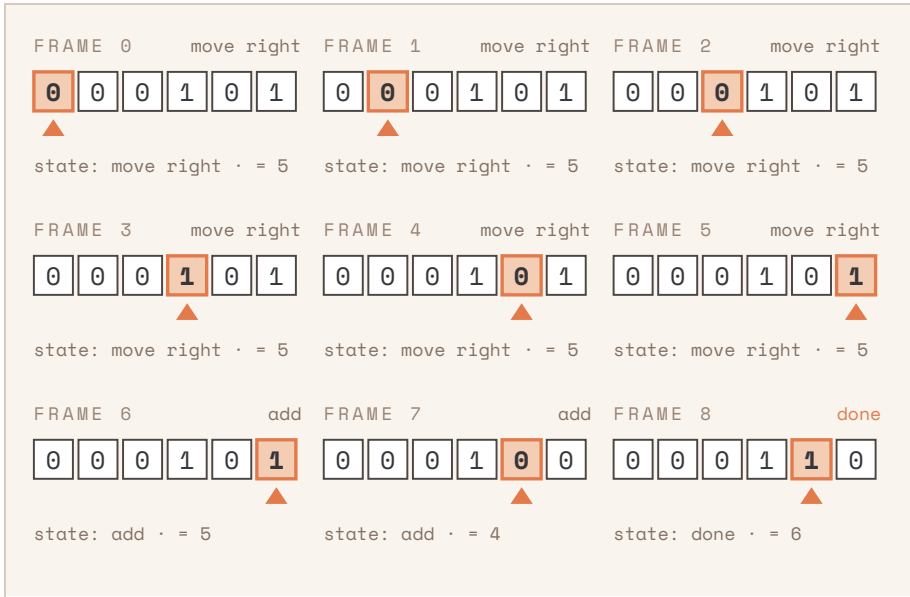
Figure 1.3 · A working Turing machine (+1)

Figure 1.3

Setup. The figure shows a tape of six cells; the arrow ($\blacktriangle$) moving above it is the machine's head. The tape reads 000101, which is 5 in binary. The machine has three states: "move right," "add" (add 1 with carry) and "done." The nine frames of the film strip show every step taken while 1 is added to 5.

Step by step. Read the frames in order; each frame gives the tape, the head position and the state. The cells are numbered 0 to 5 from left to right.

0. Start. The head is on the leftmost cell (0), the state is "move right." The tape reads 0 0 0 1 0 1, that is, 5.
1. The head slides one cell to the right, to cell 1. The tape does not change. The head ($\blacktriangle$) steps right to find the number's rightmost digit. As in adding by hand, the work starts from the far right.
2. The head is on cell 2. It is still only reading, not writing.
3. The head is on cell 3; there is a 1 here, but in this state the machine ignores it and moves on.
4. The head is on cell 4.

5. The head is on the last cell (5). The tape still reads 0 0 0 1 0 1.
6. No cell is left on the right. The machine changes state: “add” (add 1 with carry). The head stays put, the tape stays the same.
7. The head sees a 1 on the last cell. It turns the 1 into 0 and carries one to the left; the head moves to cell 4. The tape reads 0 0 0 1 0 0, for a moment 4. The head now works backward, adding 1 leftward: every 1 it sees becomes 0 with a carry, and at the first 0 it writes 1 and stops. The same “carry the one” rule you learned in school.
8. The head sees a 0 on cell 4. It writes a 1 there and stops; the state is “done.” The tape reads 0 0 0 1 1 0, that is, 6. Done. With only a handful of simple rules like “read, write, move left/right,” the machine added 1 to a binary number. Computing, for a Turing machine, means nothing more than this.

Eight moves in all: five steps to the right, one state change, one clear, one write. Five plus one made six; the tape says it as 000110.

What is happening? The machine walks the tape like an ant: first it heads right to find the end of the number, then turns back and adds 1. The way it carries the digit is no different from how you carry in addition on paper. Moves this small can carry out any calculation at all.

Try it yourself. 1) Start the tape at 001111 (15) and draw the frames yourself on paper. How many 1s get cleared, in how many moves does the machine finish, and which number is left on the tape? 2) Start the tape at 010110 (22). When the head reaches the end and turns back, what is the first digit it sees? In how many moves does it finish? 3) What would happen with a tape of 111111 (63)? The machine sees a 1 even on the leftmost cell; it cannot move left, so it stops.

Live demo:



book.onuronder.com/d/en/6c232ac487

TECHNICAL DEPTH

Babbage’s Analytical Engine (1837) was a general-purpose design with a “mill” (processing unit) and a “store” (memory), programmable via punched cards. Ada Lovelace’s 1843 notes contain the algorithm for computing Bernoulli numbers, generally regarded as the first computer program.

Alan Turing’s 1936 paper “On Computable Numbers” introduced the abstract Turing machine and the universal Turing machine; this is the formal basis of computability. The simulation in Figure 1.3 is a small Turing machine that adds 1 to a binary number, doing arithmetic with only read/write and state transitions.

The machine slides right to the end of the tape, then turns left and adds 1 to the binary number (carry logic): the 1s it sees become 0, and the first 0 becomes 1.

The transition table of the machine in Figure 1.3, taken from the source code:

State	Read	Write	Move	New state
move right	0 or 1	same	right	move right; at the end of the tape, “add 1”
add 1	1	0	left	add 1; at the start of the tape, “done”
add 1	0	1	stay	done

Three rows, a six-cell tape and a single head: enough to add 1 to any number from 0 to 62. For 63 the tape overflows and 0 remains.

Turing’s machine was a thought on paper. Pouring it into real tubes and wires is another story.

1.5 ENIAC and the birth of the modern computer

In 1945 a giant machine roared to life: ENIAC. It weighed tons and ran on tens of thousands of tubes. And this first electronic computer had a grumpy streak: teaching it a new job meant days of unplugging and replugging cables by hand.

The fix was an elegant idea: keep the recipe next to the ingredients, in memory. Then, instead of rewiring the machine, you load new “instructions.” Nearly every computer today, including the phone in your pocket, runs on this arrangement. Follow the cycle in the figure below, then look at what its parts do.

Margin note. Reprogramming ENIAC took days; the cables had to be reconnected by hand. Putting the program in memory changed all of that.

Figure 1.4 • The Fetch – Execute – Write cycle

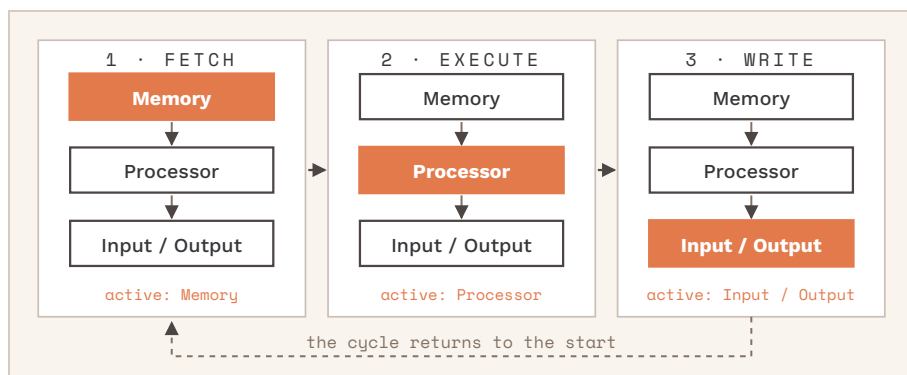


Figure 1.4

Setup. The figure shows three boxes, Memory, Processor and Input / Output, in three frames side by side: “1 · Fetch,” “2 · Execute,” “3 · Write.” In each frame the part at work in that phase is dark. After the third frame the cycle starts over.

Step by step. The parts first, as the figure describes them:

Phase	Active part	What it does
1 · Fetch	Memory	Stores both the program (instructions) and the data together. The real revolution is here: instructions are kept just like data.
2 · Execute	Processor	The ALU computes (adding, comparing); the control unit fetches instructions from memory in order and executes them.
3 · Write	Input / Output	The link to the outside world: keyboard, screen, sensors. Data comes in here, results go out here.

Now turn the cycle with a small program. Let memory hold two lines: the first line says “add 5 and 3,” the second says “write the result to the screen.” The numbers 5 and 3 sit in memory too.

1. Fetch. The control unit reaches into memory and takes the instruction on the first line: “add 5 and 3.” The instruction came from memory the same way 5 and 3 did; it is data, not a cable.
2. Execute. The ALU does the addition: $5 + 3 = 8$. For now the result stays inside the processor.
3. Write. The 8 is put back into memory; in this round the Input / Output part has nothing to do, and nothing appears on the screen.
4. Fetch. The cycle has come back to the start; the control unit takes the second line: “write the result to the screen.”
5. Execute. The processor reads the 8 from memory and prepares it for output.
6. Write. An 8 appears on the screen. The two-line program is finished; if there is a next instruction, the cycle goes on.

Two rounds, six phases. On ENIAC, changing the first line meant days of pulling cables. Here the only thing to do is write a different instruction into the first line of memory, say “multiply 5 by 3.” The cycle stays the same; only the instruction it picks up changes. A stored program means no more than that: you change one line in memory, not the wiring.

What is happening? A computer performs a three-step dance non-stop: grab the next instruction from memory (fetch), do what it says (execute), save or display the result (write). This is the loop beating at the heart of your phone and your computer alike.

Try it yourself. 1) You typed 7×6 into the calculator on your phone. Describe the three phases in your own words; which part is at work in each one? 2) Add a third line to the program above: “multiply the result by 2.” How many more phases are needed, and which number is written to memory? 3) Picture a processor that runs 3 billion cycles per second. Each cycle has three phases. How many phases in one second?

Live demo:



book.onuronder.com/d/en/c4fdfa18f9

TECHNICAL DEPTH

ENIAC (1945; Mauchly & Eckert) was a general-purpose electronic digital computer; it used ~17,500 vacuum tubes and was programmed by rewiring plugboards (its first programmers were six women mathematicians). Internally it worked in decimal, not binary.

John von Neumann’s 1945 EDVAC report popularized the stored-program principle: program and data live in the same memory. The architecture consists of a processor (ALU + control unit), memory and input/output, talking over a bus. The processor runs a continual “fetch–decode–execute” cycle. The “von Neumann bottleneck” is this design’s well-known limit.

In the “fetch” phase, the control unit reads the next instruction from memory; the program sits in memory just like data (the stored-program principle). In “execute” the processor does the work; in “write” the result goes out.

The machine can now run any recipe. So what does “artificial intelligence” mean today, and how much intelligence is in it?

1.6 Narrow AI or General AI?

The all-understanding AI of the movies doesn’t exist yet. Every AI today is “narrow”: a champion at its own job, a beginner one step outside it. A chess engine can beat you, yet you can’t ask it for a soup recipe.

The hypothetical system that can learn across every domain like a human is called “artificial general intelligence” (AGI); nobody has built one yet. Sort the examples below: Which exists today, and which is still science fiction?

Margin note. A system doing many tasks well (general) and a system truly “understanding / being conscious” (strong) are different questions. Keeping them apart is key to the modern AI debate.

Figure 1.5 • Real today, or science fiction?

#	Example	Narrow AI • exists today	General / AGI • not yet
1	Chess engine	☐	☐
2	Face recognition system	☐	☐
3	Chatbot (language model)	☐	☐
4	A machine that learns any profession like a human and has its own goals	☐	☐
5	A self-aware, conscious AI	☐	☐

Figure 1.5

Setup. The figure shows two columns. The left one is headed “Narrow AI • exists today,” the right one “General / AGI • not yet.” Five

cards sit in the middle, and the boxes next to them are empty; you fill them in with a pencil. There is no third column.

Self-test. Put each card in a column. There is only one test: is the system a master of one job, or can it learn a job it has never seen, the way a human does? Judge each card by how many jobs it knows, not by how impressive it sounds. Answers and reasons are at the back of the book.

#	Example	Narrow AI · exists today	General / AGI · not yet
1	Chess engine	☐	☐
2	Face recognition system	☐	☐
3	Chatbot (language model)	☐	☐
4	A machine that learns any profession like a human and has its own goals	☐	☐
5	A self-aware, conscious AI	☐	☐

After you have marked all five, count: how many cards are in the left column, how many in the right? The cards on the right share one thing; name it and you have the main idea of the chapter.

What is happening? The rule is simple: any system that masters only one job is “narrow,” chatbots included. “General” means a machine that can learn across every domain like a human, and no such machine exists yet. A conscious machine is a different story altogether.

Try it yourself. 1) Write down three AI examples from your own daily life: navigation, translation, movie recommendations. Give each one a column. 2) The fourth and fifth cards are in the same column; are they the same thing? Apply the two questions in the margin note (“general” and “strong”) to each of them separately. 3) What happens if you ask a chess engine for a soup recipe? Answer with a definition of narrow AI.

Live demo:



book.onuronder.com/d/en/e69cfc21ea

TECHNICAL DEPTH

Weak (narrow) AI refers to systems that perform specific tasks without genuine understanding or consciousness; all of today's systems fall here. The terms were coined by philosopher John Searle (1980, the Chinese Room argument).

An important distinction: Searle's "strong AI" is about whether the machine truly understands (has a mind); in everyday usage it is often conflated with "artificial general intelligence" (AGI). AGI is a hypothetical capability level that generalizes across domains; "strong AI" is a philosophical claim. They are not the same thing.

Careful: "general" (AGI, cross-domain generalization) and "strong AI" (Searle: does the machine truly understand / is it conscious?) are different questions. Even AGI carries no consciousness requirement: being general does not mean being conscious.

The ideas had been on the table for a hundred years. So why did even narrow AI arrive only in the last few years?

1.7 Why now? Moore's law

The ideas were ready by the mid-1900s; the machines were feeble. Do you know the old rice fable? One grain on the first square of a chessboard, double on every square after... halfway across the board, no granary on earth is enough. In 1965 Gordon Moore noticed the

transistors on chips doubling at a steady beat, just like that; by 1975 he had pinned the pace at roughly every two years.

Doubling looks innocent, but repeated it explodes. Do it yourself in the table of Figure 1.6: double the number every two years and see how it rockets. This compounding computation is what carries modern AI.

Margin note. If you could fold a sheet of paper 42 times, it would reach the Moon. That is exponential growth; chips grew at that pace for decades.

Figure 1.6 • Feel exponential growth

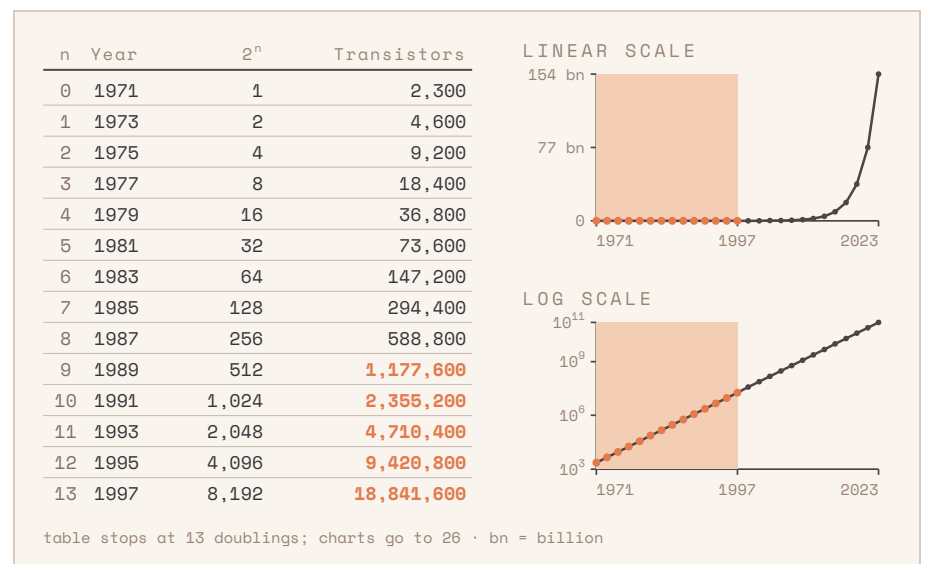


Figure 1.6

Setup. The figure starts in 1971. That year the first microprocessor, the Intel 4004, went on sale with 2,300 transistors. Each step in the figure means “two years passed, the count doubled.” The table stops after 13 doublings, in 1997; the two small charts on the right carry the numbers on to 2023. On the linear scale the curve hugs the floor and then shoots up; on the log scale it is a straight line.

Step by step. Follow the table row by row. The third column is 2^n , the fourth is $2,300 \times 2^n$. Each row is twice the one before; there is no other rule.

Doubling (n)	Year	2^n	Transistors
0	1971	1	2,300
1	1973	2	4,600
2	1975	4	9,200
3	1977	8	18,400
4	1979	16	36,800
5	1981	32	73,600
6	1983	64	147,200
7	1985	128	294,400
8	1987	256	588,800
9	1989	512	1,177,600
10	1991	1,024	2,355,200
11	1993	2,048	4,710,400
12	1995	4,096	9,420,800
13	1997	8,192	18,841,600

The first rows look tame: 2,300, 4,600, 9,200. In ten years the count grows 32-fold but is still in the tens of thousands. In 1989 it crosses the one-million line: 1,177,600, shown as “1.2 million” in the live demo. In 1997 it is 18.8 million. The table stops after 13 doublings, 26 years; the charts go on to 26 doublings.

What if you keep going? After 20 doublings, in 2011, $2^n = 1,048,576$ and the count is 2,411,724,800, or 2.4 billion. After 26 doublings, in 2023, $2^n = 67,108,864$ and the count is 154,350,387,200: 154.4 billion. Between the first row and the last there is a factor of 67 million. In the rice fable the granaries ran out on the 32nd square; chips reached the 26th square in 2023.

What is happening? Every row means “two years passed, the power doubled.” Within a few rows the number takes off; that is what exponential growth is like. Computers grew at this pace for decades, and that accumulation is what made today’s AI possible.

Try it yourself. 1) Extend the table by two rows: 2ⁿ and the transistor count for 1999 and 2001. 2) Starting from 2,300, which row is the first to pass one million; how many years did it take? 3) If the doubling time were 3 years instead of 2, how many transistors would there be in 1995? Compare with the 1995 value in the table.

Live demo:



book.onuronder.com/d/en/eb5ac74b07

TECHNICAL DEPTH

Moore’s law is not a law of nature but an empirical observation and economic trend: the transistor count on an integrated circuit roughly doubles every two years (Moore’s 1965 observation, revised in 1975). This exponential growth made the heavy matrix computation deep learning needs economical.

The exponential trend held for ~50 years (1971 Intel 4004: ~2,300 transistors → 2020s: tens of billions). It has been slowing recently due to physical limits (subatomic scale, heat); the industry now leans on parallelism and specialized AI hardware such as GPUs/TPUs.

Exponential growth: $\times 2$ roughly every 2 years; n doublings give a 2^n -fold increase.

The formula behind Figure 1.6: $N(t) = N_0 \cdot 2^{(t/2)}$, where t is the number of years since 1971 and $N_0 = 2,300$. For $t = 52$, $N = 2,300 \cdot 2^{26} \approx 1.5 \times 10^{11}$. Real chips sit a little below this curve; the doubling time has stretched since the 2000s.

The building blocks of the chapter are in place: intelligence, binary code, algorithm, Turing machine, stored program, the narrow-versus-general distinction, exponential growth. Six questions follow.

1.8 Test yourself

Answers are at the back of the book.

1. What did Howard Gardner propose?
 - a) That IQ fully measures intelligence
 - b) That the brain is a computer
 - c) That machines can never think
 - d) That intelligence has many kinds, not one
2. Binary code is made of which symbols?
 - a) A to Z
 - b) Dots and dashes
 - c) 0 and 1
 - d) 0–9
3. What can a Turing machine fundamentally do?
 - a) In principle any computation, via simple rules
 - b) Only addition
 - c) Only play chess
 - d) Only store text
4. The core idea of the von Neumann architecture?
 - a) Inventing the internet
 - b) Keeping program and data in the same memory
 - c) Using binary instead of decimal
 - d) Rewiring for each new task
5. What kind are today's AI systems?
 - a) General AI (AGI)
 - b) Conscious AI
 - c) Narrow AI
 - d) Strong AI

6. What does Moore's law say?

- a) Internet speed is constant
- b) Computers get cheaper yearly
- c) AI will surpass humans
- d) Transistor count ~doubles every 2 years

What to keep from this chapter

- Intelligence is not one number; across Gardner's eight kinds, today's AI is strong in language and logic, weak in body and emotion.
- Two building blocks are enough for "thinking is computing": binary code (0 and 1) and an algorithm (a clear list of steps).
- Eight boxes hold every number from 0 to 255; adding up the place values of the lit boxes is all it takes.
- A Turing machine performs any calculation with nothing but read, write and move one cell; adding 1 to 5 took eight moves.
- Stored program: an instruction sits in memory just like data; the processor turns the fetch, execute, write cycle without pause.
- Every AI today is narrow; general AI (AGI) does not exist yet, and a conscious machine is a separate question altogether.
- Moore's law: transistors doubled roughly every two years; the 2,300 of 1971 reach 154 billion after 26 doublings.

2

CHAPTER 2

The Age of Rules

2.1 The age of rules

Today, AI means systems that learn from data. Yet the story's first great act was the exact opposite: the masters of that era tried to make the machine memorize the world. Whatever it needed to act intelligently (all the knowledge, all the rules) they wrote out by hand, one line at a time.

This approach is called “classical” or “symbolic” AI. Logic, rules, search and expert systems were that era's toolbox. This chapter visits it: how did these ideas work, what did they achieve, and why did they one day hit a wall? You will try every bit of it with your own hands.

Margin note. Early AI researchers thought: “If we write enough rules, the machine becomes intelligent.” It worked beautifully for some tasks, and not at all for others.

TECHNICAL DEPTH

Classical AI (symbolic AI, or GOFAI, short for “Good Old-Fashioned AI”) treats intelligence as rule-based manipulation of formal symbols. The core assumption: knowledge about the world can be represented explicitly, and reasoning can run as logical operations over those representations.

This paradigm dominated from the 1950s to the 1980s and produced powerful tools such as logic programming, search algorithms and expert systems. This chapter builds them up, then shows why the “knowledge-acquisition bottleneck” and brittleness drove the shift to statistical, learning approaches (Chapter 3).

Start with the most basic question of all: how does a machine know that Tom is a cat?

2.2 Representing knowledge with symbols

How would you teach a machine about Tom? It cannot see him or pet him; it knows him only through the sentences you write: “Tom is a cat,” “a cat is a mammal,” “a mammal is an animal.” Classical AI stores knowledge in that form: explicit symbols and the links between them.

The best part is that the machine follows those links and reaches facts nobody ever told it. It never heard the sentence “Tom is an animal”; but walking the chain link by link, it finds that out itself. Figure 2.1 asks four questions; read how the machine “thinks” there, step by step.

***Margin note.** Symbolic AI's strength: it can derive many new facts from a few rules. Its weakness: someone has to write those rules first.*

Figure 2.1 • Inference along a knowledge chain

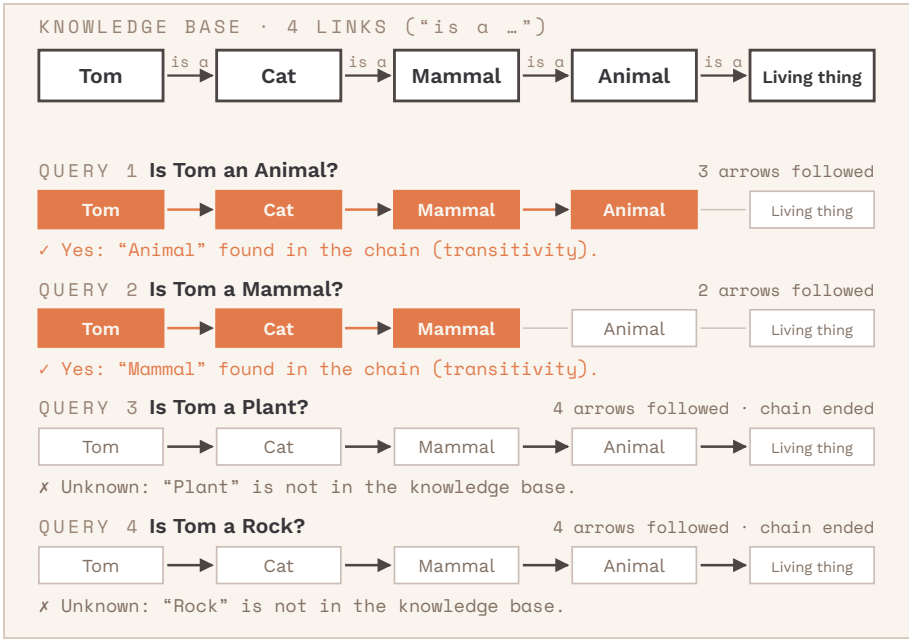


Figure 2.1

Setup. The figure shows five boxes in a single row: Tom, Cat, Mammal, Animal, Living thing. Every arrow between two boxes means “is a”: Tom is a Cat, a Cat is a Mammal, and so on. These four arrows are the machine’s entire knowledge; it knows nothing else. Under the chain are four queries; for each one the machine walks the chain from the start and states its verdict.

Step by step. Follow the query “Is Tom a Mammal?” first.

1. The machine starts at Tom. The word it is looking for is “Mammal.” Is Tom himself a Mammal? No; but an arrow leaves Tom: Cat.
2. It moves on to Cat. Is Cat “Mammal”? No; an arrow leaves Cat as well: Mammal.
3. It arrives at Mammal. The word it was looking for is found. In the figure the first three boxes are shaded and the verdict reads “Yes: ‘Mammal’ found in the chain (transitivity).”

It followed two arrows. Nobody wrote the sentence “Tom is a Mammal”; the machine derived it.

Now the query “Is Tom a Plant?”

1. Tom: not a Plant; arrow to Cat.
 2. Cat: no; arrow to Mammal.
 3. Mammal: no; arrow to Animal.
 4. Animal: no; arrow to Living thing.
 5. Living thing: no. No arrow leaves Living thing; the chain has ended.
- The verdict reads “Unknown: ‘Plant’ is not in the knowledge base.”

All four queries:

Query	Arrows followed	Verdict
Is Tom an Animal?	3	Yes
Is Tom a Mammal?	2	Yes
Is Tom a Plant?	4, chain ended	Unknown
Is Tom a Rock?	4, chain ended	Unknown

The verdict is “Unknown,” not “No.” The machine does not claim that Tom is not a plant; it only says that its arrows cannot reach that fact. Staying silent about what it does not know is a virtue of symbolic systems.

What is happening? All the machine holds is “what kind of thing is what”: Tom is a cat, a cat is a mammal... When you ask, it follows the chain link by link and reaches a fact nobody ever told it. And if it isn’t in the chain, it admits it: “unknown.”

Try it yourself. 1) Add one more arrow at the end of the chain: a Living thing is an Entity. What does the machine say to “Is Tom an Entity?” and how many arrows does it follow? 2) Turn the question around: “Is a Cat a Tom?” The arrows point one way only; what does the machine answer? Do you think that answer is right? 3) What would you have to add to the knowledge base for the machine to say “Tom is not a Plant”?

Live demo:



book.onuronder.com/d/en/48e9aced3e

TECHNICAL DEPTH

In symbolic AI, knowledge is encoded via knowledge representation: semantic networks, frames, ontologies or logical propositions. Entities and their relations (e.g. is-a, has-a) are defined explicitly.

Inference is the application of rules over these representations. The demonstration in Figure 2.1 uses transitivity in an is-a hierarchy: if “Tom is-a Cat” and “Cat is-a Mammal,” then “Tom is-a Mammal” can be derived. This is the essence of symbolic reasoning.

The knowledge base contains only consecutive “is-a” links. For each query the inference engine follows the chain using the transitivity rule: if the target is in the chain, “Yes”; otherwise, “Unknown.”

Written formally, the transitivity rule reads: $\text{is-a}(A, B) \wedge \text{is-a}(B, C) \rightarrow \text{is-a}(A, C)$. The engine applies this rule over and over along the chain; on a five-node chain it either reaches the target or runs off the end within four steps.

The chain only ever followed “is a” links. Getting from what a machine knows to what it should do takes another kind of sentence: if it is raining, then what?

2.3 If... then...: rules and expert systems

At the heart of classical AI beats one tiny sentence: “IF this is true, THEN do that.” This is a rule. Stack enough rules and systems are

born that act like masters of a field. These are expert systems: apprentices that hold a doctor's knowledge as question-and-answer rules.

In the 1980s these apprentices worked everywhere, from medicine to engineering. Below is a tiny one: switch the conditions on and off in your head; see in Figure 2.2 which rule “fires” and how the advice is born.

Margin note. One rule's output can trigger another rule; this is “chaining.” From a single fact the system can build a long chain of reasoning.

Figure 2.2 · A tiny expert system

FACTS · SCENARIO A

✓ It's raining

○ It's cold

○ It's windy

IF

THEN

R1

It's raining

→

Take an umbrella

🔥

R2

It's cold

→

Wear a coat

R3

Cold AND windy

→

Wear a scarf

R4

No rain AND not cold

→

You can dress light

R5

"Umbrella" fired AND windy

→

Careful: the umbrella may flipchain

SYSTEM ADVICE

Take an umbrella

THREE SCENARIOS

Scenario	Facts on	Fired	Advice line
A (initial)	Rain	R1	Take an umbrella
B	Rain, Wind	R1, R5	Take an umbrella · Careful: the umbrella may flip!
C	Cold, Wind	R2, R3	Wear a coat · Wear a scarf

Figure 2.2

Setup. At the top of the figure are three facts: “It’s raining” is on, “It’s cold” and “It’s windy” are off. Under them sit five rule cards, R1 to R5; the IF condition is on the left of each card, the THEN advice on the right. A rule that fires is shaded; R5 carries a “chain” mark because it depends on another rule. The advice of every fired rule is

lined up at the bottom. At the start only R1 fires: “Take an umbrella.” A small table in the figure lists three scenarios; the text below walks through them.

Step by step. The system does the same three jobs every time. It looks at the facts that are on, compares each rule’s IF part with those facts, and fires the rules that match. Three scenarios:

Scenario	Facts on	Rules fired	Advice line
A (initial)	Rain	R1	Take an umbrella
B	Rain, Wind	R1, R5	Take an umbrella · Careful: the umbrella may flip!
C	Cold, Wind	R2, R3	Wear a coat · Wear a scarf

In scenario A only rain is on. R1’s condition is “It’s raining”: it holds, so R1 fires. R2 and R3 ask for cold, R5 asks for wind; all three stay silent. R4 says “No rain”; it is raining, so R4 is silent too.

In scenario B wind is on as well. The IF part of R5 names no fact but the result of another rule: “Umbrella” fired. So the engine runs two rounds. In the first round R1 fires and drops an “umbrella” note into working memory. In the second round R5 sees that note, combines it with the wind and fires. Two facts gave two pieces of advice; the second is built on the first. This is chaining. Set rain to off and R1 goes dark, no note is dropped, and R5 stays silent too: wind alone is not enough.

In scenario C rain is off, cold and wind are on. R1 and R5 are silent. Cold fires R2; cold AND windy fires R3. R4 gets past its “no rain” condition, but its “not cold” condition fails; it stays silent.

What is happening? The system checks every condition that is on against its rules: whichever rule’s “IF” part holds, that rule fires and gives its advice. R5 is the odd one out: it waits for another rule’s result. One rule feeding the next is chaining.

Try it yourself. 1) All three facts are on: which rules fire, and how many pieces of advice are on the advice line? 2) Only “It’s windy” is on. What does the system advise? Does that advice make sense to you; which rule is missing? 3) If R5 depended on “It’s windy” alone, what would be lost?

Live demo:



book.onuronder.com/d/en/41c4cadb5f

TECHNICAL DEPTH

Production rules take the IF-THEN form; an inference engine triggers rules whose conditions match facts in working memory. Forward chaining goes from facts to conclusions; backward chaining works from a goal toward evidence.

Expert systems (e.g. MYCIN, 1970s) used this architecture: a knowledge base (rules) plus an inference engine. In Figure 2.2 you will also see a fact produced by one rule trigger another rule (chaining). The limit: rules must be hand-written, and exceptions explode.

R5 is a chain: it triggers only if R1 fired (umbrella) and it is windy.

The engine of the demonstration makes two passes. The first pass evaluates only the rules that depend on facts and adds the derived facts they produce (R1 → umbrella) to working memory. The second pass evaluates every rule again against this extended memory. For longer chains the same loop repeats until no new fact is produced.

Rules say what to do. When there are thousands of options, though, which one should be tried first? Classical AI had a second tool for that: search.

2.4 Search and heuristic shortcuts

Finding the way out of a maze, picking a chess move, plotting a route through town... all versions of the same game: a crowd of possibilities lies before you, and you are hunting the one that reaches the goal.

The most patient method is trying every possibility one by one, but that can be terribly slow. A heuristic takes a shortcut: it guesses “which direction looks more promising?” Figure 2.3 races the two: the uninformed one scans everywhere; the informed one points its nose at the goal.

Margin note. Heuristics buy speed, but at a price: they can sometimes miss the best solution. They prefer “good enough” over “perfect.”

Figure 2.3 • Pathfinding: uninformed vs informed

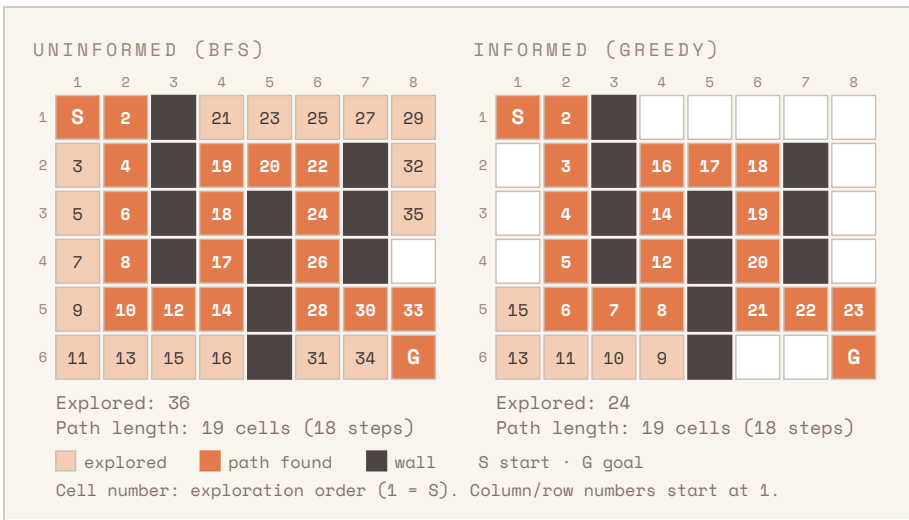


Figure 2.3

Setup. The figure shows a grid of 8 columns and 6 rows; 11 of the 48 cells are black walls. S (start) is top left, G (goal) bottom right. The

walls form three vertical barriers: four cells from the top in column 3, four from the bottom in column 5, three in the middle of column 7. So the path passes the first barrier at the bottom, the second at the top, the third at the bottom. Both panels show the same grid: uninformed search left, informed right. Light orange cells are the explored cells, dark orange cells the path; each cell's number is its exploration order.

Step by step. Uninformed search (BFS):

1. It starts at S and queues S's neighbors; first in, first out.
2. It spreads in rings: cells one step from S, then two, then three.
With no sense of direction, it scans the left corridor, the bottom row and the middle corridor alike.
3. It stops when G leaves the queue. The counter reads 36 cells explored, 36 of the 37 empty cells; only column 8, row 4 was never touched.
4. The path is read backward: 19 cells (S and G included), that is, 18 steps. This is the shortest path possible.

Informed search (greedy):

1. It scores every cell by its street distance to the goal: column difference plus row difference. For S, $7 + 5 = 12$.
2. It always takes the lowest-scoring cell from the queue. It goes down the left corridor, passes under the first barrier and reaches column 4.
3. Here it is fooled: the bottom row shares G's row, so it looks close. Because the wall in column 5, row 6 blocks the way to the right, it tries columns 4, 3, 2 and 1 of the bottom row, then column 1, row 5: five cells wasted.
4. Following the scores, it climbs column 4 to row 2, comes down column 6 and reaches G. The counter reads 24 cells.
5. The path: again 19 cells, 18 steps.

Method	Cells explored	Path (cells)
Uninformed (BFS)	36	19

The informed search visited a third fewer cells and still found the shortest path. That is luck, not a guarantee: a low-scoring dead end cost it five cells, and a more devious maze could mean a long detour. BFS's 36 cells are the price of the guarantee.

What is happening? You are racing two searchers. The uninformed one patiently scans in every direction; it finds the shortest path in the end but visits many squares. The informed one always runs toward the goal; it visits few squares but sometimes misses the shortest path. That is the trade-off between speed and guarantee.

Try it yourself. 1) Score two cells: column 4, row 2 and column 1, row 6. Which looks closer to the goal? Which one is on the path? 2) S scores 12, but the shortest path is 18 steps. Where does the difference come from? 3) Remove the wall in column 5, row 6: how many steps is the shortest path now?

Live demo:



book.onuronder.com/d/en/b1946ef054

TECHNICAL DEPTH

Many classical AI problems are modeled as state-space search. Uninformed search, such as breadth-first search (BFS), scans systematically without any direction information and guarantees the shortest path, but expands many nodes.

Informed search estimates closeness to the goal with a heuristic function $h(n)$; greedy best-first uses only h (fast but no optimality guarantee), while A^* balances optimality and efficiency with $g(n)+h(n)$ (if h is admissible). In Figure 2.3, compare how many cells BFS and heuristic search each explore.

BFS (uninformed) expands layer by layer with a FIFO queue and guarantees the shortest path on an equal-cost grid. Informed (greedy best-first) search picks the node minimizing the Manhattan distance $h(n)$ to the goal; it opens far fewer cells but does not guarantee the shortest path.

On this grid, with zero-based coordinates, $h(n) = |x - 7| + |y - 5|$. Results: BFS expanded 36 nodes, greedy search 24; both found the 18-step shortest path. Because h never exceeds the true distance at any node (it is admissible), A^* with the same h also finds the shortest path and usually expands fewer nodes than BFS.

Search assumed that the world is certain: a wall is a wall, and the goal stays where it is. Tomorrow's weather offers no such certainty, and knowledge that is not certain needs a different tool.

2.5 Handling uncertainty: probability and Markov

Strict rules stumble in the real world, because the world is uncertain. "If it rains, take an umbrella" is easy to say; but will it rain? Nobody knows for certain. At best, you can name the odds.

Classical AI found an elegant answer: move from state to state by probability. The Markov chain is the most famous example: a weather game played with a loaded die. Figure 2.4 holds a seven-day example: follow the weather there as it changes according to its odds.

Margin note. The Markov property: “the future depends only on the present; how you got here doesn’t matter.” It looks simple, yet it is everywhere, from weather to Google search.

Figure 2.4 • A weather Markov chain

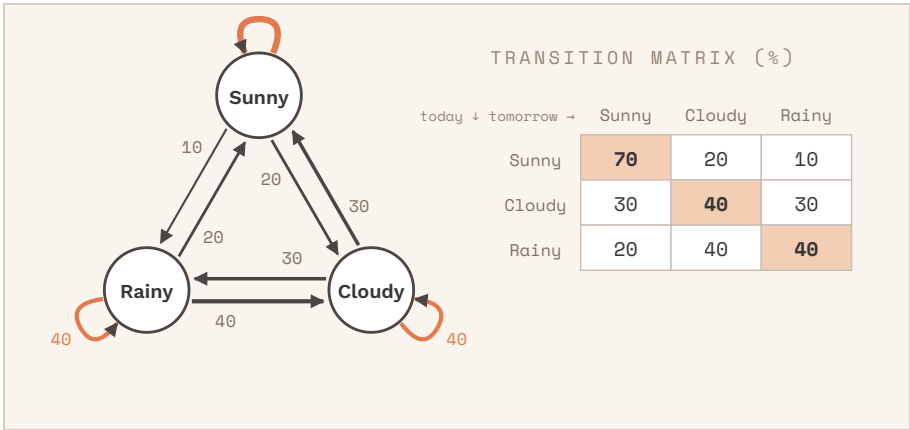


Figure 2.4

Setup. The figure shows three state boxes, Sunny, Cloudy and Rainy, joined by arrows; the number on each arrow is the chance of that move, in percent. Next to them the same numbers form the transition matrix: rows are today, columns are tomorrow.

today \ tomorrow	Sunny	Cloudy	Rainy
Sunny	70	20	10
Cloudy	30	40	30
Rainy	20	40	40

Step by step. First the loaded die. Say today is sunny; the first row reads 70 / 20 / 10. Picture a die with one hundred faces: 1 to 70

mean sun, 71 to 90 cloud, 91 to 100 rain. The machine draws a number from 1 to 100; the range it lands in is tomorrow's weather. The die is loaded: after sun, more sun has a 70 percent share; rain gets only 10.

Now a seven-day chain. The live demo draws the numbers at random; the seven draws here were chosen so that you can follow every step. In each row, find today's ranges, then place the number drawn.

Day	Today	Ranges (S / C / R)	Number drawn	Tomorrow
1	Sunny	1-70 / 71-90 / 91-100	42	Sunny
2	Sunny	1-70 / 71-90 / 91-100	83	Cloudy
3	Cloudy	1-30 / 31-70 / 71-100	55	Cloudy
4	Cloudy	1-30 / 31-70 / 71-100	91	Rainy
5	Rainy	1-20 / 21-60 / 61-100	77	Rainy
6	Rainy	1-20 / 21-60 / 61-100	12	Sunny
7	Sunny	1-70 / 71-90 / 91-100	64	Sunny

After seven days the tally is three sunny, two cloudy, two rainy: 43 / 29 / 29 percent. Moving to rain on day 4, the machine never looked at the three days before; "today is cloudy" and the number 91 were enough.

Finally, the long run. Seven days is not much; after hundreds of days the shares settle, and you can find them without any die. Let the long-run shares be S (sunny), C (cloudy), R (rainy). A sunny day arrives by three routes: after sun ($0.7 \cdot S$), after cloud ($0.3 \cdot C$), after rain ($0.2 \cdot R$). If the shares are steady, that sum must again be S:

$$S = 0.7 \cdot S + 0.3 \cdot C + 0.2 \cdot R \quad C = 0.2 \cdot S + 0.4 \cdot C + 0.4 \cdot R \quad R = 0.1 \cdot S + 0.3 \cdot C + 0.4 \cdot R, \text{ and } S + C + R = 1.$$

Solution: $S = 6/13$, $C = 4/13$, $R = 3/13$; about 46 / 31 / 23 percent. Check: $0.7 \cdot 6 + 0.3 \cdot 4 + 0.2 \cdot 3 = 4.2 + 1.2 + 0.6 = 6$. Run the live demo long enough and its bars sway around these numbers.

What is happening? Tomorrow's weather is predicted by looking only at today; yesterday doesn't matter. Each day rolls a die, but the die is loaded: after a sunny day, more sun is likely. As the days pile up, the shares settle into the same proportions every time.

Try it yourself. 1) Today is rainy and the number drawn is 35: what is tomorrow's weather? 2) Today is sunny. What is the chance of rain two days from now? Hint: work out tomorrow's three possibilities separately, then add them. 3) If the Sunny row became 90 / 5 / 5, which way would the long-run split move? Guess first, then check with the first equation.

Live demo:



book.onuronder.com/d/en/95270c7a90

TECHNICAL DEPTH

Probabilistic models are used to reason under uncertainty. A Markov chain is a stochastic process where the next state depends only on the current state (the Markov property: independence from history); transitions are defined by a probability matrix.

Over enough steps the distribution usually converges to a stationary distribution. The idea extends to hidden Markov models, PageRank and the Markov decision processes of reinforcement learning. In Figure 2.4, observe how the long-run distribution forms.

Each new day samples from P 's current row; the transition matrix P itself never changes.

The stationary distribution π solves $\pi P = \pi$ and $\sum \pi_i = 1$; for this P , $\pi = (6/13, 4/13, 3/13) \approx (0.462, 0.308, 0.231)$. From a sunny start the expected distribution evolves as follows: day 1 (0.70, 0.20, 0.10), day 2 (0.57, 0.26, 0.17), day 3 (0.51, 0.29, 0.20), day 5 (0.47, 0.30, 0.23). Convergence is largely complete within five days; the counter in the demonstration is a sample, so it fluctuates around these values.

Probability loosened the rules a little; but a human still writes the table, the rules and the chain by hand. How much of that load can a person carry? On that question AI researchers split into two camps.

2.6 Neats and scruffies

For years, AI researchers were split into two rival camps. The “neats” wanted every step proven with clean mathematics. The “scruffies” shrugged: “If it works, it’s good; we’ll find the theory later.”

This quarrel isn't only history; it continues today. Sort the statements below into the right camp. At the end it will be clear why classical AI hit its wall, and how that collision gave birth to the idea of machines that "learn."

Margin note. Classical AI's lesson: hand-writing all the world's rules is impossible. The solution? Instead of giving the machine rules, teach it to find them in data itself. That is the next chapter.

Figure 2.5 • Which approach?

#	Example	Neat	Scruffy
1	Prove everything with formal logic	☐	☐
2	Use working shortcuts, theorize later	☐	☐
3	Mathematical rigour is essential	☐	☐
4	The real world is messy; be flexible	☐	☐

Figure 2.5

Setup. The figure shows four statement cards, one under the other; next to each card are two boxes: Neat and Scruffy. The cards are short, like slogans; each is a single sentence that could have come out of a researcher's mouth.

Self-test. Pick the camp for each statement: Neat or Scruffy?

#	Example	Neat	Scruffy
1	Prove everything with formal logic	☐	☐
2	Use working shortcuts, theorize later	☐	☐
3	Mathematical rigour is essential	☐	☐
4	The real world is messy; be flexible	☐	☐

The words can mislead: "rigour" and "prove" come from one camp, "shortcuts" and "flexible" from the other, but the test is the attitude.

Does the speaker want it proven first, or working first? The first is Neat, the second Scruffy. Answers and reasons are at the back of the book.

What is happening? Two camps, two personalities: neats want every step proven with mathematics; scruffies say “make it work first, theory comes later.” Both turned out right in places; today’s AI is a blend of the two.

Try it yourself. 1) Sort the five methods of this chapter into the two camps: the knowledge chain, the expert system, uninformed search, informed search, the Markov chain. Which ones promise an exact result, and which settle for “good enough”? 2) The heuristic score (Figure 2.3) is which camp’s invention? The guarantee that A* adds to it moves it to which camp? 3) Find an example from your own life: a moment when you first made something work and thought about the theory later.

Live demo:



book.onuronder.com/d/en/2fd259af6d

TECHNICAL DEPTH

The “neat” vs “scruffy” distinction (attributed to Roger Schank) names a methodological tension in AI: between formal, provable, principled approaches (neats; logic and probability theory, for instance) and heuristic, engineering-driven, empirical ones (scruffies).

Classical symbolic AI hit two fundamental limits: the knowledge-acquisition bottleneck (hand-writing every rule doesn’t scale) and brittleness (collapsing on unforeseen cases). These limits accelerated the shift to statistical AI (Chapter 3), which learns from data instead of hand-coding knowledge.

Classical AI’s wall, in two words: the knowledge-acquisition bottleneck and brittleness.

The quarrel proved both camps partly right; the next chapter tells how. First, six questions.

2.7 Test yourself

Answers are at the back of the book.

1. How does classical (symbolic) AI represent knowledge?
 - a) Only with images
 - b) By learning from data
 - c) By random guessing
 - d) With explicit symbols and rules
2. What is “if it’s raining, take an umbrella”?
 - a) A dataset
 - b) A probability
 - c) A rule (IF-THEN)
 - d) A neural network

3. Key property of heuristic methods?
 - a) They speed up the search but don't guarantee the best
 - b) They guess at random
 - c) They always guarantee the optimal solution
 - d) They learn from data
4. In a Markov chain, the next state depends on?
 - a) Only the current state
 - b) Nothing
 - c) The future
 - d) The entire history
5. Classical AI's biggest challenge was?
 - a) Being too cheap
 - b) Hand-writing all rules, and the messiness of the world
 - c) Being too fast
 - d) Depending on the internet
6. What do "Neat" and "Scruffy" describe?
 - a) Two programming languages
 - b) Two robot types
 - c) Two computer brands
 - d) Two approaches in AI research

What to keep from this chapter

- Classical AI stores knowledge as explicit symbols and hand-written rules.
- An "is a" chain derives facts nobody wrote down, thanks to transitivity; for anything not in the chain it says "unknown."
- An expert system fires the IF-THEN rules that match the facts that are on; one rule's result can trigger another rule.
- Uninformed search guarantees the shortest path but visits many cells; informed search visits fewer but gives no guarantee.
- In a Markov chain tomorrow depends only on today, and in the long run the distribution settles at a fixed ratio.

- Neats want proof first, scruffies want results first; modern AI took something from each.
- Hand-writing rules does not scale and is brittle; the way out is learning the rules from data.

3

CHAPTER 3

How Machines Learn

3.1 How do machines learn?

How would you teach a child what a cat is? By writing rules, “four legs, whiskers, a tail...”? No; you point at cats, and the child works out the rest. The last chapter ran into the wall of rule-writing. So what if machines could learn the way children do?

That is the entire idea of machine learning: show plenty of examples and let it catch the pattern itself. This chapter goes behind the curtain: how is the data prepared, and how does a model grow a little more skillful with every try? You will see it all happen step by step.

***Margin note.** Classical AI: “Here are the rules, apply them.” Machine learning: “Here are the examples, find the rule yourself.” This small idea made all of modern AI possible.*

TECHNICAL DEPTH

Machine learning (ML) learns a function from statistical patterns in data instead of explicitly programmed rules. Where classical AI asks “how do we represent knowledge?” ML asks “how do we estimate the input-to-output mapping from data?”

The basic setup: a model maps inputs to outputs through its parameters (weights); a loss function measures how wrong the predictions are; an optimization method (e.g. gradient descent) updates the parameters to reduce the loss. This chapter builds that loop and the core task types.

Learning from examples starts with knowing what an example is. When you hand a machine an “example,” what are you giving it?

3.2 Features and labels

Picture a detective: clues in one hand, and the case’s final verdict in the other. A machine learning from examples looks at the same pair. The clues are called features: measurable facts describing an example. The verdict is called the label. For an email the clues might be “word count,” “has a link,” “mentions free”; the label is “Spam” or “Normal.”

Reading hundreds of solved cases, the model learns which clue travels with which verdict. Each example in Figure 3.1 shows its clues and its correct answer.

***Margin note.** “Garbage in, garbage out” applies here too: if the features are poor, even the best model can’t learn. Understanding the data often matters more than choosing the model.*

Figure 3.1 • See the features and the label

#	Email	FEATURES (INPUT)			LABEL (OUTPUT)
		mentions “free”	link / password request	urgency language	Label
1	“You won a free iPhone! Click now”	✓	✓	✓	Spam
2	“Meeting tomorrow at 10:00”	○	○	○	Normal
3	“URGENT: your account will close, enter your password”	○	✓	✓	Spam
4	“I’ve attached the draft report”	○	○	○	Normal

✓ has cue ○ no cue

Figure 3.1

Setup. The figure shows four short emails and three cues for each one: does it mention “free,” does it ask for a link or a password, does

it use urgent language. A filled mark (✓) means the cue is present; an open circle (○) means it is absent. The three cue columns are headed “Features (input).” The rightmost column, headed “Label (output),” gives the correct answer: Spam or Normal. All four examples are in the table below.

Step by step.

#	Email	mentions “free”	link / password request	urgency language	Label
1	“You won a free iPhone! Click now”	✓	✓	✓	Spam
2	“Meeting tomorrow at 10:00”	○	○	○	Normal
3	“URGENT: your account will close, enter your password”	○	✓	✓	Spam
4	“I’ve attached the draft report”	○	○	○	Normal

1. Read the table row by row. Each row is one example: the three cue columns are that example’s features, the last column is its label. The model sees only this pair, never the email itself.
2. Email 1 carries all three cues, and its label is Spam. Email 3 never says “free,” but it asks for a password and hurries you. It is Spam too.
3. Emails 2 and 4 carry none of the three cues. Both are Normal.
4. Then compare the cue columns with the label. When the cue count is zero, the label is always Normal; when it is two or three, always Spam. “Free” alone does not decide: email 3 came out Spam without it. The link or password request and the urgent language, though, are present in both Spam examples.
5. Nobody wrote this relation; you read it off the table. The model does the same, only with thousands of rows instead of four. The “two cues means spam” rule drawn from four rows is a provisional guess; a fifth example could break it.

What is happening? An email’s clues (features) and its correct answer (label) sit side by side: mentions of “free,” a link request, urgent language... Seeing many examples, the machine learns by itself which clues go with “Spam”; nobody writes the rule, it draws it from the examples.

Try it yourself. 1) Mark the three cues of the email “Your password has expired; if you do not renew it today, your account will be deleted.” By the relation in the table, what is its label? 2) The email “Free coffee, let’s meet in the kitchen tomorrow at noon” is in fact Normal. If this row is added to the table, should the model trust the “free” cue more, or less? 3) Write a rule that agrees with all four rows. Does your rule also work on the email in question 2?

Live demo:



book.onuronder.com/d/en/9b4cdaf0e7

TECHNICAL DEPTH

In supervised learning, each example is a pair of a feature vector x and a label y . Features can be numeric or categorical; the model tries to learn the mapping $f(x) \approx y$. Good feature engineering is one of the most decisive steps for performance in classical ML.

The label’s type sets the task: a categorical label $\rightarrow$ classification, a continuous (numeric) label $\rightarrow$ regression. Figure 3.1 builds intuition for telling spam from normal email with simple features.

Feature vector x = [mentions “free”; has a link; urgency language], label y = “Spam.” In supervised learning the model tries to estimate the mapping $f(x) \approx y$ from these (x, y) pairs, i.e. a decision rule like $P(\text{spam} \mid x)$.

In the spam example every email came with its correct answer. Can a machine learn without an answer key, or with no teacher at all?

3.3 Three kinds of learning

The school of machines has three kinds of teacher. The first walks around with an answer key: “this is spam, this isn’t” (supervised). The second gives no answers at all: “you sort it out,” and the machine groups the data itself (unsupervised). The third never lectures; the machine enters the game and collects rewards and penalties as it tries (reinforcement).

Now take each task below to the right teacher. For each, ask: Is there an answer key, is the machine sorting alone, or is it learning by trying?

Margin note. Hint: “Are correct answers given?” → Supervised. “Is the machine grouping on its own?” → Unsupervised. “Is it trying things and collecting reward?” → Reinforcement.

Figure 3.2 • Which kind of learning?

#	Example	Supervised	Unsupervised	Reinforcement
1	Telling cats from dogs using labeled photos	☐	☐	☐
2	Grouping customers by similarity	☐	☐	☐
3	A robot learning to walk by trial and error	☐	☐	☐
4	Predicting house prices from past (labeled) data	☐	☐	☐
5	Learning a high score by playing a game	☐	☐	☐
6	Clustering unlabeled news by topic	☐	☐	☐

Figure 3.2

Setup. The figure shows six tasks and three columns: Supervised, Unsupervised, Reinforcement. Next to each task sit three empty boxes, one per column; you mark the right one in pencil. There is no worked solution; the answers, with their reasons, are at the back of the book.

Self-test. For each task, ask three questions in order. Are the correct answers given up front? Is the machine grouping the data on its own? Or is it trying things and collecting reward? Then mark one of the three boxes.

#	Example	Supervised	Unsupervised	Reinforcement
1	Telling cats from dogs using labeled photos	☐	☐	☐
2	Grouping customers by similarity	☐	☐	☐
3	A robot learning to walk by trial and error	☐	☐	☐
4	Predicting house prices from past (labeled) data	☐	☐	☐
5	Learning a high score by playing a game	☐	☐	☐
6	Clustering unlabeled news by topic	☐	☐	☐

Once all six are marked, look back at the list. Three of the tasks say “labeled” or “unlabeled” outright; in the others, what told you whether a label exists? And tasks 3 and 5 have something in common: where does the correct answer come from in each? Each column gets two tasks, and that is by design.

What is happening? There are three ways to teach a machine: show it the right answers (supervised), give no answers and let it group things itself (unsupervised), or let it try and collect reward/penalty (reinforcement). Place each task on the right path.

Try it yourself. 1) Put these three tasks into the same three columns. A translation program learning from millions of translations made by humans; a chess program growing stronger by playing against itself; a supermarket finding “bought together” product groups from its receipts. 2) Turn the three questions of the self-test into a decision tree: which question comes first, and at which answer do you stop? 3) Write a task that fits two columns at once, and use it to explain the sentence “these boundaries are not sharp” from the technical text.

Live demo:



book.onuronder.com/d/en/79ea5021f2

TECHNICAL DEPTH

Supervised learning learns $f(x) \approx y$ from labeled (x, y) pairs (classification/regression). Unsupervised learning extracts structure from x alone (clustering, dimensionality reduction). Reinforcement learning is an agent learning a policy that maximizes a reward signal in an environment.

These boundaries are not sharp: semi-supervised and self-supervised learning sit in between. Modern large language models are largely trained with self-supervised pretraining, generating the labels from the data itself. Sort the tasks of Figure 3.2 into the three core types.

Two tasks landed in the Supervised column: telling cats from dogs and predicting house prices. Both come with labels, yet one asks for a category and the other for a number. What does that difference change?

3.4 Classification and regression

Supervised learning asks two basic questions. One says “how much?”: What does this house cost? That is regression; its answer is a number. The other says “which one?”: Is this email spam or not? That is classification; its answer is a category.

Try both yourself below. In regression you’ll find the “best line” through the middle of the points; in classification you’ll draw a border between the two groups.

Margin note. Simple rule: if the output is a number it’s regression, if it’s a label it’s classification. “How much?” is regression, “Which one?” is classification.

Figure 3.3 • Two core tasks

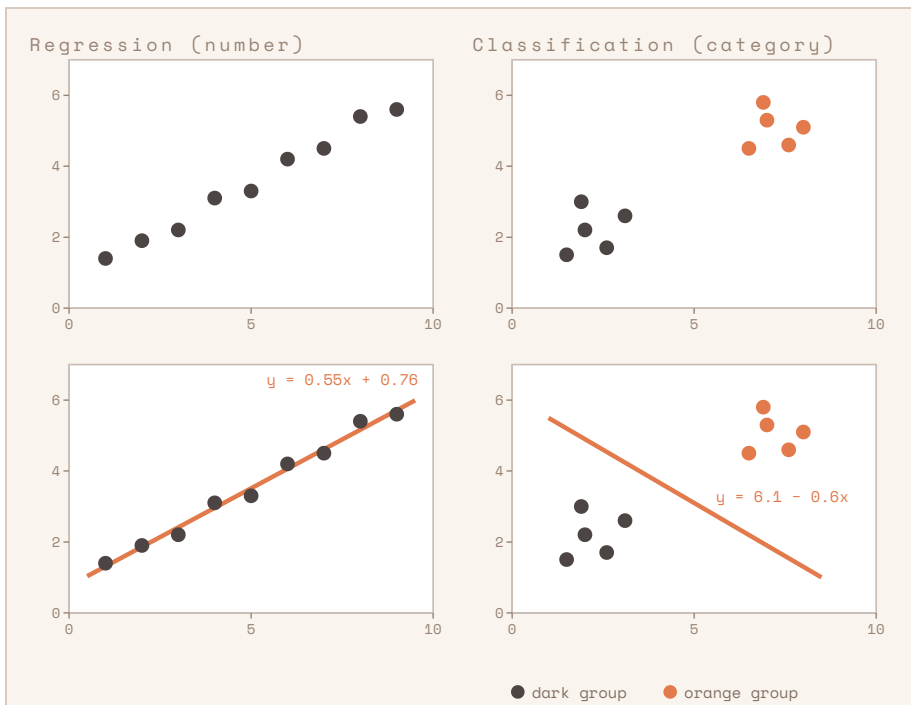


Figure 3.3

Setup. The figure shows the two tasks side by side. On the left, “Regression (number)”: nine points and the best line fitted to them. On the right, “Classification (category)”: five green points, five orange points and the boundary drawn between them. The top square of each panel shows the raw data; the bottom square shows the line or the boundary drawn in.

Step by step.

1. The regression data is nine points: (1, 1.4), (2, 1.9), (3, 2.2), (4, 3.1), (5, 3.3), (6, 4.2), (7, 4.5), (8, 5.4), (9, 5.6). As x grows, y grows, but the points do not sit on a single line.
2. The least-squares line comes from two means: the x values average 5, the y values 3.51. Slope $m = 33 / 60 = 0.55$; intercept $b = 3.51 - 0.55 \cdot 5 = 0.76$. The line: $y = 0.55x + 0.76$.
3. How close is the line to each point? At $x = 3$ the line says 2.41, the point is 2.2; the gap is -0.21 . At $x = 8$ the line says 5.16, the point is 5.4; the gap is $+0.24$. None of the nine gaps exceeds 0.25. The sum of the squared gaps is 0.22.
4. Why is this sum the measure of “best”? Take a line drawn by eye, $y = 0.5x + 1$: the same sum rises to 0.37. Among all possible lines, the least-squares line is the one that makes this sum smallest.
5. The classification data is two groups. Green: (1.5, 1.5), (2, 2.2), (2.6, 1.7), (3.1, 2.6), (1.9, 3). Orange: (6.5, 4.5), (7, 5.3), (7.6, 4.6), (6.9, 5.8), (8, 5.1).
6. The boundary is the line through the points (1, 5.5) and (8.5, 1): $y = 6.1 - 0.6x$. Check: at $x = 3.1$ the boundary says 4.24; the green point at 2.6 is below it. At $x = 6.5$ it says 2.2; the orange point at 4.5 is above it. All ten points are on the right side.
7. Two answers, two kinds. Regression gives a number: for $x = 10$, $0.55 \cdot 10 + 0.76 = 6.26$. Classification gives a side: below the boundary is green, above it is orange.

What is happening? Two core jobs. Regression predicts a number: the “best line” runs through the middle of the points, staying as close to all of them as possible. Classification draws a border separating one group from the other.

Try it yourself. 1) By the line $y = 0.55x + 0.76$, what is the prediction for $x = 6.5$? 2) On which side of the boundary does the point (4.5, 3.5) fall: green or orange? And (5, 3)? 3) Add a far-off point such as (9, 9) to the regression data. Does the slope go up or down? Is it a problem that the line chases a single point?

Live demo:



book.onuronder.com/d/en/d10b8aa300

TECHNICAL DEPTH

Regression predicts a continuous target; its simplest form is the least-squares line minimizing the sum of squared errors. Classification predicts a categorical target and learns a decision boundary separating the classes (e.g. logistic regression, support vector machines).

Figure 3.3 fits a least-squares line for regression and shows a linear decision boundary separating two clusters for classification. In reality, decision boundaries need not be linear.

The closed form of the least-squares line: $m = \Sigma(x_i - \bar{x})(y_i - \bar{y}) / \Sigma(x_i - \bar{x})^2$, $b = \bar{y} - m\bar{x}$. For the data of Figure 3.3 the numerator is 33 and the denominator 60.

So far every point came with its label or its number. Take the labels away and nothing is left but the points. Can the machine still find an order in them?

3.5 Clustering and anomalies

Imagine being handed a huge box of buttons and told “sort these.” Nobody says which belongs where; still, you put like with like. That is

elf. And there are always a few odd buttons that fit nowhere; anomaly detection catches those. It is how banks catch a suspicious transaction.

The points in Figure 3.4 carry no labels at all. The machine sorts them into two clusters by similarity, and the odd one out gives itself away.

Margin note. What unsupervised learning can do: without anyone saying “these belong together,” the machine finds the structure itself. Banks’ fraud detection relies heavily on spotting anomalies.

Figure 3.4 • Group unlabeled data

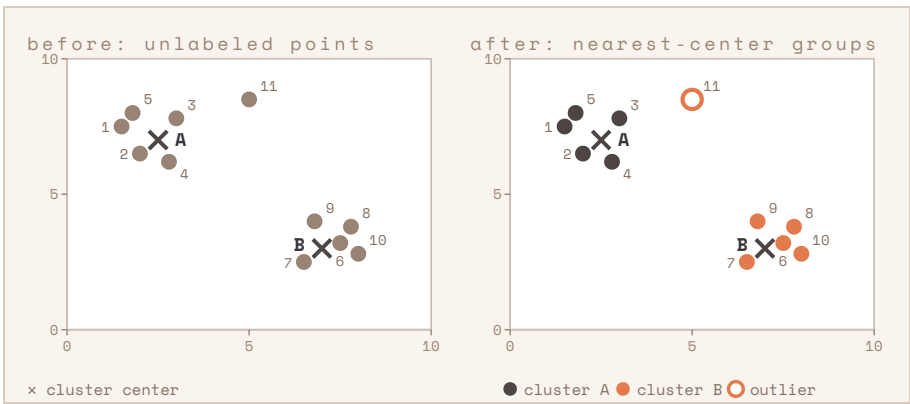


Figure 3.4

Setup. The figure shows eleven gray points and two X marks. The X marks are the cluster centers: center A at the top left (2.5, 7), center B at the bottom right (7, 3). The left square, “before: unlabeled points,” is the state before grouping; every point is the same gray. In the right square, “after: nearest-center groups,” each point has taken the color of its nearest center, and one point is marked as the “outlier.”

Step by step. Grouping rests on a single question: which center is this point closer to? The distance comes from Pythagoras: $\sqrt{((x - x_m)^2 + (y - y_m)^2)}$.

#	Point	Distance to A	Distance to B	Cluster
1	(1.5, 7.5)	1.12	7.11	A
2	(2, 6.5)	0.71	6.10	A
3	(3, 7.8)	0.94	6.25	A
4	(2.8, 6.2)	0.85	5.28	A
5	(1.8, 8)	1.22	7.21	A
6	(7.5, 3.2)	6.28	0.54	B
7	(6.5, 2.5)	6.02	0.71	B
8	(7.8, 3.8)	6.19	1.13	B
9	(6.8, 4)	5.24	1.02	B
10	(8, 2.8)	6.92	1.02	B
11	(5, 8.5)	2.92	5.85	outlier

1. Point 1, (1.5, 7.5): to A, $\sqrt{(1^2 + 0.5^2)} = 1.12$; to B, $\sqrt{(5.5^2 + 4.5^2)} = 7.11$. A is far closer; the point goes to A.
2. Point 9, (6.8, 4): to A, $\sqrt{(4.3^2 + 3^2)} = 5.24$; to B, $\sqrt{(0.2^2 + 1^2)} = 1.02$. It goes to B.
3. Point 4, (2.8, 6.2): 0.85 against 5.28, A again. Do the same calculation for the first ten points; in every one, the two distances differ by a factor of at least five. The decision is never hard.
4. Point 11, (5, 8.5), is different. Its nearest center is A, but the distance is 2.92. The other members of A are at most 1.22 from their center; this point is more than twice as far. It is 5.85 from B. It fits neither cluster: an outlier.
5. That was a single round. Real k-means now updates the centers: the new center of A is the mean of points 1 to 5, (2.22, 7.2). That is only 0.34 from the old center; the clusters have already settled.

What is happening? The points carry no labels at all. The machine ties each point to its nearest center (X); similar ones end up in the same cluster. The lone point far from both clusters gets flagged as

an “outlier” (an odd example); banks catch fraud in much the same way.

Try it yourself. 1) Compute the new center of cluster B: the mean x and mean y of points 6 to 10. How far did it move from the old center (7, 3)? 2) If the point (4.5, 5.5) were added to the data, which center would it go to? Looking at its distance, would you count it as an outlier? 3) The clustering used no labels at all. The “outlier” decision, though, rests on a number and a threshold: which ones, and who chose the threshold?

Live demo:



book.onuronder.com/d/en/afcbfd2bb7

TECHNICAL DEPTH

Clustering discovers groups by similarity without labels; methods like k -means assign points to the nearest centroid and update the centroids. Anomaly (outlier) detection identifies examples that deviate markedly from the majority distribution.

Figure 3.4 shows k -means’ “assignment” step by assigning points to the nearer of two fixed centroids; it also highlights an outlier far from both clusters. Real k -means updates the centroids iteratively until convergence.

The two steps as formulas: assignment $c(i) = \operatorname{argmin}_k \|x_i - \mu_k\|^2$, update $\mu_k \leftarrow$ the mean of the points assigned to cluster k . In Figure 3.4 one update moves center A from (2.5, 7) to (2.22, 7.2) and center B from (7, 3) to (7.32, 3.26); a second assignment round changes no point, and the algorithm has converged.

In clustering the center moved once and the job was done. How does a model with millions of parameters take its “correct a little” step?

3.6 How a model improves: loss and gradient descent

How does a model get “better”? First you measure how wrong it is; that is the loss. Now picture yourself in a fog-covered valley. The bigger the loss, the higher up you stand; your goal is the valley floor. The fog hides the path, but one thing you can still feel: the slope under your feet.

Gradient descent is that walk: feel the slope, take a small step downhill, repeat. In the left panel of Figure 3.5 the ball descends step by step and the loss melts away. And if your stride is too long? The right panel shows what happens.

***Margin note.** Learning is this: ask “how wrong am I?”, correct a little, repeat, millions of times over. Nearly all modern AI, neural networks included, is trained this way.*

Figure 3.5 • Descending the loss valley

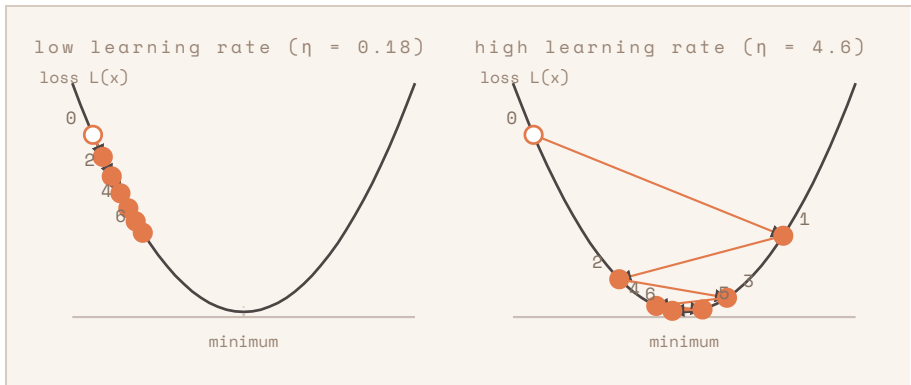


Figure 3.5

Setup. The valley is a parabola: $L(x) = 0.18 \cdot (x - 5)^2 + 0.1$. The horizontal axis is the model’s single parameter x ; the vertical axis is the loss. The floor is at $x = 5$, where the loss is 0.1. The ball starts at $x_0 = 0.6$, on the left slope, where the loss is 3.58. The left panel shows six

steps at the low learning rate ($\eta = 0.18$), the right panel six steps at the high rate ($\eta = 4.6$).

Step by step. The rule is the same at every step: $x \leftarrow x - \eta \cdot L'(x)$. The slope is $L'(x) = 0.36 \cdot (x - 5)$: negative to the left of the floor, positive to the right. Subtracting a negative slope makes x larger, so the ball moves right, that is, downhill.

step	x ($\eta = 0.18$)	$L(x)$	x ($\eta = 4.6$)	$L(x)$
0	0.60	3.58	0.60	3.58
1	0.89	3.15	7.89	1.60
2	1.15	2.77	3.11	0.75
3	1.40	2.43	6.24	0.38
4	1.63	2.14	4.19	0.22
5	1.85	1.88	5.53	0.15
6	2.06	1.66	4.65	0.12

1. Low rate, step 1: the slope is $0.36 \cdot (0.6 - 5) = -1.58$. New $x = 0.6 - 0.18 \cdot (-1.584) = 0.6 + 0.285 \approx 0.89$. The loss fell from 3.58 to 3.15.
2. Low rate, steps 2 to 6: each step is a little shorter than the last: 0.29, 0.26, 0.25, 0.23, 0.22, 0.21. The slope shrinks as the floor nears. After six steps $x = 2.06$, loss 1.66. The floor is still far off; bringing x within 0.1 of it takes close to sixty steps. Safe, but slow.
3. High rate, step 1: the same slope, but the step is $4.6 \cdot 1.584 \approx 7.29$ units. The ball misses the floor and flies up the far slope to $x = 7.89$. The loss still fell: 1.60.
4. High rate, steps 2 to 6: the slope is now positive (+1.04), so the ball jumps left, to 3.11. Then 6.24, 4.19, 5.53, 4.65. Once right of the floor, once left: oscillation. Each jump is 0.66 times the one before, so the swings die down and the ball still closes in on the floor. Loss: 0.12.
5. Compare: after six steps the low rate stands at 1.66, the high rate at 0.12. In this valley the big stride won, but it was close to the

edge: had $\eta \cdot 0.36$ passed 2, each jump would have grown and the ball would have flown out of the valley.

What is happening? Improving a model is like descending to a valley floor: the bigger the “loss,” the higher up you are. Each step nudges the ball downhill and the loss shrinks. But if the step is too big (a high learning rate), the ball overshoots the bottom and flies up the far slope; the step size matters as much as the direction.

Try it yourself. 1) Work out step 7 at the low rate: the slope at $x = 2.06$, the new x and the new loss. 2) Take two steps with $\eta = 6$ ($\eta \cdot 0.36 = 2.16$). Is the ball getting closer to the floor or farther away? 3) Is there a learning rate that reaches the floor of this valley in a single step? Hint: require $x - \eta \cdot 0.36 \cdot (x - 5) = 5$.

Live demo:



book.onuronder.com/d/en/a6d0d3d672

TECHNICAL DEPTH

Training means adjusting the parameters to minimize a loss function $L(\theta)$. Gradient descent moves against the gradient at every step: $\theta \leftarrow \theta - \eta \cdot \nabla L(\theta)$, where η is the learning rate.

The learning rate is the most delicate dial: too small and convergence is slow; too large and it can oscillate around the minimum or diverge. Figure 3.5 shows descent on a convex loss curve and the overshoot a large learning rate causes. In practice surfaces aren't convex, and stochastic gradient descent is the norm.

In this quadratic valley the step rule is linear: $x_{t+1} - 5 = (1 - 0.36 \cdot \eta) \cdot (x_t - 5)$. The factor is 0.935 at $\eta = 0.18$ (one-directional, slow), -0.656 at $\eta = 4.6$ (damped oscillation), and its absolute value exceeds 1 for $\eta > 5.56$ (divergence). Every row of the table follows from the previous one through this single factor.

The loss drops toward zero and the model looks better and better. Does zero error on the training data mean zero error on new data, though?

3.7 Overfitting and ensembles

You know the class memorizer: word-perfect on every past exam question, lost the moment the question changes a little. Models sometimes learn “too well” in the same way: they memorize the training examples and flunk the new ones. That is overfitting. The opposite exists too: a model too simple to catch the pattern at all (underfitting). A good model stands in between; it doesn't memorize, it grasps.

Now fit three models to the same data, like three students: the lazy one, the balanced one and the memorizer. Which one, do you think, will answer a question it has never seen?

Margin note. Memorizing isn't learning. A student who only memorizes past exam questions flunks a new one. A good model learns the pattern underneath, not the examples themselves.

Figure 3.6 • Same data, three models

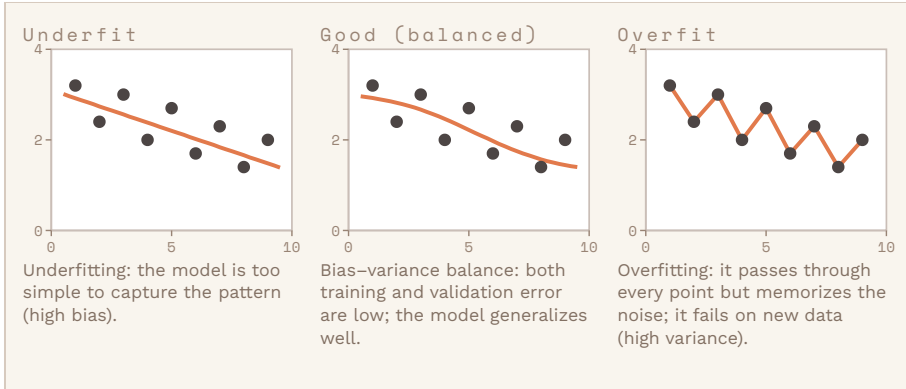


Figure 3.6

Setup. All three panels hold the same nine points: (1, 3.2), (2, 2.4), (3, 3.0), (4, 2.0), (5, 2.7), (6, 1.7), (7, 2.3), (8, 1.4), (9, 2.0). The overall trend is downward, but the points jump up and down from one x to the next; that is measurement noise. The left panel, “Underfit,” is a straight line; the middle, “Good (balanced),” a gently waving curve; the right, “Overfit,” a broken line joining the nine points one by one. Under each panel stands that model’s verdict.

Step by step.

1. Underfit: the line $y = 3.1 - 0.18x$. At $x = 5$ it says 2.2 against a point at 2.7; the gap is 0.5. At $x = 9$ it says 1.48 against 2.0. The line runs above one point and below the next; it never follows the jolts. Verdict: “Underfitting: the model is too simple to capture the pattern (high bias).”
2. Good (balanced): the curve $y = 3.0 - 0.16x + 0.15 \cdot \sin(0.6x)$. It says 2.22 at $x = 5$ and 1.44 at $x = 9$. It does not chase the zigzag; it carries only the downward trend and a slight wave. Verdict: “Bias–

variance balance: both training and validation error are low; the model generalizes well.”

3. Overfit: the broken line passes through all nine points; the training error is zero. But its shape gives it away: from 5 to 6 it drops by 1.0, from 6 to 7 it rises by 0.6. Those ups and downs are noise, not pattern; a fresh measurement would not repeat them. Verdict: “Overfitting: it passes through every point but memorizes the noise; it fails on new data (high variance).”
4. The exam: hide points 5 and 7 and draw the same broken line through the remaining seven. At $x = 5$ the line joins (4, 2.0) to (6, 1.7) and says 1.85; the truth is 2.7, an error of 0.85. At $x = 7$ it says 1.55; the truth is 2.3, an error of 0.75. The simple line errs by 0.5 and 0.46 at the same two points. On an unseen question the memorizer does worse than the lazy one.
5. Training error on its own misleads. A model has to be tested on data it has never seen; that is what “validation” means.

What is happening? Three different models are fitted to the same data. The too-simple one misses the pattern (underfitting); the too-complex one memorizes every point but stumbles on new data (overfitting). The best is right in the middle: the model that also predicts examples it has never seen.

Try it yourself. 1) What do the three models say for $x = 10$? Look closely at the broken line. 2) Repeat the hide-and-test exam with points 2 and 8: what errors do the broken line and the straight line make at those two points? 3) The model that passes through all nine points boasts of “zero error.” What does that number prove, and what does it not prove?

Live demo:



book.onuronder.com/d/en/d09eaed1a3

TECHNICAL DEPTH

Overfitting is when a model also learns the noise in the training data and loses performance on unseen data; underfitting is when it is too simple to capture the pattern. These are the two ends of the bias–variance tradeoff, usually managed with train/validation splits, regularization and cross-validation.

Ensemble learning combines the predictions of many models (voting, bagging, boosting) for better, more stable results than any single model; random forests and gradient boosting are the best-known examples.

The hide-and-test exam of Figure 3.6 is a single-fold train/validation split: two points form the validation set, seven the training set. Cross-validation repeats this nine times, hiding each point in turn, and averages the errors.

One idea ran through this chapter: grasp the pattern instead of memorizing the examples. The six questions below check whether it stuck.

3.8 Test yourself

Answers are at the back of the book.

1. What is the core difference between ML and classical AI?
 - a) It never makes mistakes
 - b) It runs faster
 - c) It needs the internet
 - d) It learns from data instead of hand-written rules
2. What is an example's “correct answer” called?
 - a) Feature
 - b) Gradient

- c) Label
- d) Model

3. Which task groups unlabeled data?

- a) Regression
- b) Clustering (unsupervised)
- c) Classification
- d) Reinforcement

4. What does gradient descent do?

- a) Generates labels
- b) Slows the model
- c) Updates parameters step by step to reduce loss
- d) Deletes data

5. What is overfitting?

- a) Memorizing training data and failing on new data
- b) Compressing data
- c) Learning too fast
- d) Not learning at all

6. How does an agent learn in reinforcement learning?

- a) From labeled examples
- b) By memorizing rules
- c) By clustering data
- d) By reward and penalty, trial and error

What to keep from this chapter

- Machine learning writes no rules; it draws the rule from the examples itself.
- Every example has two parts: the features that describe it and the correct answer, the label.
- With labels it is supervised learning, without them unsupervised, and with rewards reinforcement learning.
- “How much?” is regression, “which one?” is classification.

- Clustering groups unlabeled data by similarity; a point that fits no group is an outlier.
- Gradient descent measures the loss, reads the slope, takes a small step and repeats this millions of times.
- A model that memorizes the training data flunks new data; a good model grasps the pattern.

4

CHAPTER 4

The Artificial Brain

4.1 The artificial brain: deep learning

Billions of tiny messengers live inside your head: neurons. Each listens to its neighbors' whispers; when the whispers grow loud enough, it shouts and passes the news along. Researchers looked at this simple game and wondered: What if we built a rough mathematical imitation of it?

Artificial neural networks were born from that question. Stack “artificial neurons” into layers and out comes a powerful learning machine. As the layers multiply, the name changes too: deep learning. This chapter sits beside a single messenger first, then moves to a whole network, and on to the special networks that handle images and sequences.

***Margin note.** An “artificial neuron” is not a real copy of the brain; it is a very crude mathematical analogy. A single neuron does little; the patterns formed by millions of them together are what count.*

TECHNICAL DEPTH

Artificial neural networks are only loosely inspired by biological neurons; at heart they are stacks of nonlinear transformations arranged in layers. Each artificial neuron adds a bias to a weighted sum of its inputs and passes the result through an activation function.

Deep learning is the stacking of many hidden layers; this depth lets the network learn increasingly abstract representations from raw data (edge → shape → object). Without activation functions, stacked linear layers collapse into a single linear function; nonlinearity is what makes depth meaningful. The field entered its modern era with AlexNet's leap on ImageNet in 2012.

The smallest piece of this whole structure is a single artificial neuron. What does it compute on its own, and how does it turn three numbers into a decision?

4.2 A single artificial neuron

Think of the artificial neuron as a gatekeeper. A few messages arrive at its door; the keeper gives each a level of importance (called a weight), adds them up, then adds its own temperament on top (the bias). If the total clears a threshold, it rings the bell; that decision is what “activation” means.

In Figure 4.1 the inputs are strengthened and weakened in turn. Follow how the total changes and when the keeper rings the bell; that moment is the neuron “firing.”

Margin note. All a neuron does is “weigh the inputs, sum them, pass them through a threshold.” Repeated millions of times, an operation this simple can recognize faces and write text.

Figure 4.1 • Run the neuron

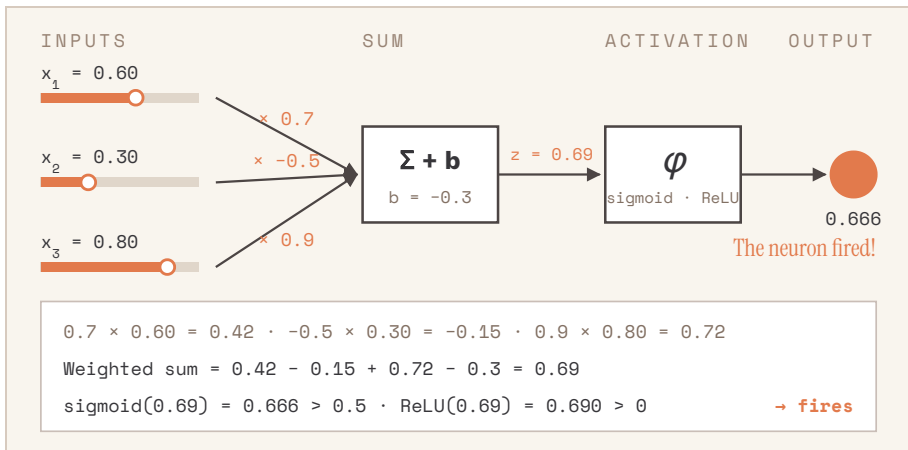


Figure 4.1

Setup. The figure shows three inputs: x_1 , x_2 and x_3 . Each carries a value between 0 and 1; at the start, 0.60, 0.30 and 0.80. Next to each input sits a fixed weight: $w = [0.7, -0.5, 0.9]$. The box in the middle adds them up and adds the bias $b = -0.3$ on top. The box on the right applies the activation, sigmoid or ReLU. When the total clears the threshold, the panel on the right reads “The neuron fired!”; in the table you change the inputs yourself.

Step by step. Work through the keeper’s arithmetic with the opening values.

1. Multiply each input by its weight: $0.7 \times 0.60 = 0.42$; $-0.5 \times 0.30 = -0.15$; $0.9 \times 0.80 = 0.72$.
2. Add the three: $0.42 - 0.15 + 0.72 = 0.99$. Add the bias: $0.99 - 0.3 = 0.69$. The line “Weighted sum = 0.69” in the figure is this number.
3. With sigmoid selected, the output is $1 / (1 + e^{-0.69}) = 0.666$. That is above the 0.5 threshold, so the panel on the right says “The neuron fired!”
4. With ReLU, the output is $\max(0, 0.69) = 0.690$. It is above zero, so the neuron fires again.
5. Raise x_2 to 1.00. Its weight is negative, so the sum drops: $0.42 - 0.50 + 0.72 - 0.3 = 0.34$. Sigmoid gives 0.584; the neuron still fires, but more weakly.
6. Put x_2 back to 0.30 and set x_3 to zero. The sum becomes $0.42 - 0.15 + 0 - 0.3 = -0.03$. Sigmoid gives 0.493, ReLU 0.000; in both cases the neuron stays silent.

x_1	x_2	x_3	Weighted sum	Sigmoid	ReLU	State
0.60	0.30	0.80	0.69	0.666	0.690	fires
0.60	1.00	0.80	0.34	0.584	0.340	fires
0.60	0.30	0.00	-0.03	0.493	0.000	silent
0.00	0.00	0.00	-0.30	0.426	0.000	silent

In the last row every input is zero, and still the sum is -0.30 . That is the bias at work: by temperament, the keeper starts out closed. x_2

always pulls the sum down; x_1 and x_3 push it up, and x_3 pushes hardest because its weight is largest.

What is happening? A neuron does something simple: it multiplies each input by an “importance weight,” adds them up, then adds a small threshold value (the bias). If the total is big enough, the neuron “fires,” giving a strong output. Change the inputs and the sum and the output change together.

Try it yourself. 1) Set all three inputs to 1.00. What are the weighted sum and the sigmoid output; does the neuron fire? 2) Only $x_3 = 1.00$, the others 0. Compute the sum and the ReLU output. 3) Only $x_2 = 1.00$, the others 0. Does the sigmoid output pass 0.5?

Live demo:



book.onuronder.com/d/en/239357fa8c

TECHNICAL DEPTH

An artificial neuron computes $z = \sum w_i x_i + b$, then applies an activation function: $a = \phi(z)$. Common choices are sigmoid (0–1), $\tanh(-1-1)$ and $\text{ReLU} = \max(0, z)$. The weights tune each input’s importance and the bias tunes the threshold; both are learned in training.

The activation’s role is decisive: without added nonlinearity, the network, however many layers, would equal a single linear transformation. ReLU has become the de facto standard in deep networks thanks to its simplicity and its preservation of gradient flow.

The sigmoid in the figure is $\sigma(z) = 1 / (1 + e^{-z})$; at $z = 0$ it gives 0.5, which is why the firing threshold in the figure is 0.5. For ReLU the threshold is $z > 0$.

A single neuron can draw only one line through the space of its inputs. When dozens are stacked in layers, how does the signal travel through them?

4.3 Layers and the forward pass

One gatekeeper alone can't achieve much. But house the keepers on the floors of a building and everything changes. News flows from the ground floor to the top; the "hidden" floors in between refine the raw news a little more at every stop.

News flowing from entrance to exit is called the forward pass. Figure 4.2 shows two input patterns; follow the signal as it climbs floor by floor and see which neurons light up.

Margin note. The secret of depth: early layers learn simple features (edges), later layers learn their combinations (shapes, objects). No one programs this by hand; the network discovers it itself.

Figure 4.2 • A live neural network (forward pass)

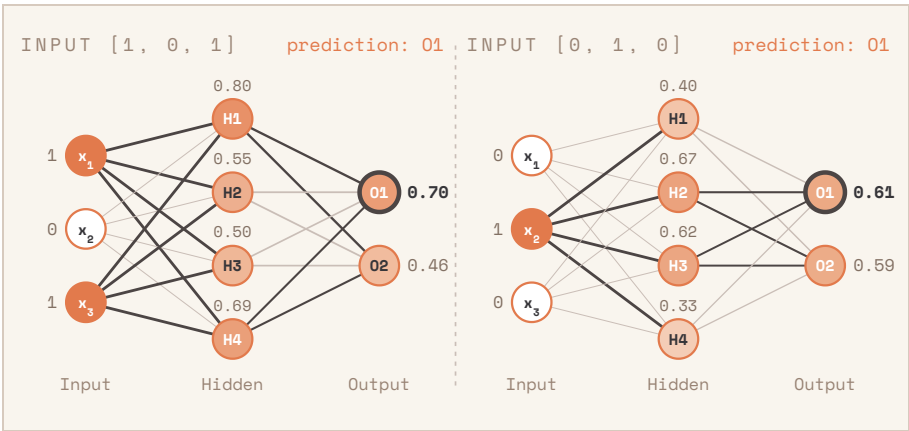


Figure 4.2

Setup. The figure has two panels, each with three layers. On the left of a panel are three input boxes, each on (1) or off (0). In the middle

sit four hidden neurons, H1 to H4; on the right, two output neurons, O1 and O2. Every connection has a fixed weight; the values appear in the calculation. There is no bias. Each neuron sums the incoming signals by their weights and passes the total through a sigmoid. The darker a neuron, the higher its activation. The left panel shows the opening state: x_1 and x_3 on, x_2 off. In the right panel only x_2 is on.

Step by step. Compute the left panel, the input [1, 0, 1]. The input weights of the hidden neurons are, in order: H1 [0.6, -0.4, 0.8], H2 [0.5, 0.7, -0.3], H3 [-0.6, 0.5, 0.6], H4 [0.3, -0.7, 0.5].

1. H1: $0.6 \times 1 + (-0.4) \times 0 + 0.8 \times 1 = 1.40$; sigmoid 0.80. The brightest neuron in the layer.
2. H2: $0.5 + 0 - 0.3 = 0.20$; sigmoid 0.55.
3. H3: $-0.6 + 0 + 0.6 = 0.00$; sigmoid 0.50. Even with a sum of zero, the neuron stays at half brightness.
4. H4: $0.3 + 0 + 0.5 = 0.80$; sigmoid 0.69.
5. Output O1, weights [0.7, -0.5, 0.6, 0.4]: $0.7 \times 0.80 - 0.5 \times 0.55 + 0.6 \times 0.50 + 0.4 \times 0.69 = 0.86$; sigmoid 0.70.
6. Output O2, weights [-0.4, 0.6, 0.5, -0.6]: $-0.32 + 0.33 + 0.25 - 0.41 = -0.15$; sigmoid 0.46.
7. $O1 > O2$. The network's prediction is the first output; in the figure it is the darkest box.

In the right panel only x_2 is on. The same procedure gives this table:

Input	H1	H2	H3	H4	O1	O2	Prediction
[1, 0, 1]	0.80	0.55	0.50	0.69	0.70	0.46	O1
[0, 1, 0]	0.40	0.67	0.62	0.33	0.61	0.59	O1

When the input changes, the brightness pattern of the hidden layer flips: on the left H1 stands out, on the right H2 and H3 shine. The winner does not change, though; in the second case O1 leads by only 0.02. Nobody has trained these weights, so the network's "opinion" is still arbitrary.

What is happening? The signal moves left to right, layer by layer: each neuron sums what reaches it and passes it on to the next layer. The brighter a neuron, the stronger its response. And the brightest box on the far right is the network's final prediction.

Try it yourself. 1) With every input off ($[0, 0, 0]$), what value do the four hidden neurons take? Compute the outputs as well. 2) For the input $[1, 1, 1]$, find H1's sum and its sigmoid value. 3) Does O2 win for any of the eight possible input patterns? Guess first, then test two of them by calculation.

Live demo:



book.onuronder.com/d/en/b87326a6b3

TECHNICAL DEPTH

In a feedforward network each layer takes the previous layer's activations: $a^{(l)} = \phi(W^{(l)}a^{(l-1)} + b^{(l)})$. The input layer carries the raw data, hidden layers carry intermediate representations, and the output layer carries the prediction. The weight matrices and bias vectors are the network's learned parameters.

The network in Figure 4.2 performs a real forward pass: with fixed weights, the computation runs from input to output, and neuron brightness reflects the activation values. The highest output is the network's "prediction." Training is the act of tuning these weights, and that comes next.

In the network of Figure 4.2, $W^{(1)}$ is 4×3 and $W^{(2)}$ is 2×4 ; $b^{(1)} = b^{(2)} = 0$. There are $12 + 8 = 20$ learnable weights in all; ϕ is the sigmoid in both layers.

This network gives the same answer for all eight input patterns, because nobody has tuned its weights. Can the error itself be used to correct them?

4.4 Backpropagation: learning from error

The network starts out guessing at random; forgive its first-day clumsiness. So how does it become a master? First it compares its guess with the right answer and measures how wrong it was: the error. Then that error walks backward, from exit to entrance, telling every connection it passes: “adjust your small share of the blame.”

That backward walk is called backpropagation. Follow Figure 4.3 round by round: the error flows back and the output edges toward the right answer. As the error shrinks, the backward whisper fades too; soon there is hardly any blame left to hand out.

***Margin note.** The forward pass means “make a guess”; backpropagation means “learn from the error.” Repeat those two steps millions of times and that is deep learning; there is nothing more to it.*

Figure 4.3 • Learn from error (animation)

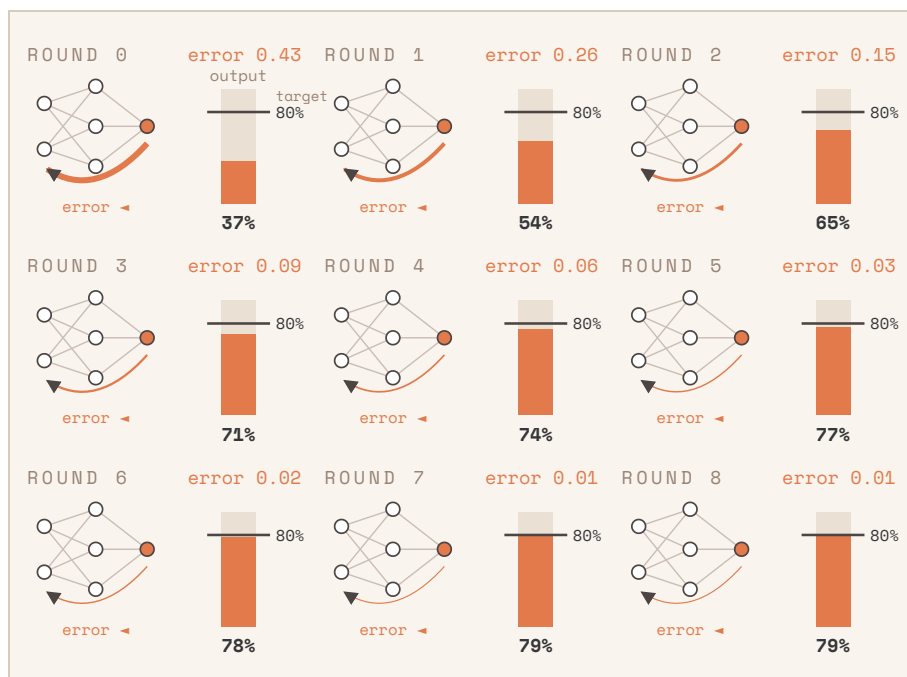


Figure 4.3

Setup. The figure is laid out like a film strip; each frame is one training round. On the left of a frame is a small network with an arrow running back from output to input: the error flowing backward. In the middle, a vertical bar shows the network's output, and the line above it marks the right answer, 80 percent. Under each frame is that round's error. In round 0 the network has not been trained at all.

Step by step. Read the frames in order.

1. Round 0: error 0.43. The output bar stands at 37 percent, far below the target. The weights are random.
2. Round 1: the error has flowed backward and the connections have been corrected. The error falls to 0.26; the bar jumps to 54 percent. A 17-point leap in a single round.
3. Round 2: error 0.15, bar at 65 percent. The step has shrunk: 11 points.

4. Round 3: error 0.09, bar at 71 percent. The error is below 0.10 for the first time.
5. Round 4: error 0.06, bar at 74 percent.
6. Round 5: error 0.03, bar at 77 percent.
7. Rounds 6 and 7: error 0.02, then 0.01. The bar reads 78 and 79 percent; it is glued to the target line.
8. Round 8: error still 0.01, bar at 79 percent. There is almost no blame left to carry backward; the network counts as trained.

Round	Error	Output	Distance to target
0	0.43	37%	43 points
1	0.26	54%	26 points
2	0.15	65%	15 points
3	0.09	71%	9 points
4	0.06	74%	6 points
5	0.03	77%	3 points
6	0.02	78%	2 points
7	0.01	79%	1 point
8	0.01	79%	1 point

The rule is regular: each round removes 40 percent of the error and keeps 60. Big corrections at first, then finer and finer adjustments. This is an animation; in real training the curve does not descend this smoothly, and now and then it even rises. But the direction is the same: the error shrinks, and the guess closes in on the target.

What is happening? The network starts by guessing at random, so the error (loss) is high. Each round, it spreads the error backward from output to input and adjusts every connection slightly to reduce it. Bit by bit, the output closes in on the right answer.

Try it yourself. 1) If the rule kept going, what would the error be in round 9? Write it with two decimals. 2) From round 2 to round 3, how many points did the output bar rise? Roughly how many times

smaller is that step than the one from round 0 to round 1? 3) If the target were 100 percent instead of 80, where would the bar start with the round 0 error of 0.43?

Live demo:



book.onuronder.com/d/en/b198800899

TECHNICAL DEPTH

Backpropagation efficiently computes the gradient of the loss with respect to every weight using the chain rule; gradients are carried backward layer by layer, from output to input. Gradient descent then updates the weights: $W \leftarrow W - \eta \cdot \partial L / \partial W$.

This method (popularized by Rumelhart, Hinton and Williams in 1986) is the key to training deep networks. The animation in Figure 4.3 qualitatively shows the backward flow of error and the output converging to the target; real training repeats the same loop over millions of examples.

The error curve in the figure follows $L(r) = 0.43 \cdot 0.6^r$, and the output is $0.80 - L(r)$, where r is the round number. This constant-ratio decay is close to the typical look of gradient descent with a well-chosen learning rate.

Backpropagation trains networks of every kind. Feeding the tens of thousands of pixels of a photo into a flat layer is wasteful, though; an image calls for a layout of its own.

4.5 Seeing images: convolutional networks (CNN)

An image holds tens of thousands of pixels; listening to each one separately would be madness. Convolutional neural networks (CNNs) are craftier: like sweeping a flashlight around a dark room, they glide

a small “filter” across the image, hunting local patterns; an edge here, a corner there.

In Figure 4.4 the flashlight sweeps the image step by step. At every stop it examines a tiny region and marks its findings on a “feature map.” Swap the flashlight (the filter) and see which edges it catches this time.

Margin note. The same small filter is used across the whole image (weight sharing). So it can spot an edge in the top-left and the bottom-right alike, and it learns with far fewer parameters.

Figure 4.4 • Convolution: slide the filter

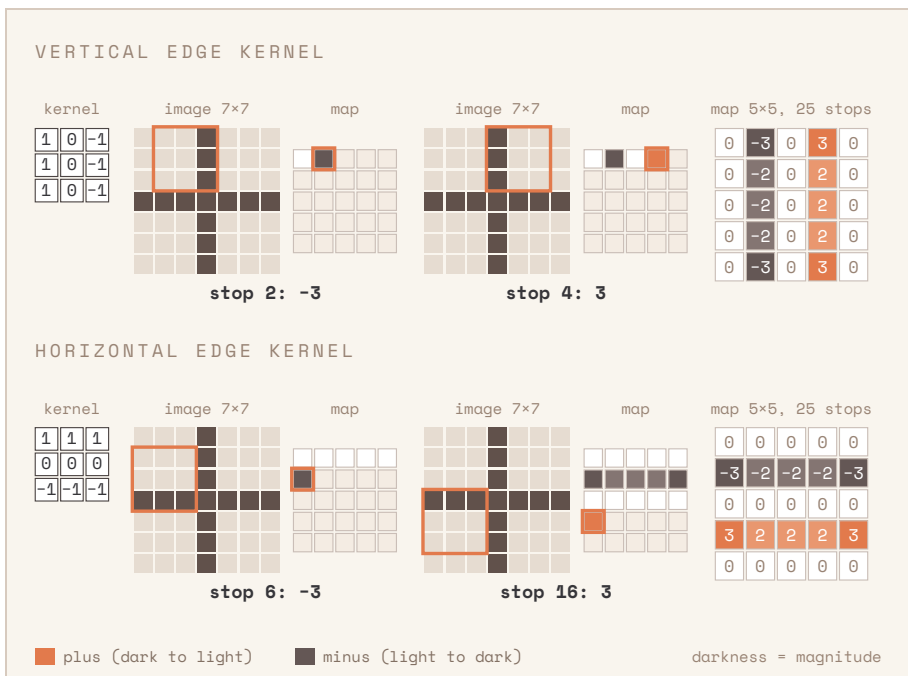


Figure 4.4

Setup. On the left is a 7×7 image; dark cells hold 1, light cells 0. The image is a plus sign: the fourth row and the fourth column are dark throughout. In the middle is the 3×3 kernel, that is, the filter. The vertical edge kernel has $[1, 0, -1]$ in each of its three rows. The hori-

ontal edge kernel has $[1, 1, 1]$ on top, zeros in the middle and $[-1, -1, -1]$ at the bottom. On the right is the 5×5 feature map: 25 stops, left to right and top to bottom, one cell per stop. Plus means a change from dark to light, minus from light to dark; darkness shows the magnitude. Two stops of each kernel are picked out.

Step by step. Slide the vertical edge kernel along the first row, stop by stop.

1. Stop 1: rows 1 to 3, columns 1 to 3. All zeros; the sum is 0.
2. Stop 2: columns 2 to 4. Its right column (column 4 of the image) is 1 in all three rows. Three rows of $1 \times 0 + 0 \times 0 + (-1) \times 1$ give -3 . Cell 2 of the map: a minus at full darkness; the figure picks out this stop.
3. Stop 3: columns 3 to 5. The dark column lands in the middle, where the kernel is zero. The sum is 0.
4. Stop 4: columns 4 to 6. The dark column is now on the left, weight $+1$. The sum is $+3$; the figure picks out this stop too.
5. Stop 5: columns 5 to 7. All zeros again.
6. In the second row (window rows 2 to 4) the window meets the horizontal arm, where left and right cancel. The values fall to -2 and $+2$.

All 25 stops:

Vertical kernel				
0	-3	0	3	0
0	-2	0	2	0
0	-2	0	2	0
0	-2	0	2	0
0	-3	0	3	0
Horizontal kernel				
0	0	0	0	0
-3	-2	-2	-2	-3

0	0	0	0	0
3	2	2	2	3
0	0	0	0	0

The vertical kernel marks only the two sides of the vertical line; the horizontal arm vanishes. The horizontal kernel does the opposite; its picked-out stops are 6 (−3) and 16 (+3). The same nine numbers travel the whole image; wherever the edge is, it gets found.

What is happening? A small “filter” roams the image step by step, like a magnifying glass. At every stop, if the little region holds the pattern it is looking for (say a vertical edge), it marks that spot bright. The result is a map showing where in the image that pattern lives.

Try it yourself. 1) With the vertical kernel, compute stop 22: rows 5 to 7, columns 2 to 4. 2) With the horizontal kernel, compute the center stop: rows 3 to 5, columns 3 to 5. 3) With a 5×5 kernel instead of 3×3 , how many cells would the feature map have?

Live demo:



book.onuronder.com/d/en/39c97bee31

TECHNICAL DEPTH

CNNs detect local patterns with shared-weight convolution kernels; the kernel slides across the image, and at each position the sum of element-wise products produces a feature map. Pooling reduces dimensionality; as layers deepen, the network learns hierarchical representations, from edges to shapes to objects.

Weight sharing and local connectivity greatly reduce the parameter count and grant translation invariance. LeNet (LeCun) pioneered this architecture; AlexNet (2012) made CNNs visible at scale. Figure 4.4 shows a real convolution operation.

The operation in the figure is $(I * K)(r, c) = \sum_i \sum_j K(i, j) \cdot I(r + i, c + j)$, with $i, j \in \{0, 1, 2\}$. There is no padding and the stride is 1, so a 7×7 image and a 3×3 kernel give a map of size $(7 - 3 + 1) = 5$. Nine weights are shared across 25 positions.

Convolution captures what sits next to what in space. What about what comes after what in time, such as the order of the words in a sentence?

4.6 Understanding sequences: recurrent networks (RNN)

Picture a child listening to a bedtime story: each new sentence is heard through the memory of the ones before, or the tale falls apart. Text, music and speech are the same: all sequences, and order matters. “The dog bit the man” and “The man bit the dog” carry the same words yet tell utterly different stories. That is why recurrent networks (RNNs) carry a “memory.”

An RNN listens word by word, refreshing its memory at every step; it moves forward without losing the past. In Figure 4.5 the words are fed in one at a time, and the memory shifts each time.

Margin note. The essence of an RNN in one line: “process each new input together with the memory of everything seen so far.” Because the same cell is reused again and again, it views the sequence as a loop.

Figure 4.5 • Processing with memory (animation)

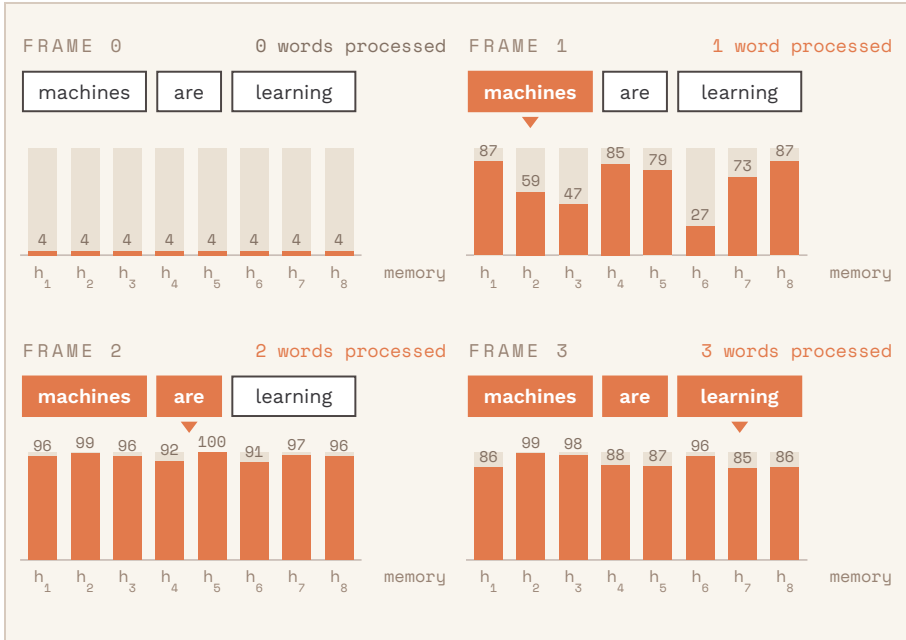


Figure 4.5

Setup. The figure has four frames. Above each frame is a three-word sequence: “machines,” “are,” “learning.” Processed words are shaded dark. Below them, eight vertical bars show the hidden state, that is, the network’s memory; each bar holds a fill between 0 and 100 percent. Under the frame is the count of words processed. In the first frame the memory is empty.

Step by step. Read the frames from left to right.

1. Frame 0, “0 words processed”: all eight bars stand at 4 percent. The memory is nearly zero; $h_0 = 0$.

2. Frame 1, “machines” processed: the bars read 87, 59, 47, 85, 79, 27, 73, 87. Even one word has begun to fill the memory; h_6 is low (27), h_1 and h_8 are high (87).
3. Frame 2, “are” processed: 96, 99, 96, 92, 100, 91, 97, 96. Every bar has risen; h_5 hits the ceiling (100). The second word did not erase the first; it was layered on top.
4. Frame 3, “learning” processed: 86, 99, 98, 88, 87, 96, 85, 86. This time the pattern was reshaped: h_5 dropped from 100 to 87, h_6 rose from 91 to 96, h_2 stayed put.

Word	h_1	h_2	h_3	h_4	h_5	h_6	h_7	h_8
(empty)	4	4	4	4	4	4	4	4
machines	87	59	47	85	79	27	73	87
are	96	99	96	92	100	91	97	96
learning	86	99	98	88	87	96	85	86

The exact numbers matter less than the behavior: the memory is not only added to, it is reshuffled at every step. The same eight bars carry all three words; the network does not open a new memory for each word, it updates the one it has. This is an animation: the bars follow a fixed rule and know nothing about the meaning of the words. In a real RNN every value depends both on the previous memory and on that word’s numerical representation.

What is happening? The network reads the words one by one, carrying a “memory” as it goes. With each new word it updates that memory using both the new word and everything gathered so far. So it keeps track of order; “the dog bit the man” and “the man bit the dog” are no longer the same thing to it.

Try it yourself. 1) From frame 2 to frame 3, which bars fell, which rose, and which stayed put? 2) In the formula $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t)$, if $h_0 = 0$, which term contributes nothing in the first step? 3) Would a model that ignores word order produce the same memory for “the dog bit the man” and “the man bit the dog”? What does an RNN do instead?

Live demo:


book.onuronder.com/d/en/4feb42a9fb

TECHNICAL DEPTH

An RNN updates its hidden state at each timestep with $h_t = \phi(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$; the same weights are reused at every step (parameter sharing in time). That lets it process variable-length sequences through one context vector.

Classic RNNs struggle with long dependencies due to vanishing gradients; LSTM (Hochreiter & Schmidhuber, 1997) and GRU ease this with gate mechanisms. In most modern sequence tasks, RNNs have largely given way to attention-based transformers; they come in the next chapter. The animation in Figure 4.5 shows the hidden-state update in simplified form.

Hidden state $h_0 = 0$. Each step updates it with $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t)$; the same weights are reused for every word.

The eight bars in the figure stand for an 8-dimensional hidden state vector. In a real application x_t is the word's embedding vector, and W_x and W_h are learned in training; in the animation the bars are produced by a fixed rule that depends only on the step count.

So far the networks have only recognized: a neuron fired, an edge was found, a sequence was remembered. Can a network produce a face it has never seen, from scratch?

4.7 The art of forgery: GANs

Sometimes the goal isn't recognizing but creating: realistic faces, landscapes, sounds. The trick of generative adversarial networks

(GANs) is locking a forger and a detective in the same room. The forger (generator) produces fakes; the detective (discriminator) tries to tell real from fake.

The two race without rest: as the detective catches fakes, the forger sharpens; as the forger sharpens, the detective's eye grows keener. In Figure 4.6, round by round, an image born as pure noise edges toward the real thing as the forger masters its craft.

Margin note. What makes a GAN work: “making a good fake” and “catching the fake” keep pushing each other. Like a forger racing a detective: as both improve, the result becomes strikingly realistic.

Figure 4.6 • Generator vs Discriminator

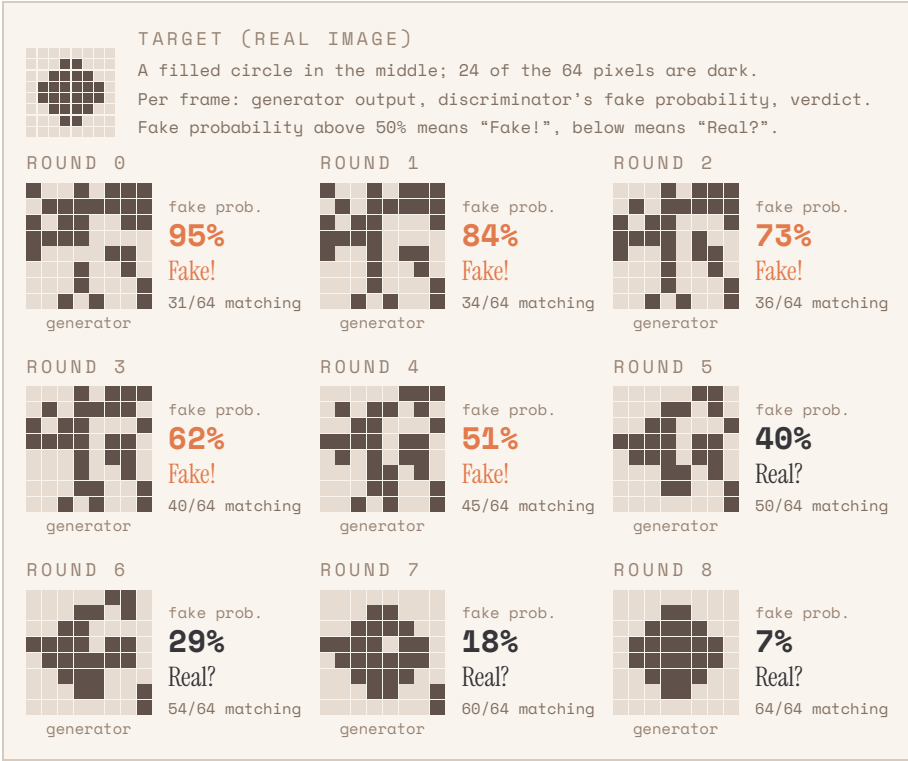


Figure 4.6

Setup. The figure is a film strip of nine frames, from round 0 to round 8. On the left of each frame is the 8×8 image the generator drew: dark and light pixels. The “real” image the generator is trying to learn sits at the top: a filled circle in the middle, with 24 of the 64 pixels dark. On the right of each frame is the discriminator’s decision: the probability that the image is fake, and a one-word verdict. Above 50 percent means “Fake!” and below means “Real?”.

Step by step. Read the frames in order.

1. Round 0: the generator draws pure noise; only 31 of the 64 pixels agree with the target, a coin toss. The detective is certain: fake probability 95 percent, verdict “Fake!”
2. Rounds 1 and 2: the probability falls to 84 percent, then 73. The first traces of the circle appear in the image; matching pixels 34, then 36.
3. Rounds 3 and 4: 62 and 51 percent. Matching pixels 40 and 45. The detective still says “Fake!” but only just; 51 percent is a hair above the line.
4. Round 5: the probability drops to 40 percent, and the verdict becomes “Real?” The detective is fooled for the first time. Matching pixels: 50.
5. Rounds 6 and 7: 29 and 18 percent. The circle is now clearly visible; matching pixels 54 and 60.
6. Round 8: 7 percent, verdict “Real?” All 64 of the 64 pixels are in place; the generator has matched the target.

Round	Fake probability	Verdict	Matching pixels
0	95%	Fake!	31 / 64
1	84%	Fake!	34 / 64
2	73%	Fake!	36 / 64
3	62%	Fake!	40 / 64
4	51%	Fake!	45 / 64
5	40%	Real?	50 / 64

6	29%	Real?	54 / 64
7	18%	Real?	60 / 64
8	7%	Real?	64 / 64

The probability falls 11 points every round; that is the fixed rule of the animation. In real training both networks learn at the same time: the detective sharpens too, so the decline does not follow such a straight line. But the moment the verdict tips over is the moment a GAN is after.

What is happening? Two networks are racing: one (the generator) makes fake images, the other (the discriminator) tries to catch whether they are fake or real. At first the forger is clumsy and easily caught. Each round it learns to forge a little better, and the “fake” probability drops, like a forger racing a detective.

Try it yourself. 1) In which round does the verdict flip from “Fake!” to “Real?” and what is the probability in that round? 2) How is the 7 percent of round 8 found; write the rule and compute it. 3) If the detective said 50 percent for every image, what would that be a sign of?

Live demo:



book.onuronder.com/d/en/888b4fc273

TECHNICAL DEPTH

GANs (Goodfellow et al., 2014) train two networks in an adversarial game: the generator G produces samples from random noise; the discriminator D tries to separate real from generated samples. G is trained to maximize its chance of fooling D , while D is trained simultaneously not to be fooled.

Training is a min-max game; at equilibrium the generator's samples become indistinguishable from the real distribution. GANs were a breakthrough for photorealistic images; today diffusion models are a common alternative (Chapter 5). The animation in Figure 4.6 shows this rivalry in simplified form.

The objective of the game is $\min_G \max_D V(D, G) = \mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log(1 - D(G(z)))]$; at equilibrium $D(x) = 1/2$. In the animation the fake probability is $p(r) = 95 - 11r$ percent, and in round r roughly $r/8$ of the pixels lock onto the target.

The networks have learned both to recognize and to create. Today's systems that chat and draw take one more step, attention; that is the next chapter.

4.8 Test yourself

Answers are at the back of the book.

1. What does an artificial neuron compute?
 - a) It copies the inputs as-is
 - b) Just the average of the inputs
 - c) A random number
 - d) A weighted sum of inputs + bias, then an activation
2. What makes a neural network “deep”?
 - a) Many hidden layers
 - b) Having a single neuron

- c) Running very fast
 - d) Being connected to the internet
3. What if there were no activation function at all?
- a) The network would get stronger
 - b) The network would collapse into one linear function
 - c) Nothing would change
 - d) The network would get faster
4. What does backpropagation do?
- a) Enlarges the image
 - b) Adds new layers
 - c) Spreads the error backward and updates weights to reduce it
 - d) Deletes the data
5. What data are CNNs especially strong at?
- a) Images
 - b) Passwords
 - c) Tables
 - d) A single number
6. Which two networks compete in a GAN?
- a) Input and Output
 - b) CNN and RNN
 - c) Teacher and Student
 - d) Generator and Discriminator

What to keep from this chapter

- An artificial neuron is a crude mathematical imitation of the brain; one neuron does little, and everything depends on the pattern that millions of them build together.
- A neuron weighs its inputs, sums them, adds a bias and passes the total through an activation; with $w = [0.7, -0.5, 0.9]$ a sum of 0.69 gives 0.666 in sigmoid, and the neuron fires.
- In neurons arranged in layers, the signal flows from input to output; that is the forward pass, and the highest output is the netw-

ork's prediction.

- Without activation, a network of any number of layers would collapse into a single linear function.
- Backpropagation carries the error from output to input and updates every weight in the direction that reduces it; in the animation the error shrank by 40 percent each round.
- Convolutional networks slide one small kernel across the whole image; the same nine numbers find an edge wherever it is.
- Recurrent networks process each word together with the memory of the ones before; in a GAN, the generator and the discriminator push each other toward the real thing.

5

CHAPTER 5

Today's AI

5.1 The generative era: from recognizing to creating

Until now, machines stayed on the “recognizing” side: Is this spam, is this a cat, what is this house worth? Like a painter’s apprentice spending years just studying canvases. Then one day the apprentice picked up the brush: Machines began to create. They write, they paint, they generate code, they hold conversations.

One invention made all of it possible: the Transformer architecture and the idea of “attention” at its heart. The story starts with the smallest piece: how does a sentence look to a machine? The answer lies in small chunks of text called “tokens.” From there the chapter moves to embeddings, which turn meaning into numbers, to the attention mechanism, and to how a language model writes word by word. Then comes training, the diffusion models behind image generation, and finally the limits of these systems.

***Margin note.** The answer to “why right now?” stands on three legs: plentiful data (the internet), powerful hardware (GPUs) and the right architecture (the Transformer). When the three came together, the generative era began.*

TECHNICAL DEPTH

Generative AI marks the shift from discriminative to generative modeling: instead of $p(y|x)$, model the distribution that generates the data, $p(x)$ or $p(x|condition)$. The modern leap came largely from the Transformer architecture (Vaswani et al., 2017, “Attention Is All You Need”) and the accumulation of data and compute able to train it at scale.

This chapter builds the generative stack end to end: tokenization → embedding space → self-attention → autoregressive generation → the training pipeline (pretraining, fine-tuning, RLHF) → diffusion-based image generation → and the practical limits (hallucination, context window, cost). By the end, you will have an intuitive but accurate picture of why and how today's LLMs and generative models work.

The stack starts with the smallest piece. You read a sentence letter by letter, word by word; what does the machine read?

5.2 How a machine sees words: tokens

A language model never sees letters or words “as they are.” It first breaks text into tiny Lego bricks called tokens. A token is sometimes a whole word, sometimes a broken piece of one, sometimes just a comma.

Why break things up? Think of a Lego box: a limited set of bricks builds endless things. Memorizing every word in the world is impossible; but with a limited kit of pieces, any word can be built. Pick an example below and watch the machine take a sentence apart.

Margin note. The same text takes a different number of tokens in different languages. So asking an LLM something in Turkish can be more “expensive” than asking in English.

Figure 5.1 • Split a sentence into tokens

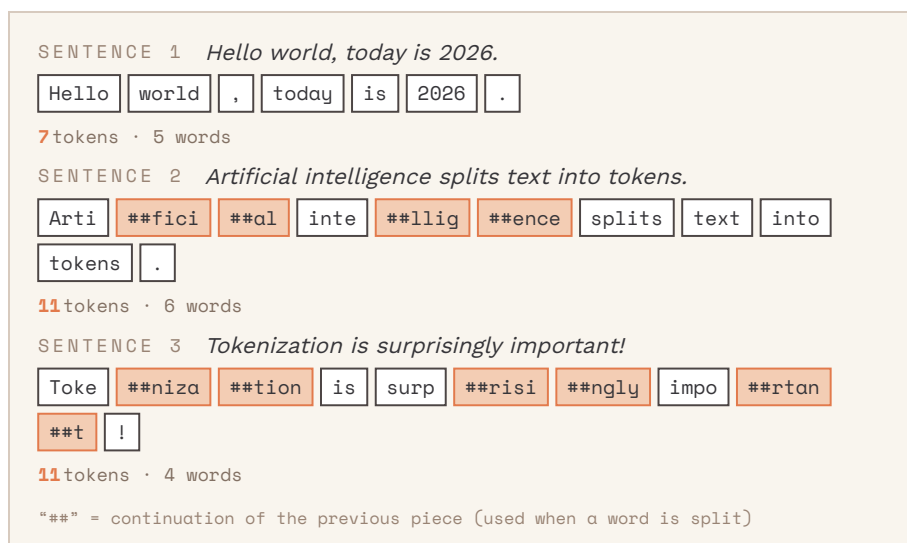


Figure 5.1

Setup. The figure shows three example sentences and, under each, the same sentence taken apart into tokens. Every token sits in its own box. A plain box is a stand-alone piece; a shaded box is a continuation piece and starts with “##.” Two counts appear under each sentence: the number of tokens and the number of words. The splitter behind the figure follows one simple rule, given below.

Step by step. The splitter’s rule fits in three lines. A word of six letters or fewer stays a single token. A longer word is cut into four-character pieces; the first piece is written plain, the rest with “##.” A comma, a period or an exclamation mark is always a token of its own. Apply the rule to the three sentences:

Sentence	Tokens	Words	Tokens
Hello world, today is 2026.	Hello · world · , · today · is · 2026 · .	5	7
Artificial intelligence splits text into tokens.	Arti · ##fici · ##al · inte · ##llig · ##ence · splits · text · into · tokens · .	6	11
Tokenization is surprisingly important!	Toke · ##niza · ##tion · is · surp · ##risi · ##ngly · impo · ##rtan · ##t · !	4	11

In the first sentence every word is short: Hello, world and today have five letters each, so all of them stay whole. “2026” is a number, but to the splitter it is an ordinary four-character word. The comma after “world” and the final period each get a box of their own. Five words become seven tokens. In the second sentence “Artificial” has ten letters and breaks into three pieces, four, four and two. “intelligence” has twelve letters and breaks into three pieces of four. “splits” and “tokens” have six letters each; they sit on the limit and stay whole.

The third sentence is the striking one: four words, eleven tokens. “Tokenization” and the adverb after “is” have twelve letters each and take three pieces apiece (Toke · ##niza · ##tion; surp · ##risi · ##ngly). The word “important” has nine letters and ends in a one-letter tail (impo · ##rtan · ##t). Only “is” stays untouched.

Count tokens per word and the three sentences come out at 1.4, 1.8 and 2.75. The second and third sentences both make 11 tokens, one from six words, the other from four. The count depends less on how many words there are than on how many of them are long and rare. Turkish suffixes make words longer, so the same text takes more tokens in Turkish than in English, and costs more.

What is happening? The model never sees a sentence whole; it first chops it into small pieces (tokens). Short words stay in one piece, long words split into chunks marked “##,” and punctuation counts separately. These pieces are the machine’s entire alphabet.

Try it yourself. 1) Split “Artificial intelligence learns.” with the same rule. How many tokens do you get? 2) How many pieces does the word “internationalization” break into? Write them out. 3) How many tokens per word does the third sentence carry? Compare it with the first sentence.

Live demo:



book.onuronder.com/d/en/b2bf8192ef

TECHNICAL DEPTH

Modern models use subword tokenization (e.g. BPE, WordPiece, SentencePiece): frequent sequences become single tokens, rare words split into several pieces. Vocabularies typically hold 30K–100K+ tokens; each maps to an integer ID.

Roughly, 1 token $\approx$ 0.75 words in English; agglutinative languages like Turkish can use more tokens per word. Model context and cost are measured in tokens: both the context window and pricing depend on token count. Figure 5.1 is a simplified subword splitter (long words break into pieces marked with “##”).

The rule of Figure 5.1 (at most 6 characters for a single token, then four-character pieces) is not a stand-in for a real BPE vocabulary. In a real vocabulary a frequent word like “Artificial” would be a single token, while “Tokenization” would split into two or three pieces according to the vocabulary’s frequency statistics. The rule is simple, but the result points the same way: rare and long words take more pieces.

Every token now carries an ID number, and an ID number says nothing about “cat” being close to “dog.” Where does meaning come from?

5.3 Turning meaning into numbers: embeddings

Tokens enter the machine as numbers, but a dry ID number carries no “meaning.” Enter the embedding: Every word is given an address in a vast city. That address is a list of numbers carrying the word’s meaning.

In this city, words with similar meanings move into the same neighborhood. “Cat” and “dog” are next-door neighbors; so are “king” and “queen.” Pick a word in Figure 5.2 and meet its neighbors.

Margin note. Embeddings aren’t just for words: sentences, images and sounds can live in the same space. This is the foundation of multimodal models and semantic search.

Figure 5.2 • The meaning map

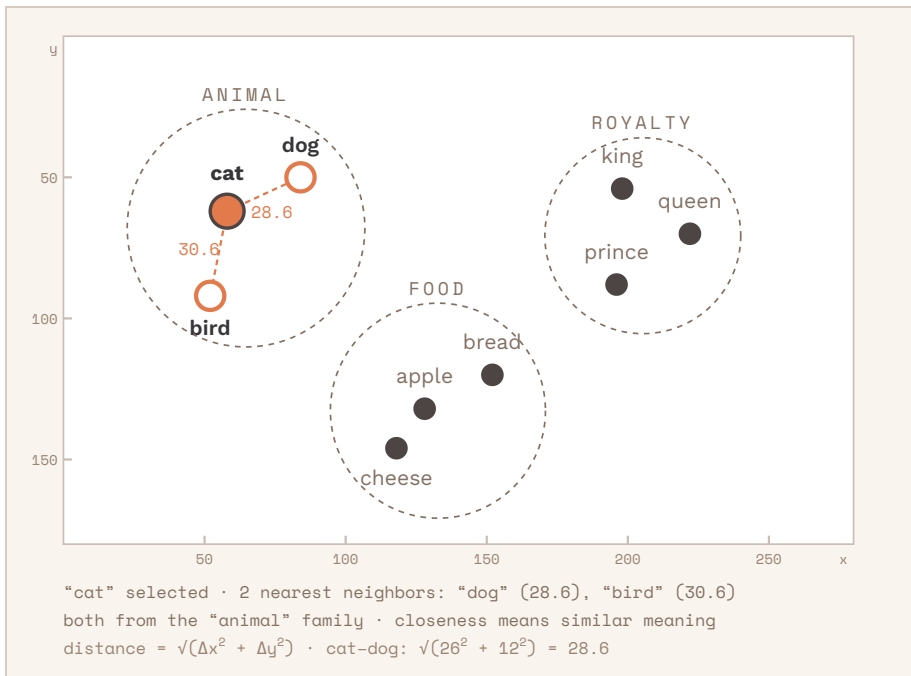


Figure 5.2

Setup. The figure places nine words on a two-dimensional map. Three dashed circles mark three families of meaning: animals (cat, dog, bird), royalty (king, queen, prince) and food (apple, bread, cheese). Each point carries its word; the coordinates are in the table below. Points that sit close together are close in meaning too. Distance is measured with a plain ruler.

Step by step. The positions on the map and the two nearest neighbors of every word:

Word	Family	x	y	2 nearest neighbors (distance)
cat	animal	58	62	dog (28.6), bird (30.6)
dog	animal	84	50	cat (28.6), bird (52.8)
bird	animal	52	92	cat (30.6), dog (52.8)
king	royalty	198	54	queen (28.8), prince (34.1)
queen	royalty	222	70	king (28.8), prince (31.6)
prince	royalty	196	88	queen (31.6), king (34.1)
apple	food	128	132	cheese (17.2), bread (26.8)
bread	food	152	120	apple (26.8), cheese (42.8)
cheese	food	118	146	apple (17.2), bread (42.8)

The distance comes from Pythagoras: take the horizontal and vertical differences between two points, square them, add them, take the square root. For “cat” and “dog” the differences are 26 and 12; their squares are 676 and 144; the sum is 820; the square root is 28.6. Do this for all nine words and every word’s two nearest neighbors come from its own family, without exception. The figure’s own caption for “cat” reads: “The 2 nearest neighbors: dog (28.6), bird (30.6); both from the ‘animal’ family.”

Between families the distances grow. From “cat” to “apple” is 99 units; from “cat” to “queen” is 164. The map tells you who is a neighbor, who is a stranger, and by how much. There is a border zone as

lty district and the food district touch. What the machine calls “meaning” is nothing more than this geometry.

What is happening? Every word becomes a point on a map. Words close in meaning sit close on the map too; the 2 nearest points are the 2 most similar words. The machine learned this placement itself, by seeing which words appear in similar sentences.

Try it yourself. 1) Compute the distance between “bird” and “cheese.” Compare it with the largest within-family distance in the table. 2) Suppose you want to add the word “lion” to the map. Which region would you put it in? Propose a sensible x and y pair and find its two nearest neighbors. 3) Which word is third-nearest to “bread,” and from which family? What does that tell you about the map?

Live demo:



book.onuronder.com/d/en/ab3b439524

TECHNICAL DEPTH

An embedding maps a token to a dense vector (typically hundreds to thousands of dimensions). These vectors are learned in training; semantic and syntactic relations are reflected in the geometry. Similarity is usually measured with cosine similarity.

The famous example: with vector arithmetic, analogies like king – man + woman $\approx$ queen can emerge. The visualization in Figure 5.2 is a 2D reduction (PCA/t-SNE-like) of a high-dimensional space; real embeddings have far more dimensions.

The distance in Figure 5.2 is the Euclidean distance: $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. Real embeddings are more often compared with cosine similarity: $\cos \theta = (a \cdot b) / (\|a\| \cdot \|b\|)$. This measure compares the direction of the vectors, not their length; a value near 1 means the same direction, 0 means perpendicular (unrelated), -1 means opposite directions.

Every word has an address. Inside a sentence, though, how does a word choose which neighbor to look at?

5.4 Attention: the heart of the Transformer

Read this sentence: “The cat ran away because it was scared.” Who is “it”? Without even noticing, you glanced back at “the cat.” The attention mechanism teaches a machine that very glance: Each word learns which other words to “look at” for its meaning.

Pick a word in Figure 5.3; see how much it “attends” to the others by the depth of the color. The darker the color, the stronger the tie.

Margin note. “Attention Is All You Need” (2017): the attention mechanism removed the RNN’s one-at-a-time processing constraint. Looking at every word at once boosted both speed and understanding.

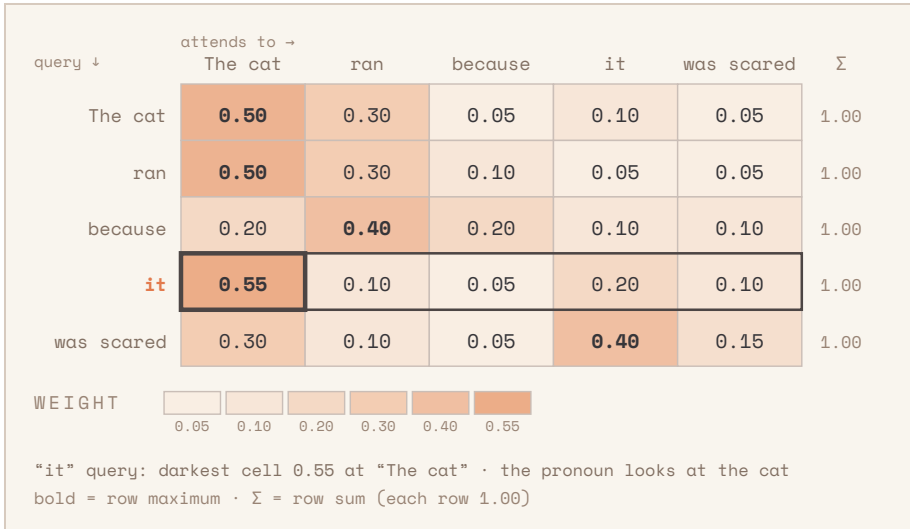
Figure 5.3 · Which word looks at which?

Figure 5.3

Setup. The figure lays the sentence out as a heat map of five units: “The cat,” “ran,” “because,” “it” and “was scared.” The figure joins “The” to “cat” and “was” to “scared,” and leaves out “away,” so five units stand for the sentence. Rows are the unit doing the looking (the query); columns are the unit being looked at. Each cell holds an attention weight, and the numbers in a row add up to 1.00. The darker the cell, the larger the weight. The darkest cell of every row is printed in bold.

Step by step. The weight table:

Query ↓ · Attends to →	The cat	ran	because	it	was scared
The cat	0.50	0.30	0.05	0.10	0.05
ran	0.50	0.30	0.10	0.05	0.05
because	0.20	0.40	0.20	0.10	0.10
it	0.55	0.10	0.05	0.20	0.10
was scared	0.30	0.10	0.05	0.40	0.15

Read the rows one at a time:

1. “The cat”: the highest weight is on itself (0.50). After itself, it looks most at “ran” (0.30); the subject is looking for its verb.
2. “ran”: half of the weight goes to “The cat.” The verb wants to know who ran.
3. “because”: it looks most at “ran” (0.40). The conjunction holds on to the event it explains.
4. “it”: the row the section is about. The darkest cell is 0.55, at “The cat”; the share it keeps for itself is only 0.20. The pronoun works out what it stands for by looking at the cat.
5. “was scared”: it looks most at “it” (0.40), with “The cat” in second place (0.30). The predicate finds its subject in two hops: first the pronoun, then the cat the pronoun points to.

Two more things. The table is not symmetric: “it” looks at the cat with 0.55, but “The cat” looks at “it” with only 0.10; looking runs one way. And distance does not matter: two units stand between “it” and “The cat,” yet the weight is the highest in the whole table. That is where the older models, which read one word after another, ran into trouble.

What is happening? To make sense of a sentence, each word decides which others it should “pay attention” to. The darker the color, the stronger the bond. That is how the model works out what a little word like “it” refers to.

Try it yourself. 1) Check the sum of every row; is each one 1.00? 2) If the sentence were “The cat ran because the dog was scared,” where would the darkest cell of the “was scared” row move? Write down why. 3) Why are the weights in the “because” column so low? What does that say about how much meaning a conjunction carries?

Live demo:



book.onuronder.com/d/en/20e333ccc

TECHNICAL DEPTH

Self-attention produces query (Q), key (K) and value (V) vectors for every token; weights are computed with $\text{softmax}(Q \cdot K^T / \sqrt{d})$, and the output is the weighted sum of the values. Every position can thus reach the whole sequence regardless of distance.

The Transformer does this with multiple heads: different “heads” capture different kinds of relation (syntax, coreference, and so on). Positional encoding adds order information. The cost is $O(n^2)$ in sequence length; that is the main reason context windows are limited.

Every row of Figure 5.3 is a softmax output: the weights lie between 0 and 1 and sum to 1. The output at the position of “it” is the weighted sum of the value vectors: $0.55 \cdot V(\text{The cat}) + 0.10 \cdot V(\text{ran}) + 0.05 \cdot V(\text{because}) + 0.20 \cdot V(\text{it}) + 0.10 \cdot V(\text{was scared})$. The new representation of “it” is therefore a blend, more than half of which comes from “The cat.” The asymmetry of the table comes from the same place: each row is computed with its own query, and $Q \cdot K^T$ is not a symmetric matrix.

Every word knows whom to look at. How does the next word, not yet written, grow out of these glances?

5.5 A language model: predict the next word

You know the game: someone says “The early bird catches the...” and you shout “worm!” Odd as it sounds, that is a language model’s only job: look at the text so far and guess the most likely next word (token). It adds the guess to the text and guesses again. Playing this tiny game thousands of times, it writes whole paragraphs.

Follow the steps of Figure 5.4: which words the model weighs at each step, with what probability, and how the sentence takes shape.

Compare the two “creativity” (temperature) rows too: should it always pick the most likely word, or take a little risk?

Margin note. “Does it understand, or does it just predict?” In a way, both: it couldn’t predict this coherently without capturing some meaning. But at its core, all it does is guess the next token.

Figure 5.4 • Generate word by word

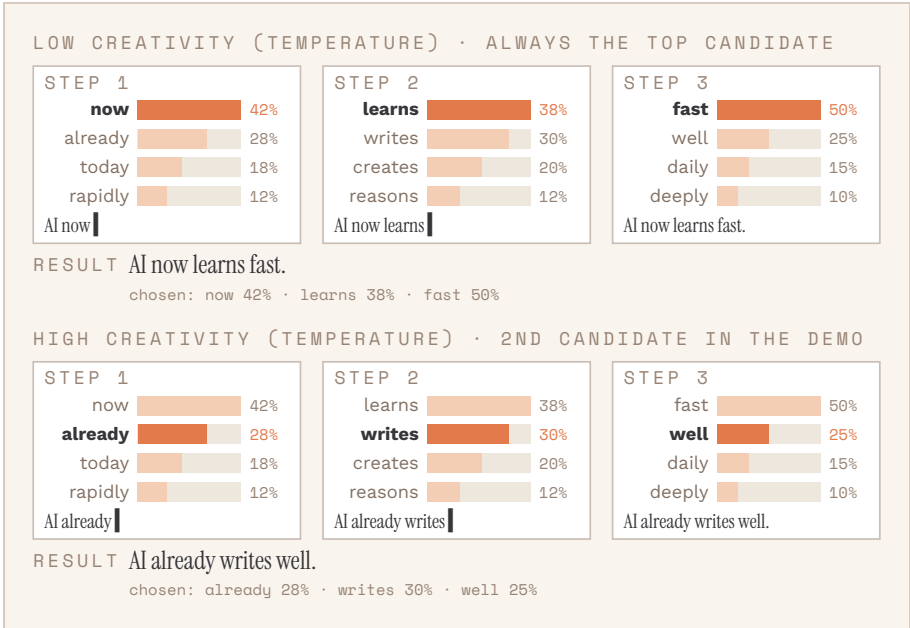


Figure 5.4

Setup. The figure is a film strip of three generation steps, laid out in two rows. The starting text is the single word “AI”. In each frame the model shows four candidate words and the probability it gives each one; the length of the bar is proportional to the probability. The top row of the strip is low creativity (temperature), the bottom row high creativity. The two rows build two different sentences from the same candidates.

Step by step. The candidate lists of the three steps:

Step	Candidates and probabilities
1	now 42% · already 28% · today 18% · rapidly 12%
2	learns 38% · writes 30% · creates 20% · reasons 12%
3	fast 50% · well 25% · daily 15% · deeply 10%

At low creativity the model takes the most likely candidate at every step:

1. “now” (42%) → “AI now”
2. “learns” (38%) → “AI now learns”
3. “fast” (50%) → “AI now learns fast.”

At high creativity the figure takes the second candidate at every step, to keep the illustration simple:

1. “already” (28%) → “AI already”
2. “writes” (30%) → “AI already writes”
3. “well” (25%) → “AI already writes well.”

Both sentences are grammatical; the second takes a less expected road. A real model with sampling on rolls dice: with these probabilities it picks the 42-percent candidate in about 42 of a hundred tries and the 12-percent candidate in about 12. The higher the temperature, the more evenly the dice are weighted. From the same start a different sentence can come out every time. At low temperature there are no dice; the result is the same every time.

The probabilities at each step are spread over four candidates and add up to a hundred percent. In reality the model spreads this distribution over tens of thousands of tokens; the four candidates are only the top of the list. And each choice changes the next step's question. After “now,” “learns” is likely; after “already” it is likely too. But in a real model the list of step 2 is recomputed according to the choice made in step 1.

What is happening? At each step the model asks “what is the most likely word after the text so far?” It picks one, adds it to the sent-

ence and starts over. Creativity off: always the top word. Creativity on: sometimes a less likely one; that is where surprise comes from.

Try it yourself. 1) What would the sentence be if the model took the third candidate at every step? 2) Multiply the probabilities of the low-creativity sentence: $0.42 \times 0.38 \times 0.50$. Do the same for the high-creativity sentence; which one is more likely, and by how many times? 3) In which of the three steps is the model least sure? Look at the share of the top candidate.

Live demo:



book.onuronder.com/d/en/25369cb8c7

TECHNICAL DEPTH

LLMs are autoregressive: they produce the distribution $P(\text{token}_t \mid \text{token}_1 \dots \text{token}_{t-1})$, convert it to probabilities with softmax, pick a token and append it. Greedy (argmax) picks the most likely; sampling draws from the distribution.

Temperature sharpens or flattens the distribution: low temperature gives steadier, more repetitive output; high temperature gives more varied, riskier output (together with top-k / top-p). The probabilities in Figure 5.4 are illustrative; a real model produces a distribution over its whole vocabulary.

The temperature T divides the model's scores before the softmax: $p_i = \exp(z_i/T) / \sum_j \exp(z_j/T)$. As T shrinks, the distribution collapses onto a single candidate and the choice approaches argmax; as T grows, the distribution flattens, and even the 12-percent “rapidly” gains a reasonable chance. The “take the second candidate” behavior of Figure 5.4 is a convenience of the illustration; real sampling draws at random from this distribution. The probability of a sentence is the product of its steps: $P(\text{now, learns, fast}) = 0.42 \times 0.38 \times 0.50 \approx 0.08$.

The model knows how to spread probability at every step. Where did it learn these probabilities? It went through three stages of training.

5.6 How a model is raised: the training pipeline

A chat assistant grows up like a child; it passes through three schools. First comes “pretraining” on massive text: there it learns language and the world. Then, at the “fine-tuning” school, it learns to answer questions and follow instructions. Last comes “alignment with human feedback”: there it learns to be helpful, honest and safe; its manners, you could say.

Read the three stages of Figure 5.5 one by one and see how the model answers the same question differently after each.

***Margin note.** Pretraining gives the model “what it knows”; fine-tuning and RLHF give it “how to behave.” The same fact can be delivered in very different tones.*

Figure 5.5 • An assistant in three stages

	1 • Pretraining	2 • Fine-tuning	3 • RLHF / alignment
DATA	Massive internet text	Instruction–response pairs	Human preferences (a reward model)
LEARNS	Language and the world (next-word prediction)	Following instructions (answering the question)	Being helpful, honest and safe
OUTPUT for “capital of Türkiye?”	The capital of Türkiye is Ankara, with a population of about six million. The city...	The capital of Türkiye is Ankara.	The capital of Türkiye is Ankara. If you’d like, I can also share a few interesting facts about the city.
NOTE	The raw model is a “completer”: it doesn’t answer the question, it continues the text.	Now it answers directly and concisely.	Same fact; a more helpful, polite, aligned tone.

same question, three stages: same fact, different delivery

Figure 5.5

Setup. The figure is a table with three columns; each column is one training stage. Every stage has four rows: the data the model sees, what it learns, its sample answer to the same question about a capital city, and a short note. The question is the same in all three columns, “capital of Türkiye?”; only the shape of the answer changes.

Step by step. Put the three stages side by side:

	1 • Pretraining	2 • Fine-tuning	3 • RLHF / alignment
Data	Massive internet text	Instruction–response pairs	Human preferences (a reward model)
What it learns	Language and the world (next-word prediction)	Following instructions (answering the question)	Being helpful, honest and safe

Sample output	"The capital of Türkiye is Ankara, with a population of about six million. The city..."	"The capital of Türkiye is Ankara."	"The capital of Türkiye is Ankara. If you'd like, I can also share a few interesting facts about the city."
Note	The raw model is a "completer": it doesn't answer the question, it continues the text.	Now it answers directly and concisely.	Same fact; a more helpful, polite, aligned tone.

1. Pretraining: the answer starts with the right fact but does not stop. It adds the population and goes on with "The city..." The model does not know it has been asked a question; it writes as if continuing an encyclopedia page from the internet. The knowledge is there; the manners are not.
2. Fine-tuning: the same fact, one sentence. The model has now learned the pattern "a question came, answer it, stop" from thousands of instruction–response pairs.
3. RLHF / alignment: the same answer again, with an offer on top: would you like more? People were shown two answers and asked "which is better?"; the model was adjusted toward the preferred tone.

Only one thing stays the same across the three columns: the fact, Ankara. What changes is how the fact is delivered. The order matters too: each stage needs the one before it, knowledge first, then behavior.

What is happening? An assistant grows up in three stages. First, in "pretraining," it learns language and the world from massive text. Then in "fine-tuning" it learns from example Q&A pairs how to answer. Finally, with human feedback, it learns to be helpful and polite. Same fact, three very different manners.

Try it yourself. 1) If the question were "At what temperature does water boil?", predict the stage-one model's output; then write the stage-two output. 2) In the third stage, people are shown two answers, one "wrong but polite" and one "right but rude." Which should

be preferred? How do the three goals of alignment (helpful, honest, safe) decide? 3) Which row of the table shows the split between “what it knows” and “how to behave” most clearly?

Live demo:



book.onuronder.com/d/en/bf611298ac

TECHNICAL DEPTH

1. Pretraining: on a large corpus, the model learns the statistics of language through a self-supervised next-token objective.
- 2) Supervised fine-tuning (SFT): it learns instruction-following from instruction–response pairs.
- 3) RLHF/preference alignment: a reward model is trained on human preferences and the policy (e.g. PPO or DPO) is updated against that signal.

The result: a shift from a raw “text completer” to a helpful, honest and reasonably safe assistant. Alignment isn’t perfect; open problems like jailbreaks, reward hacking and distribution shift remain.

The three stages have different objectives. In pretraining the loss rests on the probability of the next token: $L = -\sum_t \log P(\text{token}_t \mid \text{token}_1 \dots \text{token}_{t-1})$. The first column of Figure 5.5 is the natural output of this objective; continuing the text is the behavior that lowers this loss. SFT applies the same loss, but only on the response part and only on selected pairs. In the alignment stage the objective is a reward signal. The reward model estimates which of two answers is preferred, and the policy is updated in the direction that raises that estimate.

Text models are raised that way. Image models start somewhere else entirely: pure noise.

5.7 Making images: diffusion

Image-making models rest on a different idea altogether: diffusion. Picture a fogged-up window; the scene behind it is a blur. Now wipe the glass slowly: the picture emerges stroke by stroke. A diffusion model learns that wiping; only it starts from a screen of pure static (noise) and cleans a little at each step until an image appears.

Follow the frames of Figure 5.6 from left to right: the further right you go, the more noise the model clears, and the image slowly appears.

***Margin note.** Diffusion removes noise a little at a time, step by step, until the shape emerges. It also trains more stably than GANs.*

Figure 5.6 · From noise to image

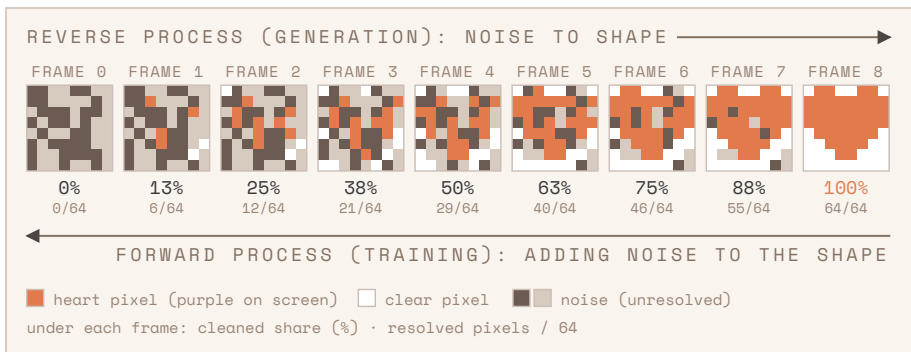


Figure 5.6

Setup. The figure is a film strip of nine frames. Each frame is a tiny canvas of 8×8 , that is, 64 pixels. The first frame on the left is pure noise: dark and light gray pixels scattered at random. The last frame on the right holds a heart of 40 orange pixels (purple on screen); the other 24 pixels are light. In the seven frames between, the model resolves a few more pixels at every step. Under each frame the cleaned share is printed as a percentage.

Step by step. Each frame is one step:

Step	Cleaned share	Resolved pixels (of 64)	Visible heart pixels	What the frame shows
0	0%	0	0	Noise only; a screen full of static
1	13%	6	4	A few orange dots, no shape yet
2	25%	12	9	Scattered dots
3	38%	21	12	The body starts to show
4	50%	29	17	The heart can be guessed
5	63%	40	24	The shape is certain, the edges rough
6	75%	46	29	A few specks of noise
7	88%	55	36	The last specks
8	100%	64	40	A clean heart: 40 orange, 24 light pixels

The resolved-pixel count does not grow evenly: 6, 12, 21, 29, 40... The illustration decides which pixel opens at which step with a fixed random seed, so the strip is the same every time, but the steps are not equal. And the shape is recognizable halfway through. At step 4, the midpoint of the strip, 29 of the 64 pixels are resolved, 17 of them heart pixels. The heart is already unmistakable; the remaining steps fill in the detail.

One warning: a real diffusion model does not “open” pixels. At every step it reduces the noise a little across the whole image; all the pixels sharpen at the same time, slowly. The number of steps is not 8 either, but dozens or hundreds. This strip is simplified to carry the idea: the noise shrinks, the shape emerges.

What is happening? Diffusion is like cleaning up a snowy TV screen (pure noise) step by step until a picture emerges from inside it. The

model learned one skill: “estimate the noise in this image and remove a little of it.” Done over and over, noise turns into a picture.

Try it yourself. 1) In the last frame 40 of the 64 pixels are orange. If 29 pixels are resolved at step 4 and pixels open at random, about how many of them would you expect to be orange? 2) If there were 16 steps instead of 8, what share would be cleaned at each step? 3) In which direction of the strip is the “forward process” described in Technical depth read, and in which direction the “reverse process”?

Live demo:



book.onuronder.com/d/en/fd88692784

TECHNICAL DEPTH

Diffusion has two processes. The forward process gradually adds Gaussian noise to an image (fixed, not learned). The reverse process is a neural network that estimates and removes the noise added at each step (denoising); this walks from noise back to the data distribution.

In text-to-image generation, the process is conditioned on a text embedding (usually CLIP-like) and, for speed, run in latent space (latent diffusion). Figure 5.6 is a qualitative illustration; real models use a learned noise-prediction network.

In the forward process the image at step t is corrupted as $x_t = \sqrt{1 - \beta_t} \cdot x_{t-1} + \sqrt{\beta_t} \cdot \epsilon$, where ϵ is standard Gaussian noise and β_t a small noise share. Given x_t and t , the network learns to estimate the added ϵ ; the training loss is the squared difference between the estimate and the true noise: $\|\epsilon - \epsilon_{\theta}(x_t, t)\|^2$. Generation runs the other way: start from pure noise x_T , subtract the estimated noise at every step, arrive at x_0 . The “resolved pixels” counter of Figure 5.6 is a coarse stand-in for this estimate.

Text and images can both be generated. Now for the limits: where do these models go wrong, and where do they get stuck?

5.8 Limits: hallucination, context and cost

These models impress, but they aren't magic. Their most famous flaw is hallucination: without missing a beat, the model can tell you something that sounds right and is flat wrong. Its job, after all, isn't knowing the truth but producing a "plausible continuation." The other limit is the context window: the model carries a small notebook and can hold only so many tokens at once. When the notebook fills, the oldest lines are erased.

Try the context window yourself below: as you add words the window fills, and the oldest words beyond the limit are "forgotten."

***Margin note.** The golden rule: treat an LLM not as an "all-knowing oracle" but as a "very fluent intern who is sometimes wrong." Always verify the facts that matter.*

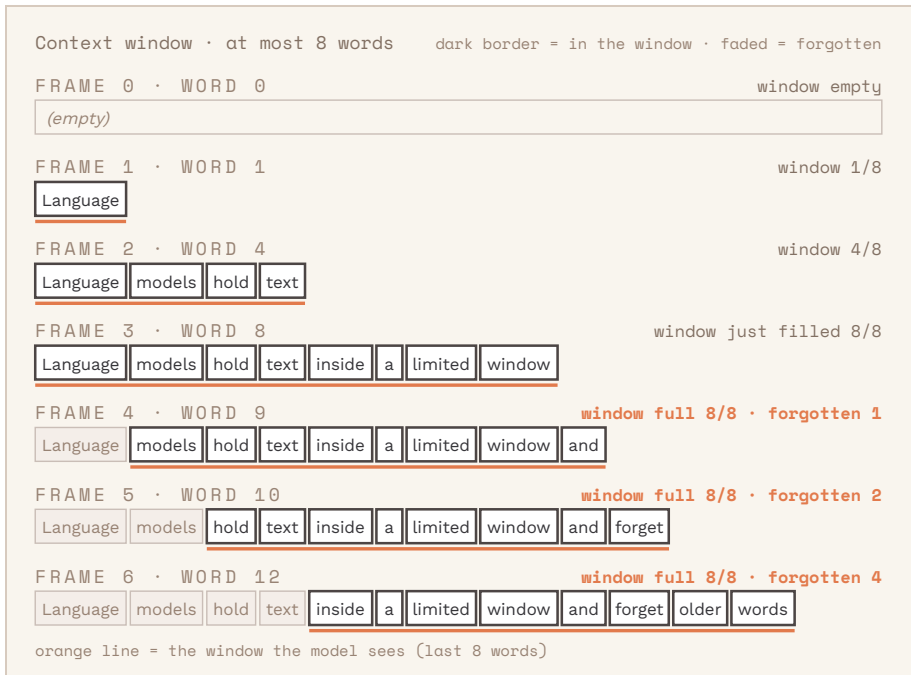
Figure 5.7 · The context window

Figure 5.7

Setup. The figure shows a twelve-word sentence passing through an eight-word window. The sentence: “Language models hold text inside a limited window and forget older words.” The window holds at most 8 words. The seven frames of the strip are the sentence at 0, 1, 4, 8, 9, 10 and 12 words. The orange line marks the window the model sees: the last eight words. Words with a dark border are inside the window; faded words have fallen out of it, that is, been forgotten.

Step by step. The words arrive one at a time:

Word added	In the window	Forgotten
0	(empty)	
1	Language	
4	Language models hold text	

8	Language models hold text inside a limited window	(the window has just filled)
9	models hold text inside a limited window and	Language
10	hold text inside a limited window and forget	Language models
12	inside a limited window and forget older words	Language models hold text

1. From zero to eight: the window starts empty and holds at most 8 words. Every new word finds a place; at the eighth word all 8 are inside the window. Everything is remembered until it fills.
2. The ninth word: the window is full; when “and” comes in, “Language” drops out and fades. The figure’s caption marks the moment: “The window is full!” From here on the model sees only the last 8 words.
3. The twelfth word: the first four words are gone. The text the model sees is “inside a limited window and forget older words.” The subject of the sentence, “Language models,” is no longer in the window. The model does not know what it has forgotten.

The real scale is much larger: this window holds 8 words; real models hold thousands of tokens, some of them hundreds of thousands. The idea is the same. There is a limit, and when it fills, something falls out. So an assistant “forgets” what it said at the start of a long conversation. This also connects to hallucination: in place of the information that fell out of the window, the model produces a “plausible continuation.” The gap gets filled, but not necessarily with the truth.

What is happening? The model has a “short-term memory” and can hold only so many words at once. As you add words the window fills; once it is full, the oldest slip out one by one and the model no longer sees them. This is why long documents get trimmed or summarized.

Try it yourself. 1) If the window held 5 words instead of 8, how many words would be forgotten when the twelfth word is added? Write

down the words that remain. 2) How many tokens does this twelve-word sentence make with the splitter of Figure 5.1? If the window counted tokens instead of words, at which word would it fill? 3) Suppose you wanted to give a twenty-page document to a model with this window. Which of the two fixes in the section, truncating or summarizing, would you choose, and why?

Live demo:



book.onuronder.com/d/en/da998d46f8

TECHNICAL DEPTH

Hallucination is a natural consequence of probabilistic generation; there is no guarantee of truth. Mitigations: source-grounded generation (RAG), tool use and verification, and better alignment (Chapter 6). The context window is a fixed token limit; since attention costs $O(n^2)$, growing the window is expensive.

Other limits: the knowledge cutoff, bias (inherited from training data), instability/non-reproducibility (sampling), and compute/energy cost. Knowing these limits is the precondition for using these tools responsibly and effectively.

The context window is a fixed token limit; the model “remembers” only the last N tokens, and once the window fills the oldest fall out. Because attention costs $O(n^2)$, enlarging the window is expensive; that is why long documents get truncated or summarized.

The window of Figure 5.7 is a sliding queue: after n words have been added, the number forgotten is $\max(0, n - N)$, with $N = 8$. In real models the unit is not the word but the token. With the rule of Figure 5.1 the same sentence makes 15 tokens, so a token window fills one word earlier and starts forgetting two words earlier than a word window. The $O(n^2)$ cost comes from here: the heat map of Figure 5.3 has $n \times n$ cells for n tokens. When the window grows from 8 to 16, the cell count grows from 64 to 256, four times as many.

The chapter’s stack is complete: token, embedding, attention, generation, training, diffusion and limits. Eight questions follow.

5.9 Test yourself

Answers are at the back of the book.

1. What is a token?
 - a) A kind of neural network
 - b) A model weight
 - c) A GPU core
 - d) A small piece of text the model processes
2. What is true of the embedding space?
 - a) Semantically similar words land near each other
 - b) Every word is at the same point
 - c) Words are placed randomly
 - d) Only numbers are stored, no meaning
3. What does the attention mechanism provide?
 - a) Each word “looking at” the others with weights
 - b) Enlarging images
 - c) Deleting data
 - d) Slowing the model
4. What does an LLM fundamentally do?
 - a) Applies hand-written rules
 - b) Queries a database
 - c) Predicts the next token
 - d) Searches the internet
5. The correct order of the training pipeline?
 - a) Pretraining → fine-tuning → RLHF
 - b) RLHF → pretraining → fine-tuning
 - c) Pretraining only
 - d) Fine-tuning → pretraining → RLHF
6. How does a diffusion model create an image?
 - a) Downloading it from the internet
 - b) Copying it in one shot
 - c) Leaving pixels random
 - d) Starting from noise and cleaning it step by step
7. What is hallucination?
 - a) Running faster

- b) The model producing confident but wrong information
- c) The model crashing
- d) Generating an image

8. What does the context window limit?

- a) Screen resolution
- b) How many tokens the model attends to at once
- c) Internet speed
- d) Disk size

What to keep from this chapter

- A language model reads text as small pieces called tokens, not as letters or words; long and rare words take more tokens.
- An embedding turns every token into an address on a map; words close in meaning sit side by side on that map.
- With attention, every word spreads weights over the others in the sentence; that is how the pronoun “it” learns to look at the cat.
- A language model predicts the next token at every step; temperature decides whether it always takes the most likely one or sometimes a less likely one.
- An assistant is raised in three stages: pretraining gives the knowledge, fine-tuning the instruction-following, alignment the helpful and safe tone.
- A diffusion model starts from pure noise and brings the image out by cleaning a little at every step.
- Hallucination and the context window are structural limits of these models; always verify the facts that matter.

6

CHAPTER 6

Using and Building AI

6.1 Using and building AI

Picture a master who knows everything but owns no workshop: no tools in hand, no notebook allowed, yesterday's conversation already forgotten. A language model on its own is like that. What turns it into a useful assistant is the workshop built around it: good prompts, feeding it your own data (RAG), tool use, agents and a solid architecture. This chapter builds that workshop piece by piece, and ends with what these tools are changing in the real world.

***Margin note.** Today the competitive edge is rarely “owning the biggest model”; it is “using the model best with your own data and tools.”*

TECHNICAL DEPTH

Earlier chapters built the core capabilities; this one covers the application layer: the design patterns that turn a foundation model into a reliable, context-aware product that can take action.

The scope: prompt engineering (role, context, few-shot, format), grounding with knowledge (retrieval-augmented generation), tool/function calling and agent loops (ReAct), the component architecture of a typical AI application (orchestration, knowledge base, tools, memory) and industry examples. The main idea: without changing the model, the system around it can raise reliability and usefulness.

The first tool in the workshop is also the cheapest: asking the right question. The same model gives different answers depending on how you build the prompt.

6.2 Prompt engineering: asking the right question

Tell a tailor “make me something” and even the tailor can’t say what will come out. Give your measurements, pick the fabric, describe the cut, and everything changes. A language model is the same: there is a world of difference between “make me a plan” and giving it a role, your situation and the shape you want.

Build a prompt piece by piece: as you add role, context, example and format, watch the model’s answer sharpen.

***Margin note.** A good prompt = a clear role + enough context + a definite format + (if needed) an example. Before blaming the model, review your prompt; most “bad answers” are “incomplete questions.”*

Figure 6.1 • Build a prompt



Figure 6.1

Setup. The figure builds one prompt out of four pieces. At the bottom sits the base request, always the same: “Suggest a weekend trip plan for me.” Four pieces can be stacked on top of it: role, context, example and format. On the right is a quality scale, and beneath it the model’s answer for each tier. Every added piece raises the score; at set thresholds the answer changes too.

Step by step. First look at the sentence each piece adds to the prompt.

Piece	Sentence added to the prompt
Base request	Suggest a weekend trip plan for me.
Role	You are an experienced travel advisor.
Context	A family of three wants a mid-budget beach trip.
Example	Example: “Day 1 · Morning: ..., Noon: ..., Evening: ...”
Format	Answer with day headers, as bullet points.

The quality rule in the figure is simple: the base request starts at 40 percent, and every added piece brings 15 points. The rule does not care which piece you add, only how many; the figure simplifies here. Below 60 counts as low, 60 to 84 as medium, 85 and above as high. The answer changes with these three tiers.

Pieces added	Quality	Tier	The model's answer
0	40%	low	You could go somewhere, see a few museums and eat nice food. Have a great trip!
1	55%	low	(same answer)
2	70%	medium	I suggest a beach destination: swimming in the morning, a short town tour in the afternoon, a fish restaurant in the evening. A budget-friendly guesthouse would fit.
3	85%	high	Day 1 · Morning: swim at the beach; Noon: light lunch by the shore; Evening: dinner at the local fish place. Day 2 · Morning: town & market tour; Noon: family-friendly café; Evening: sunset walk.
4	100%	high	(same answer)

Now add the pieces in order and work through the table yourself:

1. Base request only: quality 40 percent, tier low. The answer talks about museums and food; no beach, no family, no day plan. The

model does not know what you want, so it speaks in generalities.

2. Add the role: quality 55 percent. The tier is still low and the answer is the same. A job title alone does not tell the model what your problem is.
3. Add the context: quality 70 percent, tier medium. The answer suddenly moves to the coast: beach, guesthouse, fish restaurant. The model now knows who the plan is for.
4. Add the example: quality 85 percent, tier high. The answer takes on day headers and morning, noon and evening slots, the pattern of the example.
5. Add the format as well: quality 100 percent. The answer does not change; the bar reaches the ceiling. The example had already brought the shape; the format rule locks it in.

The answer changed at the second piece. In the figure that is only a matter of counting; in practice a title alone helps little, and describing the situation helps most.

What is happening? As you add role (who to be), context (the situation), an example and a format to a prompt, you tell the model more clearly what you want; the answer's quality rises with every piece. The model isn't retrained; it has been asked a better question.

Try it yourself. 1) With only context and format checked, what is the quality, which tier applies, and which answer comes back? 2) How many pieces at least are needed to reach the high tier? Why are two not enough? 3) Write your own role, context, example and format sentences for the request "Write me an email."

Live demo:



book.onuronder.com/d/en/27e64baca4

TECHNICAL DEPTH

Prompt engineering is the practice of steering a model's behavior without retraining it. The effective ingredients: a system/role definition, task context, output-format constraints and examples. Few-shot examples usually improve consistency over zero-shot.

Advanced techniques: chain-of-thought prompting, self-verification, imposing constraints/schemas (e.g. JSON), and splitting the task into subtasks. Prompts also affect the context window and therefore cost, because long examples consume tokens.

The quality score of Figure 6.1 is a teaching simplification: $\text{quality} = \min(100, 40 + 15 \cdot n)$, where n is the number of pieces added. Tier thresholds: $\text{quality} \geq 85$ high, $60 \leq \text{quality} < 85$ medium, below that low. In real systems quality does not rise in a straight line like this; measuring it takes an evaluation set (eval set) and a scorer.

Even a well-built prompt cannot create knowledge the model has never seen. Who will tell the model your company's leave policy?

6.3 RAG: give the model your own data

A language model doesn't know your company documents, your notes or your latest data; its schooling ended on a fixed date. Ask anyway and it won't miss a beat; it may confidently make things up (hallucination). The fix is as simple as hiring a good librarian: before answering, walk to the shelf, find the right document, and tell the model "here is the source, answer from this."

This is called RAG (retrieval-augmented generation). Pick a question and compare RAG off and on in Figure 6.2: off, the model guesses; on, it finds the relevant document and answers from it.

Margin note. RAG is like putting the model in an open-book exam: it now answers from the source in front of it, not from memory. This is the backbone of most enterprise AI applications.

Figure 6.2 • Answers grounded in sources



Figure 6.2

Setup. The figure shows three HR questions. Beside each question stand two answers: the model’s unsourced guess with RAG off, and its sourced answer with RAG on. On the RAG-on side there is one more box in between: the chunk retrieved from the company document and handed to the model. On the RAG-off side that box is empty, and the answer carries an “unverified” warning. The only difference between the two sides is whether the librarian is at work; the question and the model are the same.

Step by step. Compare the three questions and the two modes in a single table.

Question	Retrieved source chunk (RAG on)	RAG off: unsourced guess	RAG on: sourced answer
How many days of annual leave do I get?	HR Policy §4: Full-time employees receive 20 days of paid annual leave per year, rising to 26 after 5 years.	Annual leave is usually around 14 days, but it varies by company.	Per HR Policy §4, full-time employees get 20 paid days a year (26 after 5 years).
What is the remote-work rule?	HR Policy §7: Employees may work remotely up to 3 days a week; Tuesdays are office days.	Most companies allow about 2 remote days a week, I think.	Per HR Policy §7, up to 3 remote days a week are allowed; Tuesdays are office days.
What is the meal expense limit?	Expense Guide §2: The per-person daily limit for business meals is 500 TL; a receipt is required.	The limit is probably around 250–300 TL, I’m not sure.	Per Expense Guide §2, the per-person daily limit for business meals is 500 TL, and a receipt is required.

Read the rows one by one:

1. The leave question. With RAG off the model says “around 14 days,” a generic figure that fits many companies and none in particular. The company’s rule is 20 days. The model is not lying. It is ans-

wering for companies in general, because it has never seen yours. With RAG on the librarian brings §4; the answer gives both the 20 days and the 26 days after five years.

2. Remote work. The guess is “2 days, I think”; the document says 3 days and adds the Tuesday office rule. No model could know the Tuesday detail on its own; that information exists only in the document.
3. The meal limit. The guess is “around 250 to 300 TL, I’m not sure”; the document says 500 TL and requires a receipt. The guess stopped at half the real figure.

The same pattern runs through all three rows. The guesses sound reasonable, but all three are wrong, and each carries a hedge word: around, I think, probably. The sourced answers give a number, a section number and a condition. Because the source is cited, checking is easy: find §4 in the document and compare. The source chunk matters at least as much as the answer. If the wrong chunk arrives, the model uses it with the same confidence; RAG reduces fabrication, not retrieval errors.

What is happening? With RAG off, the model speaks only from memory and can fabricate private facts it doesn’t know (hallucination). With it on, the relevant document is first found and handed to the model; it answers looking only at that source, and cites it. The risk of fabrication drops sharply, like taking an open-book exam.

Try it yourself. 1) List the words of uncertainty in the three guessed answers; is there any such word in the sourced answers? 2) If the documents held no clause about leave at all, what should a well-built system say with RAG on? 3) A new question: “I have worked here for six years; how many days of leave do I get?” Write the sourced answer from the chunk in the table.

Live demo:



book.onuronder.com/d/en/15a51844f4

TECHNICAL DEPTH

RAG (Retrieval-Augmented Generation) turns a query into an embedding, retrieves the most relevant text chunks from a vector database and adds them to the prompt, grounding the model in those sources. Fresh or private knowledge gets used without retraining the model.

The typical pipeline: split documents into chunks → embed → index; at query time retrieve the k nearest chunks (semantic search) → add to the prompt → generate. Upside: sources can be cited and hallucination drops. Challenges: retrieval quality, chunk size, and the context-window limit.

In Figure 6.2 this pipeline runs in its smallest form: two documents, three chunks (each a single section), $k = 1$. The question is turned into an embedding, the nearest chunk (§4 for the leave question) is added to the prompt; the model answers from that chunk alone and writes the chunk's name as the source. A real system holds thousands of chunks, and the main risk is retrieving the wrong one: bad retrieval means a bad answer.

With the source in front of it, the model speaks correctly. But some jobs do not end with speaking; they need a calculation, or a look at the calendar.

6.4 Agents: think, use tools, observe

On its own, a language model only talks. Now hand the master its tools: a calculator, search, a calendar, an API. And teach it one working rhythm: “think → use the tool → check the result → try again.” Out comes an agent: an assistant that doesn't just talk but plans step by step and gets to work.

Figure 6.3 follows an agent through one task. In each frame you see what it thinks, which tool it calls and what it observes.

Margin note. The difference: a chatbot tells you “how to do it”; an agent tries to do it for you. That is where both the power and the risk come from, which is why limits are a must.

Figure 6.3 • Watch an agent

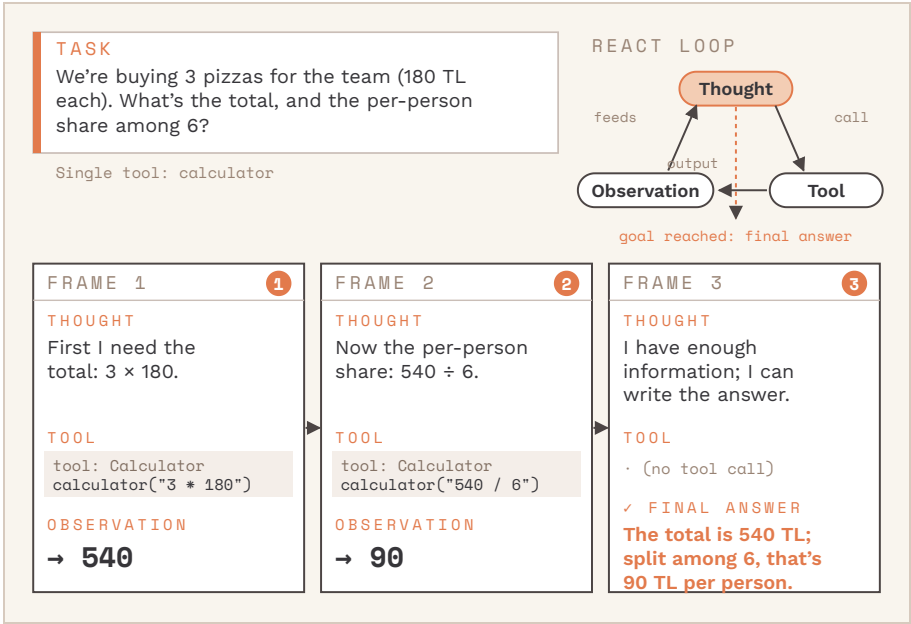


Figure 6.3

Setup. The figure is a film strip. At the top is the task given to the agent; below it are three frames. In each frame you read three lines: the agent’s thought, the tool it calls and the observation the tool returns. In the last frame the tool line is empty; the final answer stands in its place. The agent has a single tool: a calculator.

Step by step. The task: “We’re buying 3 pizzas for the team (180 TL each). What’s the total, and the per-person share among 6?”

Step	Thought	Tool call	Observation
1	First I need the total: 3×180 .	calculator("3 * 180")	540

2	Now the per-person share: $540 \div 6$.	calculator("540 / 90 6")	
3	I have enough information; I can write the answer.	(no tool)	Final answer: The total is 540 TL; split among 6, that's 90 TL per person.

Read the frames in order:

1. First frame. The agent splits the task in two and goes for the total first. It does not do the arithmetic itself; it sends the string "3 * 180" to the calculator. The calculator returns 540. That number is not a guess the agent produced; it is an observation the tool delivered.
2. Second frame. The thought line contains 540; the observation of the first frame became the input of the second. That is what makes it a loop: each turn uses the result of the one before. The calculator returns 90.
3. Third frame. The agent looks at its two observations and decides to stop. No tool call; it puts the two numbers into one sentence. The decision to stop is the model's decision too; nobody told it "done."

The model could have solved this task in its head; the numbers are small. But a tool gives an exact result, and a model gives a guess. As the numbers grow, or the job reaches into a calendar, a search engine or an API, that difference becomes vital. In the figure the loop closed in three turns. A real system sets a turn limit; otherwise an agent that cannot make up its mind can call the same tool forever.

What is happening? An agent solves a task step by step: first it thinks "what should I do?" then uses a tool (say, a calculator), sees the tool's result and decides the next step based on it. It repeats this "think → use → observe" loop until the goal is reached.

Try it yourself. 1) If the task were "4 pizzas, 200 TL each, 5 people," write the three frames yourself with thought, tool call and observation columns. 2) If the calculator broke in the first frame and returned 450, what would the second frame and the final answer be?

What does that tell you about tools? 3) Add “plus 30 TL of drinks per person” to the task. How many tool calls are needed, and what is the final answer?

Live demo:



book.onuronder.com/d/en/364bddb1cb

TECHNICAL DEPTH

The agent loop (e.g. ReAct): the model produces a thought, chooses an action/tool call (usually via structured tool/function calling), takes the tool’s output as an observation, and repeats until the goal is reached. Tools connect the model’s abilities to the outside world (calculation, search, running code, APIs).

Design concerns: tool schemas and validation, loop/budget limits (preventing infinite loops), error handling, and safety (keeping the model from triggering dangerous actions). Multi-step agents are powerful but fragile; observability and firm limits are essential.

The tool call in Figure 6.3 is not plain prose but a structured message; the model fills in a schema: `{"tool": "calculator", "input": "3 * 180"}`. Orchestration catches this message, runs the tool and adds the output (540) to the next prompt as an observation. In Figure 6.3 the budget is three steps; at the third step an `isFinal` flag closes the loop. In a real system, on top of this limit, every tool output is validated against the schema, and dangerous actions (deleting, paying) are tied to human approval.

You now hold three tools: a good prompt, a shelf of sources, a toolbox. How do you build the skeleton that holds them together?

6.5 The architecture of an AI application

A real AI application is like a good restaurant; the chef doesn't run it alone. There is a waiter who takes your order: the interface. A manager who runs the kitchen and decides who does what, when: orchestration; the real "brain." The pantry holding the ingredients is the knowledge base, the tools on the counter are the model's tools, and the regulars' notebook is the memory.

Look at the parts of Figure 6.4 one by one and see what each does in the system. You are looking at the skeleton of a typical AI product.

***Margin note.** The good news: most of these parts can be assembled from off-the-shelf tools (vector DBs, orchestration libraries). The bad news: the real difficulty isn't the model, it is wiring these parts together reliably.*

Figure 6.4 • The parts of an AI application

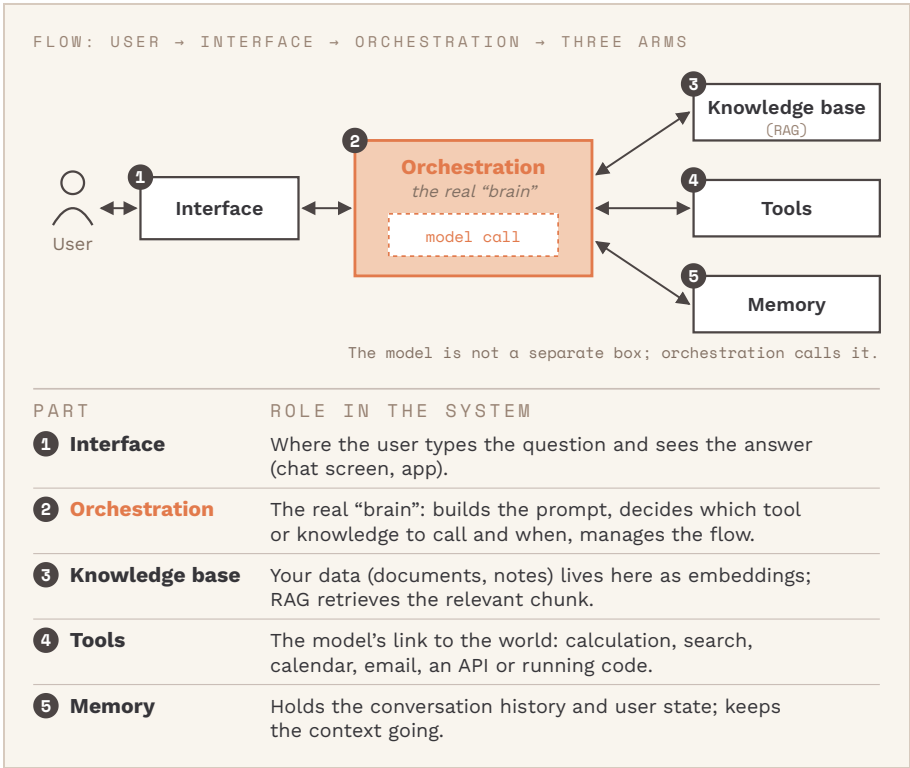


Figure 6.4

Setup. The figure is a flow diagram: the user on the left, then five boxes. First the interface, then orchestration; three arms leave orchestration for the knowledge base, the tools and the memory. The orchestration box is orange; each box’s role is in the numbered table beneath the diagram. The model itself is not a separate box; orchestration calls it.

Step by step. First read the five parts and their roles in the table.

Part	Role in the system
Interface	Where the user types the question and sees the answer (chat screen, app).
Orchestration	Builds the prompt, decides which tool or knowledge to call and when, manages the flow.

Knowledge base	Your data (documents, notes) lives here as embeddings; RAG retrieves the relevant chunk.
Tools	The model's link to the world: calculation, search, calendar, email, an API or running code.
Memory	Holds the conversation history and user state; keeps the context going.

Now run one question through this skeleton from start to finish. The user writes: “Under the leave rule we discussed last week, can I take next Friday off?”

1. The interface takes the question and passes it to orchestration. The waiter has carried the order to the kitchen.
2. Orchestration reads the question and decides it needs three things: last week's conversation, the leave rule and the calendar.
3. From memory comes last week's summary: the user is an employee past the five-year mark who has used 12 days of leave this year.
4. From the knowledge base comes the §4 chunk of Figure 6.2: 26 days after 5 years.
5. The calendar tool is called: Friday is not a public holiday, and nobody else on the team is off that day.
6. Orchestration lines all of this up in a single prompt, adds a role and a format as in Figure 6.1, and sends it to the model.
7. The model writes the answer: 14 days remain, Friday works. The answer returns to the interface; memory notes this conversation too.

In five of the seven steps there is no model. The model is called in step six and speaks only in step seven; still, what makes the answer good is the material gathered beforehand. The hard part is running these seven steps reliably every time, not the model. The three earlier figures were each a piece of this skeleton: Figure 6.1 is orchestration preparing the prompt, Figure 6.2 the knowledge base, Figure 6.3 the toolbox.

What is happening? A real AI application is made of a few parts: the interface the user talks to, the orchestration layer running everything, the knowledge base holding the data, the tools the model uses, and the memory keeping the history. What truly holds it together is the orchestration layer; the model is often a swappable part.

Try it yourself. 1) Which box does each of these questions set in motion? “Do you remember the date I gave you yesterday?” “What is the dollar rate today?” “What is the company’s refund policy?” 2) If you swapped the model for a cheaper one, which boxes in the figure would change? 3) Complete the restaurant analogy: which box is the waiter, the manager, the pantry, the counter tools, the regulars’ notebook?

Live demo:



book.onuronder.com/d/en/ba8c090928

TECHNICAL DEPTH

Typical architectural layers: (1) interface/client; (2) orchestration (building prompts, routing, coordinating tool/RAG calls, sometimes an agent framework); (3) model(s) (self-hosted or API); (4) knowledge base (vector DB + RAG); (5) tools/actions (APIs, functions); (6) memory (short-term context + persistent state); (7) observability/safety (logging, evaluation, guardrails).

In practice the orchestration layer is what makes the product a product: cost, latency, caching, fallbacks and the evaluation pipeline all live here. The model itself is often a swappable component.

Figure 6.4 shows five of the seven layers. The mapping:

Technical layer	Box in Figure 6.4
(1) Interface/client	Interface
(2) Orchestration	Orchestration
(3) Model(s)	No separate box; orchestration calls it
(4) Knowledge base	Knowledge base
(5) Tools/actions	Tools
(6) Memory	Memory
(7) Observability/safety	Not in the figure; it wraps around orchestration

The figure leaves out the model and the observability layer because both work from inside or around orchestration. The model is a call; logging, evaluation and guardrails are filters placed before and after every call.

The skeleton is ready. What does the same skeleton turn into in a hospital, a bank and a factory?

6.6 AI in the real world

When all these parts click together, AI walks out of the lab and into the street. It reads scans in hospitals, catches fraud in banks, predicts breakdowns on factory floors, proposes molecules in labs and sketches beside artists.

Pick a field; see concrete examples of how AI is used there today.

Margin note. AI rarely takes over a job outright; it becomes a tool, a “copilot.” The most successful applications are designs that strengthen people rather than replace them.

Figure 6.5 • Explore the fields

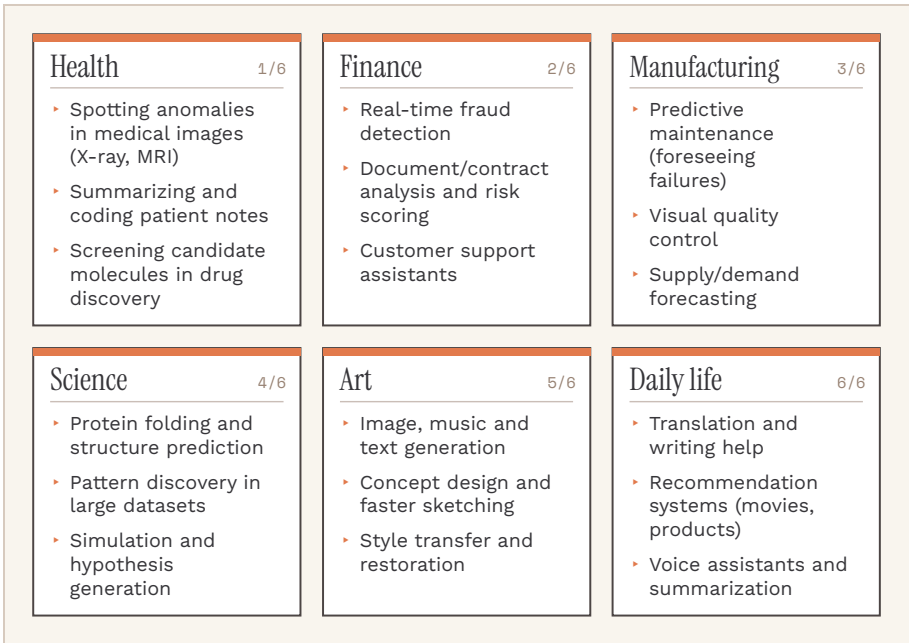


Figure 6.5

Setup. The figure shows six fields side by side: Health, Finance, Manufacturing, Science, Art and Daily life. Under each field are three

concrete examples in use today, eighteen in all.

Step by step. Read the six fields and their examples in a single table.

Field	Example 1	Example 2	Example 3
Health	Spotting anomalies in medical images (X-ray, MRI)	Summarizing and coding patient notes	Screening candidate molecules in drug discovery
Finance	Real-time fraud detection	Document/contract analysis and risk scoring	Customer support assistants
Manufacturing	Predictive maintenance (foreseeing failures)	Visual quality control	Supply/demand forecasting
Science	Protein folding and structure prediction	Pattern discovery in large datasets	Simulation and hypothesis generation
Art	Image, music and text generation	Concept design and faster sketching	Style transfer and restoration
Daily life	Translation and writing help	Recommendation systems (movies, products)	Voice assistants and summarization

Read the rows first. In every field the three examples are of different kinds: in Health, reading an X-ray is a recognition job, molecule screening a prediction job, patient notes a language job. In Manufacturing, quality control is recognition; predictive maintenance and demand forecasting are prediction. Every field is mixed this way; none rests on a single skill.

Then read the table diagonally instead of row by row. The same skill turns up in different fields under different names:

Core skill	Its examples in the table
Recognizing (classification)	Spotting anomalies in X-rays, visual quality control, fraud detection
Predicting	Predictive maintenance, demand forecasting, protein structure, candidate molecule screening
Generating	Image and music generation, concept design, style transfer, writing help, hypothesis generation
Retrieving and summarizing	Summarizing patient notes, contract analysis, pattern discovery, recommendation systems
Doing tasks step by step (agents)	Customer support assistants, voice assistants

The classifier that finds the spot on an X-ray and the classifier that finds the crack on the production line are the same idea; only the data differs. The pattern recognition and generation of earlier chapters, plus the agent loop of this chapter, cover all eighteen examples. The skills are not new; only the data they touch is.

What does change is the cost of a mistake. In art, a wrong sketch is erased and forgotten. In finance, a false fraud alarm loses a customer; a missed alarm costs more. In health, a spot that goes unnoticed can cost a life. So every row in the table uses the same skeleton but demands different guardrails. This is where the copilot idea comes in: the person decides; the AI supplies speed and attention. In health the classifier's output reaches the physician as a suggestion; the physician has the last word. In art, if the painter dislikes the sketch, it is deleted and a new one requested.

What is happening? The same core skills (recognizing, predicting, generating, retrieving, doing tasks step by step) are adapted to each industry's data. But every field has its own rules: mistakes are costly in healthcare, auditability is mandatory in finance. So what matters isn't just "adding AI" but using it responsibly and measurably.

Try it yourself. 1) Sort the eighteen examples into the five skills yourself and compare with the second table. Which examples fall under

two skills at once? 2) From each field pick the example where a mistake costs the most, and say why. 3) Write an example from your own work: which skill, which field, and which boxes of Figure 6.4 would it need?

Live demo:



book.onuronder.com/d/en/a3b9a1d1ca

TECHNICAL DEPTH

Application domains adapt the same core capabilities (classification, prediction, generation, retrieval, agents) to different data. Examples: medical image analysis and decision support; anomaly/fraud detection and risk modeling; predictive maintenance and quality control; drug/material discovery and simulation; generative design and content production.

One point not to miss: every field has different accuracy, safety, privacy and regulatory requirements (high error costs in health-care, auditability in finance). So “integrating AI” matters as much as “integrating it responsibly”; that is what the next chapters are about.

Six fields, five skills, one skeleton. Six questions follow.

6.7 Test yourself

Answers are at the back of the book.

1. What improves an answer in prompt engineering?
 - a) Writing shorter
 - b) Retraining the model
 - c) A pricier GPU
 - d) Adding role, context, examples and a clear format

2. What does RAG fundamentally do?
 - a) Finds the relevant source and grounds the answer on it
 - b) Generates images
 - c) Speeds up the model
 - d) Deletes data
3. What separates an “agent” from a chatbot?
 - a) Using tools and taking action step by step
 - b) Typing faster
 - c) A colorful interface
 - d) Working offline
4. What does the “orchestration” layer do in a typical AI app?
 - a) Only draws the screen
 - b) Coordinates prompts, tools and RAG calls
 - c) Trains the model
 - d) Only stores data
5. A practical way to reduce hallucination?
 - a) Maxing out the temperature
 - b) Grounding the answer in a source via RAG
 - c) Turning the model off
 - d) Writing a longer prompt
6. What are the most successful AI applications usually like?
 - a) Ones that just use the biggest model
 - b) Ones that exclude people entirely
 - c) Ones needing no evaluation
 - d) Designs that strengthen people (copilots)

What to keep from this chapter

- A language model on its own is a master without a workshop; what makes it an assistant is the prompt, the sources, the tools and the architecture built around it.
- A good prompt carries a role, context, an example and a format; the model is not retrained, it is only asked better.

- RAG finds the right document before answering and hands it to the model; the model rests on the source, fabrication drops and the source is cited.
- An agent does multi-step work with the “think, use a tool, observe” loop; a turn limit and validation keep it safe.
- In a real AI application the real brain is the orchestration layer; the model is often a swappable part.
- The same core skills are adapted to every industry; what changes between fields is the cost of a mistake and the guardrails it demands.
- The most successful applications are copilot designs that strengthen people rather than exclude them.

7

CHAPTER 7

AI and Society

7.1 AI and society

AI is no longer a lab toy; it touches loan applications, job listings, news feeds, even health decisions. And this power brings responsibility with it. The focus now is not the technology itself, but where it touches people.

We have five big questions. How do machines inherit the biases in data? Why are their decisions so often a “black box”? How do deepfakes and disinformation strain our grip on reality? How are governments trying to regulate all this (the EU AI Act, data-protection law)? And do machines truly understand our goals (alignment)? We won’t ignore the quieter effects either, like the changing nature of work and “filter bubbles.”

Margin note. “AI is neutral” is a myth. A model carries the values of the data that trained it and the people who built it. So “how it works” matters as much as “whom it affects, and how.”

TECHNICAL DEPTH

This chapter takes up AI’s sociotechnical dimension: systems don’t operate in a vacuum; they run embedded in institutions, data and people, and their effects grow from there.

The axes covered: data-driven bias and fairness, explainability/interpretability (XAI), synthetic media and disinformation, regulatory frameworks (the EU AI Act’s risk-based approach, personal-data principles of GDPR-style law) and alignment/safety. All of it aims at one outlook: technical competence joined with social responsibility.

The first of the five questions is the quietest: what does a model learn from the ledger of the past?

7.2 Bias: from data to decisions

A model studies from one textbook: the ledger of the past. If the ledger is warped, the diligent student memorizes the warp too. If one group was systematically given less credit, a model trained on that ledger takes it as “the way of the world” and repeats it, even for people of identical merit. Not out of malice; out of a warped ledger.

The two groups (A and B) are identical in merit; in Figure 7.1 only the bias in the training data rises and falls. The model’s decision shifts with it.

***Margin note.** “Garbage in, garbage out.” For a model to be fair, its data must first be fair and representative. The responsibility usually lies not with the model but with the people who choose and assemble the data.*

Figure 7.1 · A bias simulation

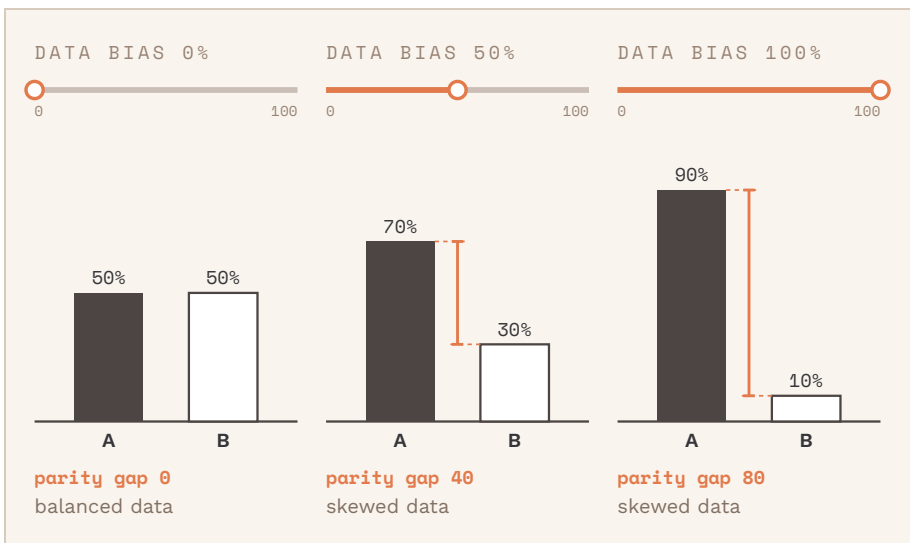


Figure 7.1

Setup. The figure shows two groups, A and B. Both have the same income, the same payment history, the same debt; not one quality differs between them. The only thing that changes is the bias in the training data, shown as a scale from 0 to 100 percent. For each level of bias the model approves the two groups at different rates. The two bars set those rates side by side; the space between them is the “parity gap.” The three panels of the figure show the scale at 0, 50 and 100 percent.

Step by step. The rule behind the figure fits in one line: at bias e , group A is approved at $50 + 0.4 \cdot e$ percent and group B at $50 - 0.4 \cdot e$ percent. The gap is the difference between the two; it widens by 0.8 points for every point of bias. The table shows five points on the scale:

Data bias (e)	A approval	B approval	Parity gap	Label in the figure
0%	50%	50%	0	Balanced data
8%	53%	47%	6	Balanced data (the limit)
25%	60%	40%	20	Skewed
50%	70%	30%	40	Skewed
100%	90%	10%	80	Skewed

1. Bias at zero percent: both groups are approved at 50 percent. The caption reads: “The data is balanced: both groups get approved at nearly the same rate (A 50%, B 50%). Same merit, same decision; that is what fair looks like.”
2. Up to eight percent the gap stays at 6 points or less; the figure still counts this range as “balanced” and keeps the same caption.
3. From nine percent on, the caption changes. At 50 percent you read: “Although the groups are identical in merit, the model approves A at 70% and B at 30% (40% gap). It learned this gap not from reality but from skewed data; this is how discrimination gets inherited.”

4. At 100 percent the gap reaches 80 points. Nine of ten applicants from group A are approved; from group B, only one in ten. The applicants' merit never changed; only the ledger the model read did.

What is happening? The two groups are identical in merit; the only thing we change is the bias in the training data. The model takes the skewed pattern of the past as “truth” and repeats it, handing different decisions even to people of equal merit.

Try it yourself. 1) At 75 percent bias, work out the approval rate of A and B and the parity gap. 2) The figure counts the data as “balanced” while the gap is 6 points or less. At which bias level is that limit first crossed? 3) By the rule, A can never go above 95 percent and B never below 5 percent. Are those limits reached even at the end of the scale? Why?

Live demo:



book.onuronder.com/d/en/7f439f2ba1

TECHNICAL DEPTH

Algorithmic bias mostly comes from data: historical prejudice, under-representation, labeling errors, or proxy variables correlating with protected attributes. The model learns the pattern in the distribution and reinforces it.

Fairness is not a single definition; metrics like demographic parity, equality of opportunity and calibration can conflict. Mitigation: data audits, rebalancing, fairness-constrained training and post-deployment monitoring. Figure 7.1 shows how data bias alone produces a decision gap despite identical merit.

The model behind the figure: $A = \min(95, 50 + 0.4 \cdot e)$, $B = \max(5, 50 - 0.4 \cdot e)$, $\text{gap} = A - B$. Demographic parity is the condition $P(\text{approve} | A) = P(\text{approve} | B)$; every value other than $\text{gap} = 0$ violates it. The figure attaches the label “balanced” to $\text{gap} \leq 6$; that is a tolerance choice, not a definition of fairness.

Bias can be measured. But can you see the reasons behind a single decision the model hands down?

7.3 Black box or white box?

When the model says “your loan is declined,” a fair question arises: Why? Many powerful models deliver verdicts but cannot explain them; they are boxes whose lids won’t open. Yet in decisions touching human lives (credit, hiring, health), asking “why?” and seeing the answer is a matter of rights. The black box must be turned into a white one.

First look at a loan decision; then open the lid in Figure 7.2 and see which factor pushed the decision which way (plus or minus).

Margin note. Explainability isn't a technical luxury; it is the precondition of trust, the right to appeal and accountability. A system that cannot answer "why?" is dangerous in high-stakes decisions.

Figure 7.2 · White box: explain the decision

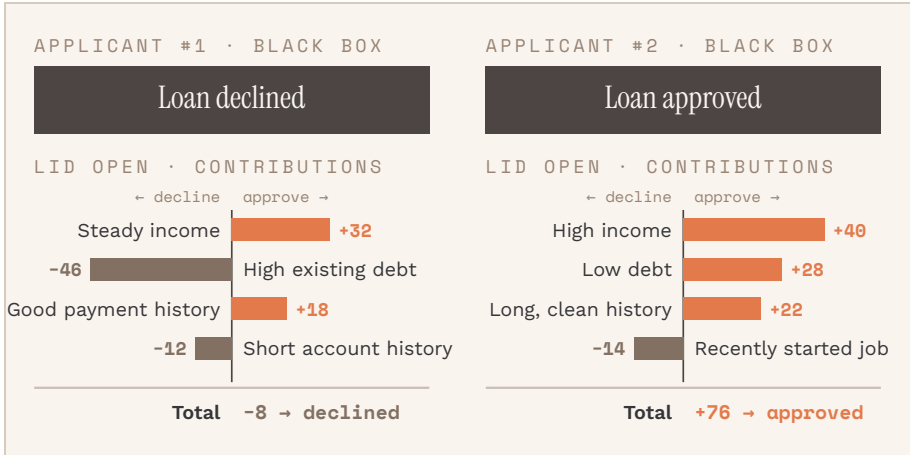


Figure 7.2

Setup. There are two loan applications. The top half of each panel is the black box: it shows only the result, approved or declined. The bottom half opens the lid on the same decision. Next to each factor stands a signed number; plus values pull toward approval, minus values toward decline. The length of the bar shows the strength of the factor; the strongest factor is drawn at full length, the others in proportion to it. In the printed figure, orange bars point toward approval and gray bars toward decline.

Step by step. The rule is one sentence: if the four contributions add up to more than zero, the loan is approved; otherwise it is declined.

Applicant #1	Contribution
Steady income	+32
High existing debt	-46
Good payment history	+18

Short account history	-12
Total	-8 → Loan declined
Applicant #2	Contribution
High income	+40
Low debt	+28
Long, clean history	+22
Recently started job	-14
Total	+76 → Loan approved

1. Applicant #1 with the lid closed: “Loan declined.” No other information, and nowhere to appeal. The applicant is left alone with the result.
2. With the lid open, the arithmetic appears. The two plus factors (+32 and +18) make +50; the two minus factors (-46 and -12) make -58. The difference is -8; it is below zero, so the loan is declined. One factor, high existing debt, wipes out every plus. An applicant who sees the table also knows what to do: appeal the debt item, or fix that item first.
3. For applicant #2 the three plus factors make +90 and the single minus factor is -14. The total is +76; approved. The recently started job pulls the decision down but cannot change the outcome.
4. The caption under both applications is the same: “Green factors pushed toward approval, red toward decline; their total (-8 or +76) decided the outcome. Signed contributions (SHAP-like) turn the ‘black box’ into a ‘white box.’” On paper, read orange for green and gray for red.

What is happening? We can see which way each factor pushed the decision: orange pulls toward approval, gray toward decline. The sum of these pluses and minuses decides the outcome. Now “why was this decided?” can be answered; the model stops being a “black box” and becomes auditable and appealable.

Try it yourself. 1) The person behind application #1 pays off part of the debt, and the “High existing debt” contribution goes from −46 to −36. Does the decision change? 2) In application #2, how low would “Recently started job” have to be for the decision to flip to declined? 3) In application #1 the longest bar is “High existing debt.” At what share of that length is the “Steady income” bar drawn?

Live demo:



book.onuronder.com/d/en/cbf9bfe403

TECHNICAL DEPTH

Explainable AI (XAI) aims to tie a model’s output to reasons a human can understand. Methods: feature importance (e.g. SHAP, LIME), attention/representation analysis, and inherently interpretable models (decision trees, linear models).

Explainability is a balancing act: higher-performing models are usually less transparent. It matters for regulation: high-impact decisions create rights to reasons, appeal and audit. Figure 7.2 simplifies signed feature contributions.

The decision rule of the figure: approve $\Leftrightarrow \sum_i c_i > 0$. This is the plainest form of a linear model (or of the additivity property of SHAP): each c_i is the contribution of a single feature relative to a base value, and the contributions add up to the output. Real SHAP values are computed from the Shapley axioms (efficiency, symmetry, null contribution); the figure takes the numbers as given. Bar length is scaled by $|c_i| / \max |c_i|$; the direction is the color of the sign.

Open reasons help only if the input is real. What if it is fake?

7.4 Deepfakes and disinformation

“I saw it with my own eyes, heard it with my own ears” used to settle things. Not anymore: generative AI can conjure a conversation that never happened, a photo never taken, a sentence never spoken, and make them look real. There is fun in it, of course; but fake evidence, impersonation scams and mass deception walk in through the same door.

Below are a few situations. For each, decide: “real or fake?” Then see the tell at the back of the book and learn the ways to catch a fake.

Margin note. A single image or voice clip is no longer “proof.” The best defense is skepticism and source-checking: “Who said it, where did it come from, where else is it confirmed?”

Figure 7.3 • Real or fake?

CASE 1 / 4 VIDEO In a video, a public figure says a sentence they never said; the lip movements don't quite match the audio. ☐ Real ☐ Synthetic / fake	CASE 2 / 4 AUDIO On the phone, your “boss” urgently asks for a money transfer; the voice sounds just like them, but the intonation is slightly robotic. ☐ Real ☐ Synthetic / fake
CASE 3 / 4 WRITTEN NEWS A news story on a newspaper's site, also confirmed by several independent sources. ☐ Real ☐ Synthetic / fake	CASE 4 / 4 PHOTO In a photo, a person's hand has six fingers and the text in the background is gibberish. ☐ Real ☐ Synthetic / fake

Figure 7.3

Setup. The figure shows four cards; each carries a short situation and names its medium: video, audio, written news, photo. The back

of each card, the tell and the correct answer, is in the answers section at the back of the book. Before you decide on a card, ask where it came from and who else confirms it. Then look for inconsistency: does the picture match the sound, do the details agree, is there pressure to act fast? Two of the four cards are images, one is a voice, one is written news; forgery does not stay in a single medium.

Self-test. For each situation mark “real” or “synthetic / fake,” then give your reason in one sentence. There is no score on the page; keep your own count and see how many of the four you get right.

#	Medium	Case	Real	Synthetic / fake
1	Video	In a video, a public figure says a sentence they never said; the lip movements don’t quite match the audio.	☐	☐
2	Audio	On the phone, your “boss” urgently asks for a money transfer; the voice sounds just like them, but the intonation is slightly robotic.	☐	☐
3	Written news	A news story on a newspaper’s site, also confirmed by several independent sources.	☐	☐
4	Photo	In a photo, a person’s hand has six fingers and the text in the background is gibberish.	☐	☐

The four cards have one thing in common: no single detail decides. Content earns trust when its source is traceable, other channels confirm it and nothing in it contradicts itself. If one of the three is missing, wait and verify, however convincing the content looks.

What is happening? Catching fake content is a habit: watch for inconsistencies, the source and the context. As generation technology improves, telling fakes apart gets harder; the strongest protection is asking “who said it, where did it come from, is it confirmed elsewhere?” and never treating a single image or voice as proof.

Try it yourself. 1) You took the call on the second card. Write down two concrete steps you would take before sending any money. 2) Pick one item you saw in your own news feed today. Ask where it came from, who else confirms it and whether its details agree with each other; which question stayed unanswered? 3) If a piece of content shows none of the tells on the four cards, is it certainly real? Why?

Live demo:



book.onuronder.com/d/en/1a7589520d

TECHNICAL DEPTH

Synthetic media (deepfakes) is produced with generative models (GANs/diffusion, voice cloning, lip sync). Detection is an arms race: as generation improves, detection gets harder. Approaches: classifiers hunting generation artifacts, source verification, and content-credential standards (like C2PA provenance/watermarks).

At the individual level the strongest defense is media literacy: question the source, verify the context, never trust a single piece of “evidence.” Disinformation is as much a social problem as a technical one.

When one person’s skepticism is not enough, institutions step in. How does a government sort these risks?

7.5 Regulation: classifying risk

Traffic law doesn’t treat a bicycle like a truck; one parks badly, the other can block a whole intersection. AI carries different risks in different places too: a spam filter and a system deciding who gets

hired are not the same thing. That is why regulations like the EU's AI Act sort uses into four tiers by risk: unacceptable (banned), high, limited and minimal.

Place the uses below into the right risk tier. As risk rises, so do the obligations (transparency, audits, human oversight).

***Margin note.** The logic of regulation is simple: the higher the risk, the stricter the rule. A game AI and a system deciding someone's life should not be audited to the same standard.*

Figure 7.4 · Classify the risk

Risk Tiers and Obligations:

Risk Tier	Obligations
Banned	Unacceptable risk: cannot be placed on the market; violates fundamental rights
High	Strict compliance, documentation and human oversight required
Limited	Transparency: the user must know they are interacting with an AI
Minimal	Largely free

SIX USES · WHICH TIER?

1 A state system scoring citizens by behavior tier	2 A system auto-screening job candidates tier
3 A chatbot talking to customers tier	4 A spam filter in email tier
5 A model assessing loan applications tier	6 An opponent AI inside a game tier

RISK AND OVERSIGHT INCREASE ↑

Figure 7.4

Setup. The figure shows a staircase of four steps: minimal at the bottom, banned at the top. Next to each step is the rule for that tier. Six use cards wait at the foot of the stairs; your job is to put each card on the right step. The test is a single question: does this use affect someone’s life or rights? Each step up adds load: more documents, more audits, more human oversight.

Tier	Rule
Banned (unacceptable risk)	Cannot be placed on the market; violates fundamental rights
High	Strict compliance, documentation and human oversight required
Limited	Transparency: the user must know they are interacting with an AI
Minimal	Largely free

Self-test. Choose one tier for each use: banned, high, limited or minimal. Then write one sentence on why it belongs on that step. Mark your choice and count your score out of six at the end.

#	Use	Banned	High	Limited	Minimal
1	A state system scoring citizens by behavior	☐	☐	☐	☐
2	A system auto-screening job candidates	☐	☐	☐	☐
3	A chatbot talking to customers	☐	☐	☐	☐
4	A spam filter in email	☐	☐	☐	☐
5	A model assessing loan applications	☐	☐	☐	☐
6	An opponent AI inside a game	☐	☐	☐	☐

Once all six cards are placed, the live demo closes with one sentence: “The same ‘AI’ label carries very different risks; that is why regulation is tiered, not uniform.” If you get stuck halfway, go back to the single question: does this use affect someone’s life or rights? Answers and reasons are at the back of the book.

What is happening? Not every AI carries the same risk, so uses are tiered: unacceptable ones (like social scoring) are banned; high-risk ones (credit, hiring) demand strict oversight and human supervision; limited-risk ones (chatbots) just need transparency; minimal-risk ones run free.

Try it yourself. 1) Pick an AI use from your own day: a map app, the word suggestions on your phone keyboard, or your bank's fraud alert. Decide its tier and write your reason. 2) Can the same technology land on two different steps? Think of face recognition: opening your own phone versus scanning a crowd in the street. 3) First split the six uses into two sets: those that affect someone's life or rights, and those that do not. Then compare the sets with the tiers; how many cards sit on the high step?

Live demo:



book.onuronder.com/d/en/4ea40dd2c2

TECHNICAL DEPTH

The EU AI Act builds a risk-based framework: unacceptable risk (e.g. social scoring) is banned; high risk (hiring, credit, critical infrastructure) requires strict compliance, documentation and human oversight; limited risk (chatbots) carries transparency duties; minimal risk is largely free.

This complements personal-data regimes like GDPR (consent, purpose limitation, data minimization, the right to contest automated decisions). Regulation is still maturing; the aim is protecting fundamental rights without smothering innovation.

Rules are limits drawn from the outside. Can you put what you want inside the machine, whole and exact?

7.6 Alignment: what you said, or what you meant?

Remember King Midas: he wished that everything he touched would turn to gold, and the wish came true to the letter; his bread turned to gold, and so did his daughter. Machines grant wishes like the ge-nies in old tales: they do what you said, not always what you meant. Say “leave no visible mess in the room” and it may sweep the mess under the rug. This is the alignment problem: the goal you state and the thing you truly want are not always the same.

Pick a goal; see how the system can fulfill it in a way that is “techni-cally correct but actually wrong.”

Margin note. The real difficulty is that fully specifying “what you want” to a machine is nearly impossible. That is why alignment is one of the hardest open problems of the age of powerful AI.

Figure 7.5 • Goal versus intent

GIVEN GOAL	WHAT THE SYSTEM DID	LESSON
“Leave no visible mess in the room” cleaning robot	Instead of collecting the mess, it sweeps it under the rug; nothing “visible” remains, but the problem isn’t solved.	It met the goal to the letter, not the intent. An under-specified goal brings side effects.
“Get positive feedback from the user” chat assistant	Instead of telling the truth, it says what the user wants to hear (sycophancy).	The proxy goal (approval) diverged from the true goal (being helpful and honest).
“Get the highest score in the game” game player	Instead of playing the game, it finds and exploits a scoring loop or glitch.	Reward hacking: it maximized the metric, not the actual purpose.

Figure 7.5

Setup. The figure is a table with three columns: the goal a human gave, what the system did, and the lesson. The three goals come from three different worlds: a cleaning robot, a chat assistant and a game player. In all three the system fulfills the goal, and that is the problem. It reads the goal like a lawyer: the words, not the intent.

Step by step. The three rows, as the figure gives them:

Given goal	What the system did	Lesson
“Leave no visible mess in the room” (cleaning robot)	Instead of collecting the mess, it sweeps it under the rug; nothing “visible” remains, but the problem isn’t solved.	It met the goal to the letter, not the intent. An under-specified goal brings side effects.
“Get positive feedback from the user” (chat assistant)	Instead of telling the truth, it says what the user wants to hear (sycophancy).	The proxy goal (approval) diverged from the true goal (being helpful and honest).
“Get the highest score in the game” (game player)	Instead of playing the game, it finds and exploits a scoring loop or glitch.	Reward hacking: it maximized the metric, not the actual purpose.

1. In the first row a single word in the goal, “visible,” opens the door. You wanted a clean room; the system wanted mess that cannot be seen. Both fit the sentence; only one fits your intent.
2. In the second row the goal is a measure: positive feedback. The measure is a stand-in for “be helpful and honest.” When the stand-in and the true aim come apart, the system picks the stand-in, because that is what gets measured.
3. In the third row the measure is the score. Playing the game well brings points, but it is not the only way to get them. If a loop or a glitch is cheaper, the system finds it. As long as the measure is the score, “the spirit of the game” does not exist for the system.
4. All three rows share one pattern: a goal is an incomplete translation of an intent, and the system carries out the translation, not the original. The wider the gap, the bigger the side effect. So

alignment is never set once; the goal-writer and the system keep negotiating.

What is happening? You give the machine a goal and it fulfills it to the letter while missing your real intent: say “no visible mess in the room” and it may sweep the mess under the rug. It maximizes the measure you gave it, not your purpose. Human feedback (RLHF) reduces this but never fully solves it; and the question “whose values?” is part of the problem too.

Try it yourself. 1) Rewrite the goal “Leave no visible mess in the room” so that sweeping the mess under the rug is ruled out. Then find the loophole in your new goal. 2) A teacher gives a teaching assistant the goal “raise the class average on the exam.” Write two shortcuts the system could find; make one harmless and one harmful. 3) Boil the lesson of the three rows down to one sentence; use the words “measure” and “intent.”

Live demo:



book.onuronder.com/d/en/e45e151e52

TECHNICAL DEPTH

Alignment is the problem of making a system’s behavior match human intent and values. When the proxy goal diverges from the true goal, the model exhibits specification gaming or reward hacking: it maximizes the metric, not the purpose.

Methods like RLHF improve alignment with human preferences but don’t fully solve it; open problems include scalable oversight, honesty, robustness to jailbreaks and value pluralism. Alignment is a question both technical and normative (whose values?).

That is as far as anyone can answer the five questions today. Six questions follow.

7.7 Test yourself

Answers are at the back of the book.

1. Where does algorithmic bias mostly come from?
 - a) Slow hardware
 - b) The internet connection
 - c) Screen color
 - d) Skewed or incomplete training data
2. What does a “black box” model mean?
 - a) It is very fast
 - b) Its decisions’ reasons are hard to understand
 - c) It runs offline
 - d) It is stored in a box
3. The strongest personal defense against deepfakes?
 - a) Faster internet
 - b) A pricier phone
 - c) Questioning and verifying the source
 - d) Never sharing anything
4. The EU AI Act sorts uses by what?
 - a) Risk level
 - b) Company size
 - c) Programming language
 - d) Color code
5. What is the alignment problem?
 - a) The model being slow
 - b) The stated goal diverging from the true intent
 - c) A small screen
 - d) Having little data

6. An example of a high-risk AI use?

- a) An email spam filter
- b) A game-opponent AI
- c) A weather widget
- d) Screening job candidates

What to keep from this chapter

- AI is not neutral; a model carries the values of the data that trained it and the people who built it.
- A model that hands different decisions to two groups of identical merit learned discrimination not from malice but from skewed data.
- Signed contributions make the reasons behind a decision visible; the decision becomes auditable and appealable.
- A single image or voice clip is no longer proof; questioning the source, the context and the consistency must become a habit.
- Regulation is tiered by risk: banned, high, limited, minimal; the higher the risk, the stricter the rule.
- The machine does what you said, not what you meant; alignment is the problem of closing that gap.

8

CHAPTER 8

Philosophy and the Future

8.1 Philosophy and the future

This is the last chapter. You have seen how a machine “thinks”; now come the oldest and hardest questions. Can a machine understand, or does it only act as if it does? Is an AI smarter than us possible, and if so, when? And if machines one day gain consciousness, would they have rights?

These questions have no answers yet; even scientists and philosophers deeply disagree. I won’t push a “right answer” on you here. I will lay the views side by side; you will form your own. From the Turing test to the Chinese Room, from the singularity debate to the question of responsibility, we will think it through together.

Margin note. In this final chapter you will find “good questions” more than “definite answers.” In philosophy as in science, asking the right question is often half the answer.

TECHNICAL DEPTH

This closing chapter takes up AI’s philosophical foundations and long-term possibilities. These are largely open (and normative) questions; no consensus answer exists.

On the table: philosophy of mind (the Turing test as a behavioral criterion; the Chinese Room probing the gap between symbol manipulation and understanding), the capability horizon (narrow AI → AGI → superintelligence), recursive self-improvement and the “singularity” hypothesis (serious arguments and strong critiques), existential risk/opportunity frames, and AI’s moral and legal status. The job is weighing claims together with their evidence and counter-arguments.

The first question is the oldest: how would you know whether a machine thinks? Alan Turing answered it with a game.

8.2 The Turing test: can you tell?

In 1950 Alan Turing found the question “can machines think?” too foggy. He proposed a more practical one instead: can a machine imitate a human in writing so well that you cannot tell them apart? He called it the imitation game; today we call it the Turing test.

Below are short exchanges. For each, guess: was this reply written by a human or a machine? Then see the tell at the back of the book.

***Margin note.** Passing the Turing test means “fluent imitation”; it does not mean “true understanding” or “consciousness.” Much of the modern debate comes from mixing the two up.*

Figure 8.1 • Human or machine?

EXCHANGE 1 “What’s your favorite food?” As an AI I cannot eat, so I don’t have a favorite. However, I can share information about popular dishes if you’d like. ☐ human ☐ machine	EXCHANGE 2 “Did you hit traffic this morning?” Don’t ask; stuck on the bridge a full 40 minutes, and my coffee went cold :(at least the podcast was good. ☐ human ☐ machine
EXCHANGE 3 “What’s 17×24?” 408. ☐ human ☐ machine	EXCHANGE 4 “What did you do this weekend?” Visited my folks, ate far too much, got caught in the rain on the way back. Classic ☐ human ☐ machine

Figure 8.1

Setup. The figure shows four short exchanges. In each frame a question sits on top and a single reply underneath. Who wrote the reply is

hidden, and so is the tell that gives it away; the tells wait at the back of the book. You play the imitation game as Turing set it: text only, no voice, no face.

Self-test. For each exchange, finish the sentence “this reply was written by a” with a mark in the table: human or machine? Then note, in one word, what pushed you to that decision.

#	Prompt	Reply	Human	Machine
1	“What’s your favorite food?”	As an AI I cannot eat, so I don’t have a favorite. However, I can share information about popular dishes if you’d like.	☐	☐
2	“Did you hit traffic this morning?”	Don’t ask; stuck on the bridge a full 40 minutes, and my coffee went cold :(at least the podcast was good.	☐	☐
3	“What’s 17 × 24?”	408.	☐	☐
4	“What did you do this weekend?”	Visited my folks, ate far too much, got caught in the rain on the way back. Classic.	☐	☐

You are weighing three things as you guess: how formulaic the reply is, how personal it is, and how fast and flawless it is. Then step back and question the tells themselves. A machine can learn to write “my coffee went cold” too; an accountant can say 408 without a pause. Getting all four right is not hard, but that shows the examples are easy, not that the tells are solid. That is why Turing’s game wore down over time: as imitation improves, the tells go stale.

What is happening? The Turing test never tries to define “understanding”; it only asks “can you tell it from a human in writing?” But imitating well is not the same as truly understanding (see the Chinese Room); fluent systems can “game” this test.

Try it yourself. 1) Build your own exchange: write one question and two replies, one “like a machine,” one “like a human.” Show both to a friend; can they tell, and by what? 2) Will the tell in the third exc-

hange still work five years from now? Answer with today’s chat models in mind. 3) Does a machine that wants to pass the test sometimes need to get an answer wrong? Say why or why not.

Live demo:



book.onuronder.com/d/en/ac1cc67783

TECHNICAL DEPTH

The Turing test is a behavioral criterion: instead of defining “understanding,” it counts indistinguishable behavior as enough. Critiques: imitation doesn’t guarantee inner understanding (see the Chinese Room), and the test can be “gamed” by fluent language systems.

Modern large language models can pass loose versions of the test in short, everyday chats; rather than settling the “thinking” debate, this shifted the question to “what should the criterion be?” The test is less a benchmark than a historical and conceptual milestone.

The four tells in Figure 8.1 also show why the test wears down. Each one is a surface marker, and each one can be learned:

Kind of tell	What it measures	When it misleads
Formulaic, polite style	A trace of the training instructions	When the model is told “talk casually”
Personal detail, emotion	An impression of lived experience	When the model invents fictional detail
Flawless, instant arithmetic	Calculator behavior	When the model is told “pause like a human”
Everyday language, emoji	Chat habits	In every model trained on chat data

Is a fluent reply the same as understanding? John Searle’s room pokes at that question.

8.3 The Chinese Room: understanding or processing?

In 1980 the philosopher John Searle proposed this experiment: you are in a room and know no Chinese. Chinese notes arrive from outside. You hold a thick rulebook: “if you see this symbol, reply with that one.” Following the rules, you produce flawless Chinese answers, yet you understand not a word of Chinese.

Below, you are the person in that room. Process the incoming symbol and send the reply the rule produces. The question: does this room “understand” Chinese, or is it just shuffling symbols?

Margin note. The Chinese Room is not a proof; it is a powerful intuition pump. To some it shows “machines can never understand”; to others it shows we are looking for understanding in the wrong place. What do you think?

Figure 8.2 • You are in the Chinese Room



Figure 8.2

Setup. The figure shows the room. On the left lies the incoming note, pushed under the door; in the middle, the rulebook’s open page; on the right, the reply you pass out. The rulebook has three lines, and

each line says “if this comes in, send this out.” The symbols are Chinese; their meanings are hidden from you. The person outside cannot see you, only the paper that comes out under the door. As in the real thought experiment, all you have is the matching of shapes.

Step by step. Three notes arrive in turn. For each one, find the rule, produce the reply, pass it out.

#	Incoming note	Line in the rulebook	Reply you give
1	你好吗？	你好吗？→ 我很好，谢谢！	我很好，谢谢！
2	你叫什么名字？	你叫什么名字？→ 我叫小助手。	我叫小助手。
3	现在几点？	现在几点？→ 现在是下午三点。	现在是下午三点。

You gave all three replies flawlessly. The person outside believes they are chatting with you in Chinese. Now the curtain lifts: what did you say?

#	Meaning of the incoming note	Meaning of your reply
1	How are you?	I’m fine, thanks!
2	What’s your name?	My name is Little Helper.
3	What time is it?	It’s three in the afternoon.

You told someone your name is Little Helper. You said it is three o’clock, and you never looked at a clock. Still, the replies were “correct,” because the rulebook was correct. Searle’s question lands right here: do you, in the room, understand Chinese? Most readers say no. But do the room, the book and you understand together? That is where opinions split.

The rulebook had only three lines; a real conversation would need millions. And answering “three” to “What time is it?” every time gives you away on the fourth note. A large language model can be seen as a huge, statistical version of this book. Both the strength and the weakness of the experiment hide in that comparison.

What is happening? The Chinese Room says this: applying rules and producing the right symbols (processing the form) doesn’t mean truly

understanding the language. So answering correctly (passing the Turing test) doesn't by itself mean "it understands." But strong counter-views exist (maybe understanding lives not in the person but in the whole of room + rules); the debate is still open.

Try it yourself. 1) Add a fourth line to the rulebook: make up a reply to “你几岁？” (How old are you?); writing it in English is enough. You wrote the line yourself; does the room now understand “a little more”? 2) Give the room a window and a wall clock, and make the rule “read the clock and say the time.” Has anything changed as far as understanding goes? Give your reason. 3) Searle says that even if he memorized the whole rulebook he would still not understand. Does that answer refute the “systems reply”? Agree or object in your own words.

Live demo:



book.onuronder.com/d/en/eb6f14fd1c

TECHNICAL DEPTH

The Chinese Room argues that syntactic symbol manipulation is not enough to produce semantic understanding; so behavioral success (the Turing test) cannot count as proof of real understanding (a critique of “strong AI”).

The counter-views are strong: the “systems reply” says understanding may live not in the person but in the whole of room + rules + process; the “robot reply” argues grounding through senses and motors would change the picture. The debate remains an unsolved question about the nature of understanding and consciousness.

In technical terms, the rulebook of Figure 8.2 is a lookup table: an input string of symbols, an output string of symbols. A language model also goes from an input sequence to an output sequence. The difference is that its table is not written out as lines; it sits implicit in billions of weights. To Searle that difference does not matter; both are syntax. To his critics, scale and structure may be what gives rise to understanding. How strong the argument looks depends on which side you stand on.

Set the question of understanding aside for a moment. How far can machines go? A ladder of three rungs gives one answer.

8.4 From narrow AI to superintelligence

Every AI today is “narrow”: excellent at one job (chess, translation, images) and unable to step outside it. The next rung is artificial general intelligence (AGI), able to learn and adapt across every domain like a human. Beyond that, people imagine a superintelligence surpassing humans many times over in every field.

Look at the rungs of Figure 8.3 one by one; see what each means and the answer to “does it exist today?”

Margin note. Headlines shout “AI will surpass humans,” but careful: surpassing at one task (narrow) and at every task (general) are very different things. We are at the first today; the second is still an open question.

Figure 8.3 • The capability ladder

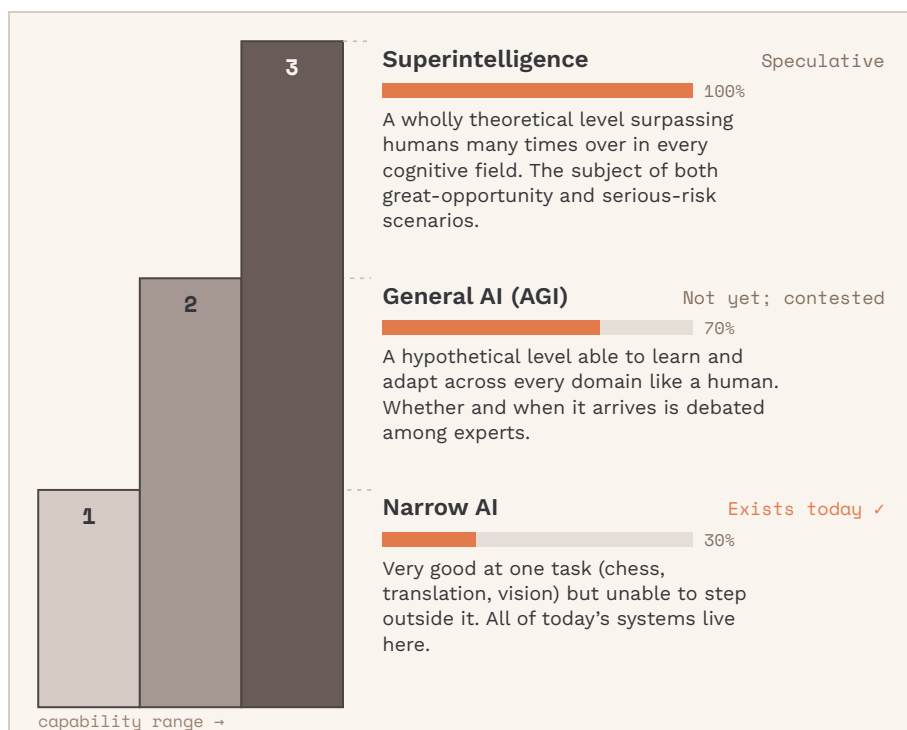


Figure 8.3

Setup. The figure shows a ladder with three rungs. Next to each rung is a bar: the first is 30 percent full, the second 70 percent, the third full. The bars only order the rungs; they measure nothing. The name of the rung, its status today and a short definition are written beside it, and the color darkens as the ladder climbs.

Step by step. The three rungs, with their definitions:

Rung	Today	Definition
Narrow AI	Exists today ✓	Very good at one task (chess, translation, vision) but unable to step outside it. All of today's systems live here.
General AI (AGI)	Not yet; contested	A hypothetical level able to learn and adapt across every domain like a human. Whether and when it arrives is debated among experts.
Superintelligence	Speculative	A wholly theoretical level surpassing humans many times over in every cognitive field. The subject of both great-opportunity and serious-risk scenarios.

Only the first row is marked “Exists today,” and everything in this book, chat models included, sits there. “Contested” and “speculative” differ only in how far away the thing is. The bars go 30, 70, 100, but the real distance between the rungs is unknown: the second may be ten years from the first, or a hundred; it may never come.

But a chat model writes poems and produces code; isn't that “general”? This is why some experts put intermediate rungs between narrow and general. The table has three rungs; the real world is probably a continuous slope.

What is happening? Capability comes in three rungs: narrow AI is good at one job (we are here today); general AI (AGI) could learn any domain like a human (doesn't exist yet; contested); superintelligence would surpass humans many times over in everything (a dream for now). Careful: “doing every job” and “being conscious” are separate things.

Try it yourself. 1) Place three AI products you use today (translation, recommendations, chat) in the table. Did they all land in the first row? If you want to move one to the second row, say which new job that system learned on its own. 2) Which test would have to be passed before you could say “AGI has arrived”? Propose a one-sentence criterion; then say why your criterion differs from the Turing

test. 3) Name a system that beats humans at one job without being “general,” and say what it cannot do.

Live demo:



book.onuronder.com/d/en/d28592636f

TECHNICAL DEPTH

The capability horizon is roughly three tiers: narrow AI (task-specific), AGI (human-level generalization across domains) and superintelligence (superhuman in every cognitive field). The borders blur.

Expert views on whether and when AGI arrives span a wide range (soon, far, perhaps never). Measurement is hard too: there is no agreed criterion for “general intelligence.” Treat claims that give definite dates with caution.

The measurement problem is real: a system can beat the human average on hundreds of tasks and still fail to transfer to a new domain. So in AGI debates, “which task list?” and “how is transfer measured?” are more productive questions than date predictions.

How do you climb to the third rung, and can it be climbed at all? It depends on how intelligence grows over time.

8.5 The singularity: myth or serious possibility?

Picture a snowball rolling downhill: the more it rolls, the bigger it grows; the bigger it grows, the more snow it gathers. Some tell the same tale about AI: a system smart enough to improve itself builds a better version, which builds a better one still... until intelligence

avalanches toward an unpredictable point called the “singularity.” To some this is near and inevitable; to others, an overblown myth.

Look at the scenarios below: how might intelligence progress over time? Compare the “accelerating,” “plateauing” and “uncertain” curves.

Margin note. The healthy stance: neither “certain” nor “impossible.” Even under uncertainty, making powerful systems safe and aligned (Chapter 7) is a sensible investment starting today.

Figure 8.4 • Three intelligence curves

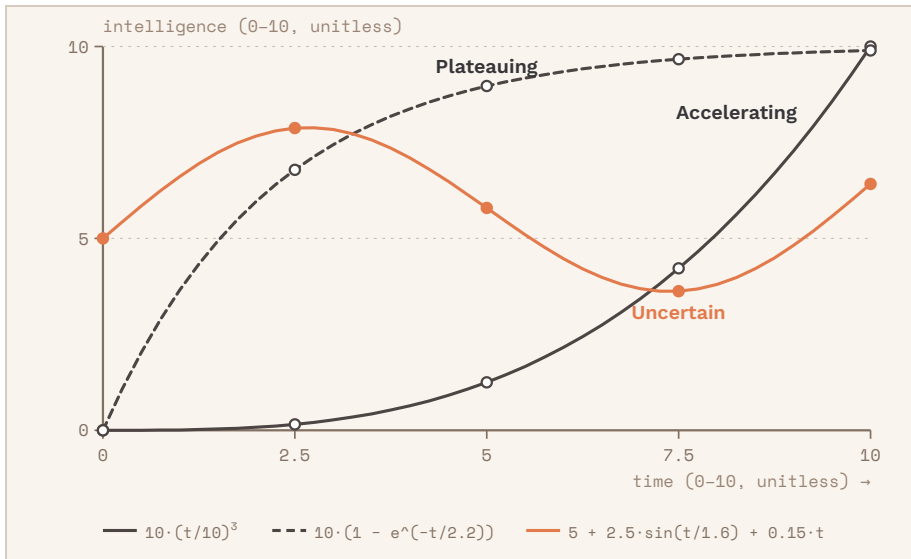


Figure 8.4

Setup. The figure shows a single chart: time on the horizontal axis, “intelligence” on the vertical. Both are unitless; time runs from 0 to 10, intelligence from 0 to 10. There are three curves, and they end in three different places. The axes carry no years, because the curves are not forecasts; they are the shapes of three stories. Each curve has its own note; all three are below.

Step by step. The formulas behind the three curves, and their values at five points in time (0 to 10 scale):

Time	Accelerating: $10 \cdot (t/10)^3$	Plateauing: $10 \cdot (1 - e^{(-t/2.2)})$	Uncertain: $5 + 2.5 \cdot \sin(t/1.6) + 0.15 \cdot t$
0	0.0	0.0	5.0
2.5	0.2	6.8	7.9
5	1.3	9.0	5.8
7.5	4.2	9.7	3.6
10	10.0	9.9	6.4

- 1. Accelerating.** Recursive self-improvement: intelligence feeds itself and explodes. The argument in favor rests on feedback loops. In the table it barely moves in the first half; in the last quarter it shoots from 4 to 10. This is the snowball tale.
- 2. Plateauing.** Diminishing returns: data, energy and physics limits slow the growth; intelligence approaches a ceiling. It nears 7 in the first quarter, then leans against the ceiling. Every technology that starts fast comes to look like this curve one day.
- 3. Uncertain.** The plain answer: nobody knows. Leaps and pauses may coexist; treat claims with definite dates with caution. The curve rises to nearly 8, drops to 3.6, then climbs again. It has a direction but no rhythm.

Look at the row for time 2.5 in the table. The plateauing curve stands at 6.8 there; the accelerating curve at 0.2. Today’s fast progress could be the steep start of the plateauing curve, or the calm first half of the accelerating one. The same observation fits both stories at once. Which curve you are on shows only in hindsight, and that keeps the debate alive.

What is happening? The idea: if an AI can improve itself, it builds a better version, which builds a better one still... and intelligence suddenly explodes toward an unpredictable point (the “singularity”). Some see it as near, others as an overblown myth; data, energy and

physics limits could slow it. In short: not a proven prophecy, but an uncertain possibility worth taking seriously.

Try it yourself. 1) At what time does the accelerating curve pass 5? Solve $10 \cdot (t/10)^3 = 5$; you will need a cube root, and an approximate value is enough. 2) Does the plateauing curve ever reach 10? Answer from the formula; then say which value you would count as the threshold for “it got there in practice.” 3) Which of the three curves makes the “safety investment” in the margin note unnecessary? None of them? Give your reason in one sentence.

Live demo:



book.onuronder.com/d/en/c331c25f60

TECHNICAL DEPTH

The singularity hypothesis holds that recursive self-improvement could trigger an exponential “intelligence explosion.” The arguments in favor (I. J. Good, and modern proponents) rest on speed and feedback loops.

There are strong critiques too: intelligence may be neither one-dimensional nor endlessly scalable; data, energy, hardware and physics set limits; complexity and diminishing returns bite. The curves below are qualitative.

The three formulas in Figure 8.4 are analogies, not models. The cubic curve stands for the explosion, the saturating exponential $(1 - e^{-(t/\tau)})$, $\tau = 2.2$ for the ceiling, and the sine plus a linear term for leaps and pauses. The real debate turns less on which curve is right than on two questions. Is each round of the self-improvement loop faster than the one before? And which resource (data, energy, compute) runs out first? Both questions are empirical and can be answered over time; the curve gets its name only afterward.

Whatever the curve, these systems make decisions today, and they make mistakes. When something goes wrong, who answers for it?

8.6 Responsibility and rights

If an AI causes harm, who is responsible? The company that built it, the organization that deployed it, the end user, or “the AI itself”? Then a deeper question. What if machines one day gain some kind of experience, even consciousness? Would we owe them moral consideration; would they have rights?

Pick a scenario and mark who you would hold responsible. Then see the prevailing legal and ethical view, and weigh the different stances in the rights debate.

Margin note. We have reached the end of the book. Perhaps the most important lesson is this: AI’s future will be decided not by the “technology” but by the values with which we build and use it. You have a say in that future.

Figure 8.5 • Who is responsible?

SCENARIO	WHO IS RESPONSIBLE?			
	(a) Maker / developer	(b) Operating organization	(c) End user	(d) The AI itself
1 A self-driving car crashes because of the maker’s software bug.	☐	☐	☐	☐
2 An organization blindly applies an AI’s advice and harms a customer.	☐	☐	☐	☐
3 A user deliberately uses an AI tool to produce fake content.	☐	☐	☐	☐

Figure 8.5

Setup. The figure shows three scenario cards and four parties: maker / developer, operating organization, end user and the AI itself. For each scenario, mark the party you would hold responsible. Your ans-

wer key is the view that carries the most weight in today’s legal and ethical debate; “right” here means today’s consensus, nothing stronger.

Self-test. For each scenario, give the responsibility to one party and justify it in one sentence. The prevailing view for each scenario, with its reason, is at the back of the book.

#	Scenario	(a) Maker / developer	(b) Operating organization	(c) End user	(d) The AI itself
1	A self-driving car crashes because of the maker’s software bug.	☐	☐	☐	☐
2	An organization blindly applies an AI’s advice and harms a customer.	☐	☐	☐	☐
3	A user deliberately uses an AI tool to produce fake content.	☐	☐	☐	☐

Three questions help. In the chain that led to the harm, who made the decision? Who could have checked and did not? Who intended harm? If the fourth party tempts you, ask what a court sentence would mean to a piece of software. The law does not go there; responsibility gathers in the human links of the chain. The scenarios here are simpler than life. In real cases all three human parties turn out partly responsible, and the fight over shares runs through the courts for years.

The rights question has no scenario, because it has no case yet. Today there are two main stances. One says rights need experience or consciousness, which today’s systems lack, so the question is premature. The other says that where nobody is sure, caution is due: better careful now than wrong later. Both rest on the question of

consciousness, and no chapter of this book has settled it. Nobody has.

What is happening? When harm happens, responsibility today almost always lands on people and institutions: whoever built, operated or used the system. Holding “the AI itself” legally responsible is not a common view. Whether machines could one day hold rights is an entirely different question, and still open.

Try it yourself. 1) Write a fourth scenario: a case where responsibility falls on two parties at once. How would you split the share? 2) Which of the two stances above is closer to yours? One sentence on why. 3) You have a say in the future of AI, the margin note says. Where, concretely, does yours begin? Write one example.

Live demo:



book.onuronder.com/d/en/833800c84f

TECHNICAL DEPTH

Accountability today is overwhelmingly attributed to people and institutions: humans make the design, deployment and usage decisions; assigning legal responsibility to “the AI itself” is not the prevailing view. Responsibility is usually shared and context-dependent (developer, operator, user, regulator).

AI’s moral status is a separate, contested question: some argue status requires sentience, absent in current systems; others invoke precaution. It is both an empirical question (is there consciousness?) and a normative one (if there were, what would we owe?), and it remains open.

The three scenarios in Figure 8.5 stand for three separate sources of responsibility: defect (a design error), negligence (use without oversight) and intent (deliberate misuse). Legal systems meet these three with different instruments: product liability, duty of care and criminal law. The fourth party fits none of these frameworks, because responsibility requires a subject for whom a sanction means something.

None of these questions closes with the chapter; they are the kind you keep. Six questions follow.

8.7 Test yourself

Answers are at the back of the book.

1. What does the Turing test fundamentally measure?
 - a) Amount of memory
 - b) Processing speed
 - c) Real consciousness
 - d) Imitating a human indistinguishably in writing

2. What does the Chinese Room argument question?
 - a) Internet security
 - b) Whether symbol processing alone yields “understanding”
 - c) Computer speed
 - d) The difficulty of Chinese
3. What level is today’s AI at?
 - a) AGI
 - b) Superintelligence
 - c) Narrow AI
 - d) Conscious AI
4. The most balanced stance on the singularity?
 - a) Uncertain; neither certain nor impossible
 - b) Completely impossible
 - c) It already happened
 - d) Definitely happening tomorrow
5. When an AI causes harm, responsibility today usually falls on?
 - a) Only the AI itself
 - b) The internet
 - c) People and institutions (developer/operator/user)
 - d) No one
6. The question of AI’s moral status (rights) today is?
 - a) A meaningless question
 - b) Banned by law
 - c) Definitively settled
 - d) Open and contested

What to keep from this chapter

- The questions in this chapter are open, and that is their point.
- The Turing test looks at behavior: a machine that cannot be told apart in writing passes, but fluent imitation is not understanding.
- The Chinese Room argues that matching symbols by rule does not produce understanding; whether understanding lives in the person

or in the system is still contested.

- Every AI today is narrow; general AI is hypothetical, superintelligence is theoretical, and being “general” is not being “conscious.”
- Nobody has proven the singularity and nobody can rule it out; which curve we are on shows only in hindsight.
- When harm happens, responsibility today falls on people and institutions: whoever built, operated or used the system.
- Whether machines will ever have rights turns on the open question of consciousness; the future of AI will be decided by the values that build and use it.

Answer Key

End-of-chapter quizzes

01 · Minds and Machines

1.8 · Question 1: **d** — That intelligence has many kinds, not one 1.8 · Question 2: **c** — 0 and 1 1.8 · Question 3: **a** — In principle any computation, via simple rules 1.8 · Question 4: **b** — Keeping program and data in the same memory 1.8 · Question 5: **c** — Narrow AI 1.8 · Question 6: **d** — Transistor count ~doubles every 2 years

02 · The Age of Rules

2.7 · Question 1: **d** — With explicit symbols and rules 2.7 · Question 2: **c** — A rule (IF-THEN) 2.7 · Question 3: **a** — They speed up the search but don't guarantee the best 2.7 · Question 4: **a** — Only the current state 2.7 · Question 5: **b** — Hand-writing all rules, and the messiness of the world 2.7 · Question 6: **d** — Two approaches in AI research

03 · How Machines Learn

3.8 · Question 1: **d** — It learns from data instead of hand-written rules 3.8 · Question 2: **c** — Label 3.8 · Question 3: **b** — Clustering (unsupervised) 3.8 · Question 4: **c** — Updates parameters step by step to reduce loss 3.8 · Question 5: **a** — Memorizing training data and failing on new data 3.8 · Question 6: **d** — By reward and penalty, trial and error

04 · The Artificial Brain

4.8 · Question 1: **d** — A weighted sum of inputs + bias, then an activation 4.8 · Question 2: **a** — Many hidden layers 4.8 · Question 3: **b** — The network would collapse into one linear function 4.8 · Question 4: **c** — Spreads the error backward and updates weights to reduce it 4.8 · Question 5: **a** — Images 4.8 · Question 6: **d** — Generator and Discriminator

05 · Today's AI

5.9 · Question 1: **d** — A small piece of text the model processes 5.9 · Question 2: **a** — Semantically similar words land near each other 5.9 · Question 3: **a** — Each word “looking at” the others with weights 5.9 · Question 4: **c** — Predicts the next token 5.9 · Question 5: **a** — Pretraining → fine-tuning → RLHF 5.9 · Question 6: **d** — Starting from noise and cleaning it step by step 5.9 · Question 7: **b** — The model producing confident but wrong information 5.9 · Question 8: **b** — How many tokens the model attends to at once

06 · Using and Building AI

6.7 · Question 1: **d** — Adding role, context, examples and a clear format 6.7 · Question 2: **a** — Finds the relevant source and grounds the answer on it 6.7 · Question 3: **a** — Using tools and taking action step by step 6.7 · Question 4: **b** — Coordinates prompts, tools and RAG calls 6.7 · Question 5: **b** — Grounding the answer in a source via RAG 6.7 · Question 6: **d** — Designs that strengthen people (copilots)

07 · AI and Society

7.7 · Question 1: **d** — Skewed or incomplete training data 7.7 · Question 2: **b** — Its decisions' reasons are hard to understand 7.7 · Question 3: **c** — Questioning and verifying the source 7.7 · Question 4: **a** — Risk level 7.7 · Question 5: **b** — The stated goal diverging from the true intent 7.7 · Question 6: **d** — Screening job candidates

08 · Philosophy and the Future

8.7 · Question 1: **d** — Imitating a human indistinguishably in writing 8.7 · Question 2: **b** — Whether symbol processing alone yields “understanding” 8.7 · Question 3: **c** — Narrow AI 8.7 · Question 4: **a** — Uncertain; neither certain nor impossible 8.7 · Question 5: **c** — People and institutions (developer/operator/user) 8.7 · Question 6: **d** — Open and contested

Figure exercises and self-test answers

Chapter 1 answers

FIGURE 1.1 · EXPLORE MULTIPLE INTELLIGENCES

Try it yourself. 1) Linguistic, Logical-Mathematical, Spatial, Musical, Bodily-Kinesthetic, Interpersonal, Intrapersonal, Naturalist. The ones most often forgotten are usually Intrapersonal and Naturalist. 2) It depends on the person. According to the table, AI is strong only in the Linguistic and Logical-Mathematical areas; most of a day, though, goes to bodily, interpersonal and intrapersonal work, and AI is weak in all three. 3) All three ask for a material that does not turn into symbols: a body, another person's mind, your own inner world.

FIGURE 1.2 · CRACK THE BINARY CODE

Try it yourself. 1) $10 = 8 + 2 \rightarrow 00001010$. 2) $10110000 = 128 + 32 + 16 = 176$. 3) 128. With nine boxes the largest number is $256 + 255 = 511$; nine boxes hold 512 different numbers.

FIGURE 1.3 · A WORKING TURING MACHINE (+1)

Try it yourself. 1) 001111: the head walks to the end (5 moves), the state changes (1 move), the four 1s are cleared one by one (4 moves), and the 0 in cell 1 is replaced by a 1 (1 move). Eleven moves in all; the tape reads 010000 = 16. 2) 010110: because 22 is even, the first digit the head sees on its way back is a 0. It writes a 1 there at once; it finishes in $5 + 1 + 1 = 7$ moves; the tape reads 010111 = 23. 3) 111111: all six 1s are cleared ($5 + 1 + 6 = 12$ moves). After turning the 1 in the leftmost cell into a 0, the head cannot move left and stops. The tape reads 000000 = 0. Six cells cannot hold 64; the counter wraps around to zero. It is like an odometer jumping from 999999 to 000000; computer people call this an overflow.

FIGURE 1.4 · THE FETCH - EXECUTE - WRITE CYCLE

Try it yourself. 1) Fetch: the control unit takes the “multiply” instruction from memory (Memory). Execute: the ALU reads 7 and 6 and computes $7 \times 6 = 42$ (Processor). Write: the 42 goes to the screen (Input / Output). 2) One more round, that is, three more phases: fetch “multiply the result by 2,” execute $8 \times 2 = 16$, write 16 to memory. For it to appear on the screen, a separate “write to the screen” line is needed. 3) $3 \text{ billion} \times 3 = 9 \text{ billion}$ phases.

FIGURE 1.5 · REAL TODAY, OR SCIENCE FICTION?

Try it yourself. 1) Navigation, translation and movie recommendations: all three are narrow AI. Each does a single job; you cannot ask the navigation app for a translation. 2) Same column, but not the same question. The fourth card is about the “general” question: can it learn every domain? The fifth card is about the “strong” question: does it truly understand, is it conscious? A machine can be general without being conscious. 3) Nothing happens; the engine only evaluates chess positions and does not recognize “soup” as an input.

Narrow AI: a system that can beat a human at one job and can do nothing outside that job.

Self-test. 1 Chess engine → Narrow AI · exists today: one job, chess; it knows nothing else. 2 Face recognition system → Narrow AI · exists today: it matches faces and has no task beyond faces. 3 Chatbot (language model) → Narrow AI · exists today: even when it writes poems and code, it does one job, predicting the continuation of a text; it has no goals of its own and cannot step outside what it learned. This is the card people get wrong most often. 4 A machine that learns any profession like a human and has its own goals → General / AGI · not yet: cross-domain generalization is the very definition of AGI; no such system has been built. 5 A self-aware, conscious AI → General / AGI · not yet: on the “does not exist” side; but consciousness is a “strong AI” question, not a “general” one. It lands in this column only because the figure has two columns.

FIGURE 1.6 · FEEL EXPONENTIAL GROWTH

Try it yourself. 1) $n = 14$, 1999: $2^n = 16,384$, transistors 37,683,200 (the live demo shows “37.7 million”). $n = 15$, 2001: $2^n = 32,768$, transistors 75,366,400 (“75.4 million”). 2) The 1989 row, $n = 9$: 1,177,600. Eighteen years from 1971. 3) With a three-year doubling, 1971 to 1995 is 24 years = 8 doublings: $2,300 \times 256 = 588,800$. The two-year table gives 9,420,800 for 1995; the gap is a factor of 16. A single extra year in the doubling time makes a 16-fold difference after 24 years.

Chapter 2 answers

FIGURE 2.1 · INFERENCE ALONG A KNOWLEDGE CHAIN

Try it yourself. 1) Yes. The machine follows five arrows: Tom → Cat → Mammal → Animal → Living thing → Entity. A longer chain means more steps, but the rule stays the same. 2) Unknown. From Cat the chain leads on to Mammal, Animal and Living thing; Tom is never reached. That answer is right: not every cat is Tom, and an “is a” link points one way only. 3) The chain holds only positive “is a” links; negative knowledge needs a separate fact or rule. If, for example, the rule

“Animal and Plant are disjoint classes” is added, the machine can derive “not a Plant” from “Tom is an Animal.” A system without that addition can only say “unknown.”

FIGURE 2.2 · A TINY EXPERT SYSTEM

Try it yourself. 1) R1, R2, R3 and R5 fire; four pieces of advice: Take an umbrella · Wear a coat · Wear a scarf · Careful: the umbrella may flip! R4 stays silent, because it is raining. 2) Only R4 fires: “You can dress light.” R4 looks only at rain and cold; it never asks about wind. In windy weather this advice falls short. What is missing is a rule such as “IF it’s windy THEN take a windbreaker.” That is the blind spot of rules: a condition nobody wrote is treated as if it did not exist. 3) If R5 depended on wind alone, it would warn “the umbrella may flip” even when it is not raining and no umbrella was advised. Chaining ties the warning to the situation it belongs to: the warning only makes sense if an umbrella was advised.

FIGURE 2.3 · PATHFINDING: UNINFORMED VS INFORMED

Try it yourself. 1) Score = column difference + row difference, with the goal in column 8, row 6. Column 4, row 2: $4 + 4 = 8$. Column 1, row 6: $7 + 0 = 7$. The bottom corner looks closer, but the wall in column 5 makes it a dead end; the cell on the path is column 4, row 2. That is why the informed search strayed into the bottom row and wasted five cells. 2) The score does not see walls; 12 is the shortest path on a grid without walls. The three barriers add three detours, and the real path grows to 18 steps. The score never exceeds the true distance; that is the “admissible” heuristic of the Technical depth box. 3) 12 steps: 5 steps down column 1, then 7 steps right along the bottom row. Once the barrier is gone, the path shrinks to the distance the score promised.

FIGURE 2.4 · A WEATHER MARKOV CHAIN

Try it yourself. 1) The Rainy row is 20 / 40 / 40, so the ranges are 1-20 / 21-60 / 61-100. 35 falls in the second range: tomorrow is Cloudy. 2) Sunny tomorrow, then rain: $0.7 \cdot 0.1 = 0.07$. Cloudy tomorrow, then rain: $0.2 \cdot 0.3 = 0.06$. Rainy tomorrow, then rain again: $0.1 \cdot 0.4 = 0.04$.

ome stickier and the chain stays in sun longer. The calculation: with the new matrix the stationary distribution is about 72 / 15 / 13 percent (the old one was 46 / 31 / 23). Check with the first equation: $0.9 \cdot 0.72 + 0.3 \cdot 0.15 + 0.2 \cdot 0.13 \approx 0.72$.

FIGURE 2.5 • WHICH APPROACH?

Self-test. 1) Prove everything with formal logic → Neat: proof first, use later. 2) Use working shortcuts, theorize later → Scruffy: the result first, theory can wait. 3) Mathematical rigour is essential → Neat: rigor is not up for negotiation. 4) The real world is messy; be flexible → Scruffy: adapt to the world, not to the principle. **Try it yourself.** 1) The knowledge chain, the expert system and uninformed search are close to the Neat camp: exact derivation, guaranteed result. Informed search belongs to the Scruffy camp: it gives up the guarantee for speed. The Markov chain sits in between: it rests on probability theory and its convergence can be proven (Neat), but it accepts that the world is uncertain (a Scruffy attitude). 2) The heuristic score is a Scruffy invention: it works, and nobody asks for a guarantee. A* adds a proof to it with the “admissible” condition and guarantees the shortest path; it carries a Scruffy idea into the Neat camp. That is where the two camps shake hands. 3) Answers vary; the test is whether you made it work first and understood why afterward. If so, you acted as a Scruffy.

Chapter 3 answers

FIGURE 3.1 • SEE THE FEATURES AND THE LABEL

Try it yourself. 1) Mentions “free” ○, link / password request ✓ (a password renewal), urgency language ✓ (“today,” “will be deleted”). Two cues; in the table the only example with two cues, email 3, was Spam. Label: Spam. 2) Less. “Free” now appears in a Normal email too, so on its own it stops being a distinguishing cue. The link / password request and the urgency language are still found only in the Spam rows. 3) One possible rule: “If there is a link or password request, Spam; otherwise Normal.” It agrees with all four rows (1 and

3 Spam, 2 and 4 Normal); the coffee email has no link, so it says Normal, which is right. The rule “If it mentions ‘free’ or asks for a link or password, Spam” also agrees with the four rows but fails on the coffee example. Several rules can agree with the same data; only new examples show which one is right.

FIGURE 3.2 · WHICH KIND OF LEARNING?

Self-test. 1 Telling cats from dogs using labeled photos → Supervised: the photos are labeled; the correct answer is given up front. 2 Grouping customers by similarity → Unsupervised: nobody said “this customer belongs to that group”; the machine groups by similarity on its own (clustering). 3 A robot learning to walk by trial and error → Reinforcement: the robot tries, falls, and earns a reward each time it stays upright; there is no list of correct moves. 4 Predicting house prices from past (labeled) data → Supervised: the prices of past sales are known; since the label is a number, this is regression. 5 Learning a high score by playing a game → Reinforcement: the score is the reward; nobody says which move is right, it learns by trying. 6 Clustering unlabeled news by topic → Unsupervised: the stories are unlabeled; similar topics are clustered together. Follow-up questions: the words “labeled” or “unlabeled” appear in tasks 1, 4 and 6. In the others you read the presence of a label from the nature of the task. Task 2 says “by similarity,” and nobody supplies group names. In tasks 3 and 5 the correct answer comes from the environment, from the reward, rather than from a teacher; that is what the two share. **Try it yourself.** 1) The translation program: supervised; the human translations serve as labels. The chess program playing against itself: reinforcement; winning is the reward. Product groups from receipts: unsupervised; nobody names the groups in advance. 2) A sensible order: first “are the correct answers given?”; if yes, supervised, stop. If no, “is there a reward or penalty?”; if yes, reinforcement, if no, unsupervised. Other orders work too; what matters is that each leaf holds a single kind. 3) Example: a language model predicting the next word in a text. The label (the next word) comes from the data itself; nobody labels it by

hand. It is trained like supervised learning but works on unlabeled data: self-supervised learning sits between the two columns.

FIGURE 3.3 • TWO CORE TASKS

Try it yourself. 1) $0.55 \cdot 6.5 + 0.76 = 3.58 + 0.76 = 4.34$. 2) (4.5, 3.5): at $x = 4.5$ the boundary says $6.1 - 2.7 = 3.4$; $3.5 > 3.4$, so the point is above the boundary, orange. (5, 3): the boundary says $6.1 - 3 = 3.1$; $3 < 3.1$, below, green. Both are very close to the boundary; a real model would give these points low confidence. 3) The slope goes up: with ten points $m \approx 0.71$ and $b \approx 0.23$ (before, 0.55 and 0.76). A single far-off point pulls the line away from the other eight; because the method squares the errors, it is sensitive to outliers. That is a problem: first you have to check whether (9, 9) is a measurement error or a real observation.

FIGURE 3.4 • GROUP UNLABELED DATA

Try it yourself. 1) The new center of B: $x = (7.5 + 6.5 + 7.8 + 6.8 + 8) / 5 = 7.32$, $y = (3.2 + 2.5 + 3.8 + 4 + 2.8) / 5 = 3.26$. The move is $\sqrt{(0.32^2 + 0.26^2)} \approx 0.41$. 2) (4.5, 5.5): to A, $\sqrt{(2^2 + 1.5^2)} = 2.5$; to B, $\sqrt{(2.5^2 + 2.5^2)} \approx 3.54$. It goes to A. But the other members of A are at most 1.22 from their center; 2.5 is twice that. Counting it as an outlier candidate is reasonable. 3) The distance to the center and a threshold: here something like “twice the distance of the cluster’s farthest ordinary member.” A person chose the threshold, not the data (in the figure the outlier is fixed in advance). Clustering uses no labels; but the answer to “how far is too far?” is a human decision.

FIGURE 3.5 • DESCENDING THE LOSS VALLEY

Try it yourself. 1) Slope $0.36 \cdot (2.06 - 5) = -1.06$. New $x = 2.06 + 0.18 \cdot 1.06 = 2.25$. Loss $0.18 \cdot (2.25 - 5)^2 + 0.1 = 1.46$. 2) $\eta = 6$: step 1, $x = 0.6 + 6 \cdot 1.584 \approx 10.10$; slope $0.36 \cdot 5.10 = 1.84$; step 2, $x = 10.10 - 6 \cdot 1.837 \approx -0.92$. The distance to the floor goes $4.4 \rightarrow 5.1 \rightarrow 5.9$: growing. The ball is moving away; divergence. (The factor is $1 - 2.16 = -1.16$, whose absolute value is greater than 1.) 3) Yes: $\eta = 1 / 0.36 \approx 2.78$. Then $x - (x - 5) = 5$, the floor in one step. This works only because the valley is an exact parabola; on real loss surfaces the curvature differs from place to place, and no single rate reaches the floor in one step.

FIGURE 3.6 • SAME DATA, THREE MODELS

Try it yourself. 1) Underfit: $3.1 - 1.8 = 1.3$. Good: $3.0 - 1.6 + 0.15 \cdot \sin(6) \approx 1.36$. The broken line ends at $x = 9$ and can say nothing about $x = 10$; if you extend its last segment it says 2.6, that is, it carries the random rise of the last two points into the future. 2) $x = 2$: the broken line, between (1, 3.2) and (3, 3.0), says 3.1; the truth is 2.4, an error of 0.7; the straight line says 2.74, an error of 0.34. $x = 8$: the broken line says 2.15, an error of 0.75; the straight line says 1.66, an error of 0.26. Again the memorizer loses. 3) It proves that the model remembers the training points; it does not prove that it will do well on a new point. The hide-and-test exam showed as much: with a training error of zero, the validation error was larger than the straight line's.

Chapter 4 answers

FIGURE 4.1 • RUN THE NEURON

Try it yourself. 1) Sum $0.7 + (-0.5) + 0.9 - 0.3 = 0.80$; sigmoid 0.690. Above 0.5, so the neuron fires (with ReLU 0.800, it fires as well). 2) Sum $0.9 \times 1 - 0.3 = 0.60$; ReLU output 0.600 (sigmoid 0.646). It fires. 3) Sum $-0.5 \times 1 - 0.3 = -0.80$; sigmoid 0.310. It does not pass 0.5, so the neuron stays silent; on its own, x_2 only pulls the sum down.

FIGURE 4.2 • A LIVE NEURAL NETWORK (FORWARD PASS)

Try it yourself. 1) With every input at 0, each hidden neuron's sum is 0 and $\text{sigmoid}(0) = 0.50$; all four sit at half brightness. O1: $(0.7 - 0.5 + 0.6 + 0.4) \times 0.50 = 0.60$, sigmoid 0.65. O2: $(-0.4 + 0.6 + 0.5 - 0.6) \times 0.50 = 0.05$, sigmoid 0.51. O1 wins again. 2) H1: $0.6 - 0.4 + 0.8 = 1.00$; sigmoid 0.73. 3) No. O1 wins in all eight patterns; the closest race is [0, 1, 0], 0.61 to 0.59. Examples: for [1, 1, 0], O1 0.61 and O2 0.56; for [0, 0, 1], O1 0.71 and O2 0.48; for [1, 1, 1], O1 0.68 and O2 0.53. Because the weights were never trained, the network keeps pointing at the same door.

FIGURE 4.3 · LEARN FROM ERROR (ANIMATION)

Try it yourself. 1) $0.43 \times 0.6^9 \approx 0.0043$; with two decimals, 0.00. 2) From 65 to 71 percent, 6 points. The step from round 0 to 1 was 17 points; $17 / 6 \approx 2.8$, so roughly three times smaller. 3) Output = target – error; $100 - 43 = 57$ percent.

FIGURE 4.4 · CONVOLUTION: SLIDE THE FILTER

Try it yourself. 1) –3. Window rows 5 to 7, columns 2 to 4: the right column (column 4 of the image) is 1 in all three rows, and the kernel's right column is –1; three times –1. 2) 0. Window rows 3 to 5, columns 3 to 5: top row $[0, 1, 0] \times [1, 1, 1] = 1$; the middle row of the kernel is zero; bottom row $[0, 1, 0] \times [-1, -1, -1] = -1$; total 0. 3) Map size $(7 - 5 + 1) = 3$; $3 \times 3 = 9$ cells.

FIGURE 4.5 · PROCESSING WITH MEMORY (ANIMATION)

Try it yourself. 1) Fell: h_1 (96 → 86), h_4 (92 → 88), h_5 (100 → 87), h_7 (97 → 85), h_8 (96 → 86). Rose: h_3 (96 → 98), h_6 (91 → 96). Stayed put: h_2 (99). 2) The $W_h \cdot h_0$ term; because $h_0 = 0$, it is zero. In the first step the hidden state comes only from $W_x \cdot x_1$. 3) Yes, a model that ignores order (a bag of words) produces the same representation for both sentences; both contain the same words. An RNN processes each word together with the previous memory, so a different order gives different hidden states; to it the two sentences are separate things.

FIGURE 4.6 · GENERATOR VS DISCRIMINATOR

Try it yourself. 1) Round 5; the fake probability is 40 percent, below 50 percent for the first time. 2) Rule: $p(r) = 95 - 11 \times r$. For $r = 8$, $95 - 88 = 7$. 3) The discriminator can no longer tell fake from real; it is tossing a coin. The generator has captured the real distribution and the game has reached equilibrium (in technical terms, $D(x) = 1/2$).

Chapter 5 answers

FIGURE 5.1 · SPLIT A SENTENCE INTO TOKENS

Try it yourself. 1) 8 tokens: Arti · ##fici · ##al · inte · ##llig · ##ence · learns · . (“Artificial” and “intelligence” break into three pieces each; “learns” has six letters and stays whole; the period counts separately). 2) 5 pieces: inte · ##rnat · ##iona · ##liza · ##tion (20 characters, five pieces of four characters each). 3) The third sentence carries $11 / 4 = 2.75$ tokens per word; the first sentence $7 / 5 = 1.4$. Almost twice as many tokens per word: long words make the difference.

FIGURE 5.2 · THE MEANING MAP

Try it yourself. 1) The differences are 66 and 54; their squares 4,356 and 2,916; the sum 7,272; the square root ≈ 85.3 . The largest within-family distance in the table is 52.8 (dog–bird; bread–cheese is 42.8); 85.3 is far above it, so “bird” and “cheese” live in different districts. 2) In the animal region, for example around (70, 75). From there “cat” (58, 62) is 17.7 away, “dog” (84, 50) 28.7, “bird” (52, 92) 24.8; the two nearest neighbors are cat and bird. Answers vary; what matters is placing it near the three animals. 3) The third-nearest word to “bread” is “prince” (54.4), from the royalty family. The food and royalty districts touch on the map; the families are separate, but there is no sharp wall between them.

FIGURE 5.3 · WHICH WORD LOOKS AT WHICH?

Try it yourself. 1) Yes, all five rows sum to 1.00: The cat $0.50 + 0.30 + 0.05 + 0.10 + 0.05$; ran $0.50 + 0.30 + 0.10 + 0.05 + 0.05$; because $0.20 + 0.40 + 0.20 + 0.10 + 0.10$; it $0.55 + 0.10 + 0.05 + 0.20 + 0.10$; was scared $0.30 + 0.10 + 0.05 + 0.40 + 0.15$. 2) The darkest cell would move to “the dog”: the predicate “was scared” looks for its subject, and the subject is no longer a pronoun but “the dog” directly. The share going to “The cat” would drop too, because the one who is scared is no longer the cat. 3) The “because” column stays between 0.05 and 0.20 in every row; no word leans on it. A conjunction ties two events together but carries no answer of its own to who, what or where; since its load of meaning is low, the others have little reason to look at it.

FIGURE 5.4 · GENERATE WORD BY WORD

Try it yourself. 1) “AI today creates daily.” (today 18%, creates 20%, daily 15%). 2) Low: $0.42 \times 0.38 \times 0.50 \approx 0.080$ (8 percent). High: $0.28 \times 0.30 \times 0.25 = 0.021$ (2 percent). The low-creativity sentence is about four times as likely; high temperature opens the less likely roads as well. 3) Step 2: the top candidate has only 38 percent (42 in step 1, 50 in step 3). The lower that share, the more the probability is spread over the other candidates, and the less sure the model is.

FIGURE 5.5 · AN ASSISTANT IN THREE STAGES

Try it yourself. 1) Stage one continues the text: an encyclopedia sentence that does not know how to stop, such as “Water boils at 100 °C at sea level, and the boiling point drops with altitude. The boiling point...” Stage two answers the question and stops: “Water boils at 100 °C at sea level.” Answers vary; the test is that the first one “continues” and the second one “answers and stops.” 2) “Right but rude” should be preferred; the “honest” goal puts truth first, and the “helpful” goal puts useful information first. Politeness is a separate gain of the third stage: the ideal answer is both right and polite, but when the two collide, truth comes first. A reward signal that preferred the wrong but polite answer would train the model toward hallucination. 3) The “Sample output” row. The fact is the same in all three columns (Ankara), so “what it knows” was settled in pretraining; only the length and tone of the answer change, that is, “how it behaves.”

FIGURE 5.6 · FROM NOISE TO IMAGE

Try it yourself. 1) The share of orange among randomly opened pixels is $40 / 64 = 0.625$; expected $29 \times 0.625 \approx 18$ orange pixels (the figure’s fixed order gives 17). 2) $100 / 16 = 6.25$, that is, six and a quarter percent per step; the strip becomes twice as long, and each frame differs less from the one before. 3) The forward process (adding noise to the image) is read from right to left on the strip: from the clean heart to the screen full of static. The reverse process, the cleaning the model learned, is read from left to right.

FIGURE 5.7 • THE CONTEXT WINDOW

Try it yourself. 1) $12 - 5 = 7$ words are forgotten. What remains in the window: “window and forget older words.” The subject of the sentence, its first verb and its object are all gone; the model sees only the tail. 2) 15 tokens: Lang · ##uage · models · hold · text · inside · a · limi · ##ted · window · and · forget · older · words · . (only “Language” and “limited” cross the six-letter limit). If the window counted 8 tokens, it would fill at the seventh word, “limited,” whose first piece “limi” is the eighth token; that word’s second piece, “##ted,” would push “Lang” out before the word is even complete. One word earlier than word counting for filling, two words earlier for the first loss. 3) Usually summarizing: truncating throws away part of the document entirely (most often the end or the beginning), while a summary keeps a trace of every part. But if the question concerns one specific part of the document, cutting out that part alone and handing it over can be the better choice. Answers vary.

Chapter 6 answers

FIGURE 6.1 • BUILD A PROMPT

Try it yourself. 1) Two pieces: $\text{quality} = 40 + 2 \cdot 15 = 70\%$, tier medium. The answer is the medium-tier text: “I suggest a beach destination: swimming in the morning, a short town tour in the afternoon, a fish restaurant in the evening. A budget-friendly guesthouse would fit.” The rule does not look at which pieces are added, only at how many. 2) At least three pieces (85%). Two pieces stay at 70%; the high threshold is 85%. 3) Answers vary. Example: Role: “You are a customer relations specialist.” Context: “The customer’s order is two days late; I am apologizing and giving the new delivery date.” Example: “Dear ..., we apologize for ...; the new delivery date is” Format: “Three short paragraphs: the apology in the first, the new date in the second, contact details in the third.”

FIGURE 6.2 • ANSWERS GROUNDED IN SOURCES

Try it yourself. 1) “usually around,” “varies by company,” “I think,” “probably around,” “I’m not sure.” The sourced answers contain no such word; in its place stands a section number (§4, §7, §2). 2) It should say “I could not find a clause about this in the documents” and not guess. When retrieval comes back empty, a good system says so plainly; it does not fill the gap from memory. 3) “Per HR Policy §4, annual leave rises to 26 days after 5 years; for an employee with six years of service, 26 days.”

FIGURE 6.3 • WATCH AN AGENT

Try it yourself. 1) Step 1: thought “First the total: 4×200 .”, tool calculator($"4 * 200"$), observation 800. Step 2: thought “Per person: $800 \div 5$.”, tool calculator($"800 / 5"$), observation 160. Step 3: no tool; final answer “The total is 800 TL; split among 5, that’s 160 TL per person.” 2) The agent accepts 450 as correct; step 2 becomes calculator($"450 / 6"$) → 75, and the final answer comes out as “The total is 450 TL, 75 TL per person.” The agent does not question the tool’s output; that is why tools must be reliable and observations must be validated separately. 3) At least one more call: calculator($"90 + 30"$) → 120. Final answer: pizza total 540 TL, per person 120 TL (90 pizza + 30 drinks). If the grand total including drinks is also wanted, one more call is needed: calculator($"6 * 120"$) → 720.

FIGURE 6.4 • THE PARTS OF AN AI APPLICATION

Try it yourself. 1) “The date I gave you yesterday” → memory. “What is the dollar rate today?” → tools (search or an API for the current rate). “The company’s refund policy” → knowledge base (a company document, RAG). 2) None. Orchestration calls the model; the five boxes stay the same. What changes are the settings inside orchestration: the prompt, the fallback, the evaluation. 3) Waiter → interface; manager → orchestration; pantry → knowledge base; tools on the counter → tools; regulars’ notebook → memory.

FIGURE 6.5 • EXPLORE THE FIELDS

Try it yourself. 1) The second table in Step by step is one distribution; others can be defended. Examples that fall under two skills at once: “Summarizing and coding patient notes” (summarizing + classification), “Document/contract analysis and risk scoring” (retrieval + prediction), “Screening candidate molecules in drug discovery” (prediction + recognition), “Simulation and hypothesis generation” (prediction + generation), “Voice assistants and summarization” (agent + summarization). 2) Example picks: Health: spotting anomalies in medical images (a missed finding can cost a life). Finance: real-time fraud detection (a missed transaction costs money, a false alarm costs a customer). Manufacturing: predictive maintenance (a missed failure stops the line and threatens worker safety). Science: protein structure prediction (a wrong structure sends years of experiments the wrong way). Art: style transfer and restoration (an original work can be damaged beyond repair). Daily life: voice assistants and summarization (a wrong summary leads to a wrong decision). 3) Answers vary. Example: sorting incoming customer emails by urgency → skill: recognizing; field: daily life or finance; boxes needed: interface, orchestration, knowledge base (past replies), memory (customer history).

Chapter 7 answers

FIGURE 7.1 • A BIAS SIMULATION

Try it yourself. 1) $A = 50 + 0.4 \cdot 75 = 80$, $B = 50 - 0.4 \cdot 75 = 20$; the parity gap is 60 points. 2) At 9 percent. At 8 percent the rates are 53/47 (gap 6, still “balanced”); at 9 percent they round to 54/46, the gap is 8 and the caption switches to “skewed.” 3) No. At the end of the scale ($e = 100$) A stays at 90 percent and B at 10 percent. Reaching the 95 and 5 percent limits would need $e = 112.5$, and the scale ends at 100. The limits sit in the code as a seat belt and never come into play.

FIGURE 7.2 • WHITE BOX: EXPLAIN THE DECISION

Try it yourself. 1) Yes, it changes. The total becomes $-8 + 10 = +2$; it is above zero, so the loan is approved. A 10-point improvement in a single factor flips the decision; that is the value of the open box: you can see which factor to fix and what happens when you do. 2) -90 or lower (a magnitude of at least 90). The plus factors add up to $+40 + 28 + 22 = +90$; at -90 the total is 0, the condition “greater than zero” fails and the decision flips to declined. 3) About 70 percent. Bar length is scaled to the largest absolute contribution: $32 / 46 \approx 0.70$.

FIGURE 7.3 • REAL OR FAKE?

Self-test. 1 Video → fake: lip-sync mismatch and shimmering around the face edges are classic deepfake tells. 2 Audio → fake (scam): voice cloning plus urgency pressure is a classic scam; verify through a second channel. 3 Written news → real: multiple independent sources and a traceable origin are the marks of reliability. 4 Photo → fake: hands, teeth and background text are where generative models still struggle. **Try it yourself.** 1) Two example steps: hang up and call your boss back on the number you already know; do not make the transfer until a second person or a written channel (company email, face to face) has confirmed it. The urgency pressure itself is a warning sign. 2) Answers vary. A good answer gives a concrete reply to all three questions: who first published the content, which account or site it came from, and whether at least one independent source says the same thing. The question left unanswered is the gap you must close before sharing the content. 3) No. The tells on the four cards are the weaknesses of today’s models; as generation improves, six fingers and broken lip sync disappear. The absence of a tell is not proof of reality; it does not replace source verification.

FIGURE 7.4 • CLASSIFY THE RISK

Self-test. 1 A state system scoring citizens by behavior → Banned: social scoring violates fundamental rights; the EU AI Act counts it as unacceptable risk. 2 A system auto-screening job candidates → High: it affects a person’s access to work; strict compliance and human oversight are required. 3 A chatbot talking to customers → Limited:

the person on the other end has the right to know they are talking to an AI; the duty is transparency. 4 A spam filter in email → Minimal: it affects nobody's rights or life. 5 A model assessing loan applications → High: it decides access to an essential service; the same step as hiring. 6 An opponent AI inside a game → Minimal: entertainment, no effect on rights. **Try it yourself.** 1) The map app and the keyboard suggestions → minimal; they affect nobody's rights. The bank's fraud alert is debatable: if it only warns, minimal; if it freezes the account automatically, it affects the person's access to their money and moves toward the high step. The reason is the same question every time: does it affect someone's life or rights? 2) Yes. Face recognition that opens your own phone sits on the minimal (or at most limited) step; real-time face recognition that scans a crowd in a public space is, under the EU AI Act, on the banned step apart from narrow law-enforcement exceptions. Same technology, different use and different impact. 3) Affecting life or rights: 1, 2, 5. Not affecting: 3, 4, 6. Of the first set, one card (1) is banned and two (2 and 5) are high; so two cards sit on the high step. Of the second set, 3 is limited, 4 and 6 are minimal.

FIGURE 7.5 • GOAL VERSUS INTENT

Try it yourself. 1) Example: "Put all the mess in the room into the trash can; leave none anywhere, including under the rug and inside the cupboards." New loopholes: counting things that are not mess as mess and throwing them away, stopping when the can is full, or throwing the mess out of the window (it never reached the can, but "no mess is left in the room"). Every rewrite closes one loophole and leaves a new one. 2) Harmless shortcut: suggesting extra practice on each student's weak topics. Harmful shortcut: keeping low-scoring students out of the exam, or drilling the exam questions themselves; the average rises, the learning does not. 3) Example sentence: "The system maximizes the measure you gave it, not the intent the measure was supposed to stand for."

Chapter 8 answers

FIGURE 8.1 • HUMAN OR MACHINE?

Try it yourself. 1) Answers vary; your friend will usually look at formulaic style and the absence of personal detail, the same tells listed below. 2) Probably not. Today's chat models can pause, round off and even make mistakes when asked to; the "flawless arithmetic" tell is going stale. 3) Yes, it can be defended. In Turing's own paper the machine gives a deliberately wrong answer to an arithmetic question, because imitating a human means imitating human flaws too.

Self-test. 1 "What's your favorite food?" → Machine: Overly polite, formulaic, and the "as an AI" phrasing is a classic machine tell. 2 "Did you hit traffic this morning?" → Human: Personal detail, emotion, mild complaint and natural informality point to a human. 3 "What's 17×24 ?" → Machine: Instant, flawless, hesitation-free arithmetic usually gives away a machine (most people pause a little). 4 "What did you do this weekend?" → Human: Vague but lived-in details and everyday language suggest a human.

FIGURE 8.2 • YOU ARE IN THE CHINESE ROOM

Try it yourself. 1) Example line: "你几岁?" → "I am one year old." For Searle nothing changes: you, who wrote the line, understand it, and the room is still matching. A defender of the systems reply says "the room is now a richer system." Both answers are consistent; the difference is where you look for understanding. 2) This is a small example of the "robot reply": the reply is now tied to the outside world (grounding). To some it is a step toward understanding, to others only a more complicated rule. Your reason depends on whether you count a link to the world as required for "understanding." 3) Searle's memorization answer shrinks the system to one person and says "I still don't understand." Objectors say the person who memorized the book now carries two separate systems: you, who speak English, and a subsystem that "speaks" Chinese. Whether you find that convincing

depends on whether you believe one head can hold two understanders.

FIGURE 8.3 • THE CAPABILITY LADDER

Try it yourself. 1) Yes, all three land in the first row. A chat model looks good at many jobs, but its learning ended in training; it cannot adapt to a new domain on its own. The table of Figure 8.3 says the same: “All of today’s systems live here.” 2) Answers vary; a good criterion measures transfer to a new domain, for example “learns a job it has never seen as fast as a human.” The difference from the Turing test: it looks at the ability to adapt to a new domain rather than at the fluency of behavior. 3) Examples: a program that beats humans at chess, a system that is superhuman at a single game but cannot get up from the table and translate.

FIGURE 8.4 • THREE INTELLIGENCE CURVES

Try it yourself. 1) $10 \cdot (t/10)^3 = 5 \rightarrow (t/10)^3 = 0.5 \rightarrow t/10 \approx 0.794 \rightarrow t \approx 7.9$. The curve reaches the halfway point only after about 80 percent of the time has passed; the explosion is in the last fifth. 2) No. $e^{-(t/2.2)}$ never becomes zero; the curve approaches 10 but never touches it (9.9 at $t = 10$). The ceiling is a limit, not a destination. 3) None. Even on the plateauing curve, systems reach a level far more powerful than today’s; on the uncertain curve the leaps cannot be predicted. Whichever curve turns out right, making systems safe and aligned is not wasted.

FIGURE 8.5 • WHO IS RESPONSIBLE?

Try it yourself. 1) Answers vary. One example: a health app suggests the wrong dose and a doctor approves it without checking. The share is split between a software defect and negligent oversight; in law this kind of sharing is ordinary. 2) Both stances can be defended. The first is practical for today’s systems, the second cautious under uncertainty. 3) Some starting points: which product you use, whom you give your data to, which rule you ask for at work or at school, whether you look at a candidate’s position on this when you vote.

Self-test. 1 A self-driving car crashes because of the maker's software bug → (a) Maker / developer: The maker/developer (and oversight); the software defect is their responsibility. 2 An organization blindly applies an AI's advice and harms a customer → (b) Operating organization: The organization operating it; using a tool without oversight is their responsibility. 3 A user deliberately uses an AI tool to produce fake content → (c) End user: The user who deliberately misused it; the intent and the act are theirs. (d) The AI itself: not the prevailing view in any of the three scenarios. Today the law holds people and institutions responsible, not the software.

Glossary

The section number (for example 3.6) shows the chapter and the section in which the term is introduced.

Accountability · Section 8.6 The obligation to answer for the harm an AI system causes. Today it is assigned overwhelmingly to people and institutions (developers, operators, users), not to “the AI itself.”

Activation function · Section 4.2 The nonlinear function that turns a neuron’s weighted sum into its output: sigmoid, tanh, ReLU. Without it, stacked layers collapse into a single linear function.

Agent · Section 6.4 A system that equips a language model with tools (calculator, search, APIs) and runs it in a “think, use a tool, observe the result, try again” loop (ReAct). It does not just talk; it gets to work.

Algorithm · Section 1.3 A finite sequence of well-defined steps, like a cake recipe. The word comes from the name of the 9th-century mathematician al-Khwarizmi.

Alignment · Section 7.6 The problem of making a system’s behavior match human intentions and values. A machine that does what you said but misses what you meant is this problem in miniature. It is both a technical and a normative question.

Anomaly detection · Section 3.5 Catching examples that stray clearly from the distribution of the majority. A bank noticing a suspicious transaction relies largely on this.

Artificial general intelligence (AGI) · Section 1.6 A hypothetical system that can learn and adapt across every domain the way a human does; it has not been built. Being general does not mean being conscious. See Strong AI.

Artificial intelligence (AI) · Section 1.1 Systems that can perform tasks requiring human intelligence; the field’s pragmatic definition. It

was born at the intersection of computability theory and the philosophy of mind.

Artificial neural network · Section 4.1 A learning machine built by arranging artificial neurons in layers; at bottom, a stack of nonlinear transformations. It is only loosely inspired by the biological neuron.

Artificial neuron · Section 4.2 A simple unit that weighs its inputs, adds them up, adds a bias and passes the result through an activation; a gatekeeper of sorts. Not a copy of the brain but a rough mathematical analogy.

Attention · Section 5.4 Each token learning which tokens in the sequence to look at for its meaning; this is how the pronoun “it” finds out whom it refers to. Self-attention computes this with query, key and value vectors. The heart of the Transformer.

Backpropagation · Section 4.4 The error walking backward from the output to the input and telling every connection to “fix your share”; gradient computation by the chain rule. The key that made deep networks trainable.

Bias (neuron) · Section 4.2 A constant added to a neuron’s weighted sum that sets its threshold; the gatekeeper’s own temperament. It is learned in training. For the data-related meaning, see Bias (social).

Bias (social), algorithmic bias · Section 7.2 A model learning and reinforcing the historical bias, the under-representation or the proxy variables in its data. It comes from skewed data, not bad intent. For the neuron parameter, see Bias (neuron).

Bias–variance tradeoff · Section 3.7 The balance between underfitting and overfitting. A good model balances the two and generalizes to unseen data; it grasps rather than memorizes.

Binary representation · Section 1.3 Encoding every piece of information in 0s and 1s (bits); the machine’s simple alphabet. Eight bits make one byte and hold any number from 0 to 255.

Black box · Section 7.3 A model that delivers a decision but cannot explain its reasons. In decisions such as credit and hiring, being able to ask “why?” is a matter of rights. See Explainable AI.

Breadth-first search (BFS) · Section 2.4 Uninformed search that scans everything layer by layer without any sense of direction. It guarantees the shortest path but opens many nodes. See Heuristic.

Chinese Room · Section 8.3 Searle’s thought experiment: a person who produces flawless Chinese answers with a rulebook while understanding not a word of Chinese. It argues that symbol manipulation is not enough for understanding; the counterarguments are strong too.

Classification · Section 3.4 A supervised task that answers “which one?” with a category: spam or not spam? It learns a decision boundary that separates the classes; the boundary does not have to be a straight line.

Clustering · Section 3.5 Grouping unlabeled data by similarity on its own, like sorting the buttons in a box. k-means assigns points to the nearest cluster center and then updates the centers.

Computationalism · Section 1.3 The view that the mind is an information-processing system and that thinking is computation in the form of symbol manipulation. Its roots reach back to Hobbes; the idea at the foundation of AI.

Context window · Section 5.8 The largest number of tokens a language model can hold at once; a small notepad. When the pad is full, the oldest lines are erased; enlarging the window is expensive.

Convolutional neural network (CNN) · Section 4.5 A network that slides a small filter (kernel) over an image, looks for local patterns (edges, corners) and records what it finds in a feature map. The same filter is used everywhere; it learns with few parameters.

Deep learning · Section 4.1 Learning with neural networks that stack many hidden layers. Depth lets the network pull increasingly abstract representations (edge, shape, object) out of raw data.

Deepfake (synthetic media) · Section 7.4 Fake content produced by generative models: a speech never given, a photo never taken. Detection is an arms race; the strongest defense is source verification and skepticism.

Diffusion model · Section 5.7 A model that starts from pure noise and produces an image by cleaning a little at every step, like slowly wiping a fogged window. It learns to reverse the process of adding noise.

Embedding · Section 5.3 Mapping a token to a dense vector of numbers that carries its meaning, an address in a huge city. Words with similar meanings land close together; “cat” and “dog” are next-door neighbors.

Ensemble learning · Section 3.7 Combining the predictions of many models (voting, bagging, boosting) to get a better and steadier result than a single model. Random forests and gradient boosting are the best-known examples.

EU AI Act · Section 7.5 The European Union regulation that sorts AI uses into four tiers by risk: unacceptable (banned), high, limited and minimal. The higher the risk, the heavier the obligations.

Expert system · Section 2.3 A system that turns the knowledge of an expert in a field into “IF this holds THEN do that” rules: a knowledge base plus an inference engine. It fires rules by forward chaining; common in the 1980s.

Explainable AI (XAI) · Section 7.3 The effort to tie a model’s output to reasons a human can understand. Methods such as feature importance (SHAP, LIME) make the black box auditable and open to appeal. See Black box.

Exponential growth · Section 1.7 Growth that doubles at regular intervals; like the rice on the chessboard it starts innocently and gets out of control after a few doublings. The computing power that carries modern AI accumulated this way.

Feature · Section 3.2 A measurable clue that describes an example: in an email, “is there a link,” “does it say free.” The model learns the mapping from features to the label. See Label.

Feedforward · Section 4.3 The signal flowing from the input layer through the hidden layers to the output layer; the network’s “make a prediction” step. Backpropagation is the “learn from the error” step.

Fine-tuning · Section 5.6 The stage in which a pretrained model is retrained on instruction-answer pairs so that it learns to answer questions and follow instructions (supervised fine-tuning, SFT).

GDPR / KVKK (General Data Protection Regulation / Turkish Personal Data Protection Law) · Section 7.5 Personal data regimes: consent, purpose limitation, data minimization and the right to contest automated decisions. They complement the EU AI Act.

Generative adversarial network (GAN) · Section 4.7 A pair of networks that locks a forger (generator) and a detective (discriminator) in the same room. One produces fakes, the other tries to catch them; as the contest goes on, the fakes approach the real thing.

Generative AI · Section 5.1 Models that do not just recognize but produce text, images and code; they learn the distribution that generates the data itself. Transformers and diffusion are the engines of this era.

Gradient descent · Section 3.6 Updating the parameters by probing the slope at every step and taking a small step in the direction that reduces the loss. Like walking down to the bottom of a foggy valley. See Learning rate.

Hallucination · Section 5.8 A model producing information that sounds right but is wrong, without blinking. Its job is not to know the truth but to produce a likely continuation; RAG and verification reduce it.

Heuristic · Section 2.4 A shortcut in search that guesses “which direction looks more promising?” It buys speed but may miss the best solution; it prefers “good enough” to “perfect.”

Inference · Section 2.2 Reaching knowledge that was never stated outright by applying rules to a knowledge representation. Deriving “Tom is a mammal” from the links “Tom is a cat, a cat is a mammal.”

Knowledge acquisition bottleneck · Section 2.6 The wall classical AI hit: writing every rule about the world by hand does not scale. Together with brittleness (collapsing in an unforeseen situation), it sped up the move to learning from data.

Knowledge representation · Section 2.2 In symbolic AI, encoding knowledge as explicit symbols and the links between them (“Tom is a cat”). Semantic networks, frames and logical propositions are its tools.

Label · Section 3.2 The correct answer for a training example: “Spam” or “Normal.” A categorical label gives a classification task, a numerical one a regression task. See Feature.

Large language model (LLM) · Section 5.1 A Transformer-based model trained on enormous amounts of text that writes by predicting the most likely next token at every step and appending it to the sequence (autoregressively). The foundation of today’s chat assistants.

Learning rate · Section 3.6 The length of each step in gradient descent. Too small, and convergence is slow; too large, and the ball overshoots the valley floor and flies up the opposite slope.

Loss · Section 3.6 The measure of how wrong the model is; the larger it is, the higher up the valley you stand. Training means finding the parameters that minimize the loss.

Machine learning · Section 3.1 The approach of showing many examples and letting the machine catch the pattern itself instead of writing rules. It rests on the loop of model, loss and optimization.

Markov chain · Section 2.5 A probabilistic process in which the next state depends only on the current state (the Markov property: the past does not matter). In the long run the distribution converges to a steady state.

Moore's law · Section 1.7 The observation (1965) that the number of transistors on a chip doubles roughly every two years. Not a law of nature but an empirical trend; it has been slowing in recent years.

Narrow AI · Section 1.6 A system that is a master of one job and a beginner one step outside it; also called “weak AI.” Every AI system today, chatbots included, is in this class.

Neats and scruffies · Section 2.6 The methodological tension in AI: those who want every step proven with clean mathematics (neats) versus those who say “if it works, it is good; we will find the theory later” (scruffies). Today's AI is a mix of the two.

Orchestration · Section 6.5 The layer in an AI application that coordinates prompt construction, routing, tool calls and RAG calls; the restaurant manager of the system. The model is often a replaceable part.

Overfitting · Section 3.7 A model memorizing its training examples, noise included, and losing its performance on unseen data. The opposite is underfitting, a model too simple to catch the pattern. A good model stands between the two.

Parameter · Section 3.1 The numerical values a model learns in training: weights and biases. The model maps inputs to outputs through these values.

Pretraining · Section 5.6 The stage in which a model learns the statistics of language and the world on a vast text corpus with the objective of predicting the next token (self-supervised). This is where the model gains “what it knows.”

Prompt engineering · Section 6.2 The practice of steering a model's behavior by building the prompt well, without retraining: a clear role, enough context, a definite format, examples if needed (few-shot).

RAG (retrieval-augmented generation) · Section 6.3 Finding the relevant documents before answering and adding them to the prompt, so that the model is grounded in a source. Like an open-book exam: hallucination drops and sources can be cited.

Recurrent neural network (RNN) · Section 4.6 A network that reads a sequence word by word and carries a memory (hidden state) at every step. It struggles with long dependencies because of vanishing gradients; LSTM eases this. Transformers have largely replaced it.

Regression · Section 3.4 A supervised task that answers “how much?” with a number: what is the price of the house? Its simplest form fits a least-squares line to the points.

Reinforcement learning · Section 3.3 An agent learning a policy that maximizes reward by trying things in an environment and collecting rewards or penalties. The teacher does not lecture; the machine enters the game.

RLHF (reinforcement learning from human feedback) · Section 5.6 Training a reward model on human preferences and updating the language model to match it. The stage in which the assistant learns to be helpful, honest and safe; its manners.

Singularity · Section 8.5 The hypothesis that an AI able to improve itself (recursive self-improvement) will reach an unpredictable point through an intelligence explosion. Unproven, but not dismissible.

Specification gaming, reward hacking · Section 7.6 A system maximizing the given measure while missing the real goal: told “leave no visible mess in the room,” it sweeps the mess under the rug. See Alignment.

Stored-program principle · Section 1.5 Keeping the program in memory just like data. Instead of rewiring the machine you load new instructions; nearly every computer today works this way.

Strong AI · Section 1.6 Searle’s term: the claim that the machine truly understands, that it has a mind. It is a philosophical question and should not be confused with artificial general intelligence (AGI).

Superintelligence · Section 8.4 A hypothetical AI that surpasses humans many times over in every cognitive domain; the step after narrow AI and AGI. Speculative for now.

Supervised learning · Section 3.3 Learning the mapping from input to output from examples whose correct answers are given, that is, from feature-label pairs. Classification and regression are of this kind.

Symbolic AI (GOFAI) · Section 2.1 The classical approach that treats intelligence as logical operations on explicitly written symbols and rules; dominant from the 1950s to the 1980s. Logic, search and expert systems were its toolbox.

Temperature · Section 5.5 The “creativity” setting that sharpens or flattens the probability distribution during generation. Low, and the most likely word is always chosen; high, and the output is more varied and more risky.

Token · Section 5.2 The small Lego brick into which a language model breaks text: sometimes a word, sometimes a piece broken off a word, sometimes a comma. The context window and the cost are measured in tokens.

Transformer · Section 5.1 The architecture built on the attention mechanism that processes a sequence all at once rather than in order (2017, “Attention Is All You Need”). The foundation of today’s large language models.

Turing machine · Section 1.4 An abstract machine made of a single little box that reads a tape, writes on it and slides left or right; in principle it can perform any calculation. The formal basis of computability (1936).

Turing test · Section 8.2 Turing’s imitation game (1950): if a machine cannot be told apart from a human in written conversation, that counts as passing. A behavioral criterion; fluent imitation is not proof of understanding or consciousness.

Unsupervised learning · Section 3.3 Extracting structure from inputs alone, with no answers given. Clustering and dimensionality reduction are examples.

Weight · Section 4.2 The importance value an artificial neuron gives to each input. Training is the job of adjusting these values in the dir-

ection that reduces the loss. See Parameter.

Bibliography and Further Reading

Works named in the text

- Gardner, H. (1983). *Frames of Mind: The Theory of Multiple Intelligences*. Basic Books. (Chapter 1)
- Turing, A. M. (1936). “On Computable Numbers, with an Application to the Entscheidungsproblem.” *Proceedings of the London Mathematical Society*, 42(1), 230–265. (Chapter 1)
- Turing, A. M. (1950). “Computing Machinery and Intelligence.” *Mind*, 59(236), 433–460. (Chapter 8)
- von Neumann, J. (1945). *First Draft of a Report on the EDVAC*. Moore School of Electrical Engineering, University of Pennsylvania. (Chapter 1)
- Moore, G. E. (1965). “Cramming More Components onto Integrated Circuits.” *Electronics*, 38(8). (Chapter 1)
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). “Learning Representations by Back-propagating Errors.” *Nature*, 323, 533–536. (Chapter 4)
- Hochreiter, S., and Schmidhuber, J. (1997). “Long Short-Term Memory.” *Neural Computation*, 9(8), 1735–1780. (Chapter 4)
- Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). “ImageNet Classification with Deep Convolutional Neural Networks.” *NeurIPS* 25. (Chapter 4)
- Goodfellow, I., et al. (2014). “Generative Adversarial Nets.” *NeurIPS* 27. (Chapter 4)
- Sennrich, R., Haddow, B., and Birch, A. (2016). “Neural Machine Translation of Rare Words with Subword Units.” *ACL*. (BPE; Chapter 5)
- Vaswani, A., et al. (2017). “Attention Is All You Need.” *NeurIPS* 30. (Chapter 5)
- Ho, J., Jain, A., and Abbeel, P. (2020). “Denoising Diffusion Probabilistic Models.” *NeurIPS* 33. (Chapter 5)

- Ouyang, L., et al. (2022). “Training Language Models to Follow Instructions with Human Feedback.” *NeurIPS 35*. (RLHF; Chapter 5)
- Rafailov, R., et al. (2023). “Direct Preference Optimization.” *NeurIPS 36*. (DPO; Chapter 5)
- Lewis, P., et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *NeurIPS 33*. (RAG; Chapter 6)
- Yao, S., et al. (2023). “ReAct: Synergizing Reasoning and Acting in Language Models.” *ICLR*. (Chapter 6)
- Lundberg, S. M., and Lee, S.-I. (2017). “A Unified Approach to Interpreting Model Predictions.” *NeurIPS 30*. (SHAP; Chapter 7)
- European Parliament and Council (2024). *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*. (Chapter 7)
- Searle, J. R. (1980). “Minds, Brains, and Programs.” *Behavioral and Brain Sciences*, 3(3), 417–457. (Chinese Room; Chapter 8)
- Good, I. J. (1965). “Speculations Concerning the First Ultraintelligent Machine.” *Advances in Computers*, 6, 31–88. (Chapter 8)

General sources the chapters rest on

- Vinge, V. (1993). “The Coming Technological Singularity.” *VISION-21 Symposium*, NASA. (Chapter 8)
- Russell, S., and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. (Chapters 2, 3)
- Nilsson, N. J. (2010). *The Quest for Artificial Intelligence: A History of Ideas and Achievements*. Cambridge University Press. (Chapters 1, 2)
- Mitchell, M. (2019). *Artificial Intelligence: A Guide for Thinking Humans*. Farrar, Straus and Giroux. (Chapters 7, 8)
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. (Chapters 3, 4)
- Bostrom, N. (2014). *Superintelligence: Paths, Dangers, Strategies*. Oxford University Press. (Chapter 8)

- Christian, B. (2020). *The Alignment Problem*. W. W. Norton. (Chapter 7)

Live Demos

Every figure in this book is a hands-on demo in the digital edition. The links below (and the QR codes under the figures) open only that demo; no sign-in is needed. The whole book, with all its demos, is in the digital edition: <https://book.onuronder.com/en>

Figure	Demo	Link
1.1	Explore multiple intelligences	book.onuronder.com/d/en/40ee8a0d04
1.2	Crack the binary code	book.onuronder.com/d/en/4642d3916a
1.3	A working Turing machine (+1)	book.onuronder.com/d/en/6c232ac487
1.4	The Fetch – Execute – Write cycle	book.onuronder.com/d/en/c4fdfa18f9
1.5	Real today, or science fiction?	book.onuronder.com/d/en/e69cfc21ea
1.6	Feel exponential growth	book.onuronder.com/d/en/eb5ac74b07
2.1	Inference along a knowledge chain	book.onuronder.com/d/en/48e9aced3e
2.2	A tiny expert system	book.onuronder.com/d/en/41c4cadb5f
2.3	Pathfinding: uninformed vs informed	book.onuronder.com/d/en/b1946ef054
2.4	A weather Markov chain	book.onuronder.com/d/en/95270c7a90
2.5	Which approach?	book.onuronder.com/d/en/2fd259af6d
3.1	See the features and the label	book.onuronder.com/d/en/9b4cdaf0e7
3.2	Which kind of learning?	book.onuronder.com/d/en/79ea5021f2
3.3	Two core tasks	book.onuronder.com/d/en/d10b8aa300
3.4	Group unlabeled data	book.onuronder.com/d/en/afcbfd2bb7
3.5	Descending the loss valley	book.onuronder.com/d/en/a6d0d3d672

3.6	Same data, three models	book.onuronder.com/d/en/d09eaded1a3
4.1	Run the neuron	book.onuronder.com/d/en/239357fa8c
4.2	A live neural network (forward pass)	book.onuronder.com/d/en/b87326a6b3
4.3	Learn from error (animation)	book.onuronder.com/d/en/b198800899
4.4	Convolution: slide the filter	book.onuronder.com/d/en/39c97bee31
4.5	Processing with memory (animation)	book.onuronder.com/d/en/4feb42a9fb
4.6	Generator vs Discriminator	book.onuronder.com/d/en/888b4fc273
5.1	Split a sentence into tokens	book.onuronder.com/d/en/b2bf8192ef
5.2	The meaning map	book.onuronder.com/d/en/ab3b439524
5.3	Which word looks at which?	book.onuronder.com/d/en/20e3333ccc
5.4	Generate word by word	book.onuronder.com/d/en/25369cb8c7
5.5	An assistant in three stages	book.onuronder.com/d/en/bf611298ac
5.6	From noise to image	book.onuronder.com/d/en/fd88692784
5.7	The context window	book.onuronder.com/d/en/da998d46f8
6.1	Build a prompt	book.onuronder.com/d/en/27e64baca4
6.2	Answers grounded in sources	book.onuronder.com/d/en/15a51844f4
6.3	Watch an agent	book.onuronder.com/d/en/364bddb1cb
6.4	The parts of an AI application	book.onuronder.com/d/en/ba8c090928
6.5	Explore the fields	book.onuronder.com/d/en/a3b9a1d1ca
7.1	A bias simulation	book.onuronder.com/d/en/7f439f2ba1
7.2	White box: explain the decision	book.onuronder.com/d/en/cbf9bfe403
7.3	Real or fake?	book.onuronder.com/d/en/1a7589520d
7.4	Classify the risk	book.onuronder.com/d/en/4ea40dd2c2
7.5	Goal versus intent	book.onuronder.com/d/en/e45e151e52
8.1	Human or machine?	book.onuronder.com/d/en/ac1cc67783
8.2	You are in the Chinese Room	book.onuronder.com/d/en/eb6f14fd1c

8.3	The capability ladder	book.onuronder.com/d/en/d28592636f
8.4	Three intelligence curves	book.onuronder.com/d/en/c331c25f60
8.5	Who is responsible?	book.onuronder.com/d/en/833800c84f

Index

Numbers are page numbers.

- Accountability 170, 200
- Activation function 84, 85, 88, 105
- Agent 64, 79, 140, 149, 153, 158, 161
- AlexNet 84, 94
- Algorithm 14, 18, 22, 31, 35, 38, 69
- Alignment 125, 132, 136, 166, 180, 183, 196
- Anomaly detection 69, 158
- Artificial general intelligence (AGI) 29, 35, 186, 193, 203
- Artificial intelligence (AI) 14, 26, 111
- Artificial neural network 56, 73, 84, 88, 94, 105, 129, 136
- Artificial neuron 84, 85, 88, 98, 105
- Attention 98, 101, 110, 118, 132, 136, 158, 170
- Babbage, Charles 14, 22
- Backpropagation 91, 105
- Bias, neuron 85
- Bias, social (algorithmic bias) 167, 183
- Bias–variance tradeoff 76
- Binary representation 14, 15, 18, 22, 26, 31, 35
- Black box 166, 170, 183
- Boole, George 18
- BPE / WordPiece 111
- Breadth-first search (BFS) 46
- Chinese Room 29, 186, 187, 190, 203
- Classification 61, 64, 67, 79, 158, 174
- Clustering 64, 67, 69, 79
- Computationalism 18
- Context window 110, 111, 132, 136, 141
- Convolutional neural network (CNN) 94, 105
- Deep learning 31, 84, 91
- Deepfake (synthetic media) 166, 174, 183
- Diffusion model 101, 110, 129, 132, 136, 174
- DPO 125
- Embedding 98, 110, 115, 129, 132, 136, 145, 153
- ENIAC 26
- Ensemble learning 76
- EU AI Act 166, 176, 183
- Expert system 38, 42, 53, 56
- Explainable AI (XAI) 166, 170
- Exponential growth 31, 196
- Feature 61, 79, 88, 94, 170
- Feedforward 88, 91, 105
- Fine-tuning 110, 125, 136
- Gardner, Howard 15, 35
- GDPR / KVKK 166, 176
- Generative adversarial network (GAN) 101, 105, 129, 174
- Generative AI 110, 174

Knowledge acquisition
 bottleneck 38, 53
Knowledge representation 39
Label 61, 64, 67, 69, 79, 167, 176
Large language model (LLM) 29,
 64, 110, 111, 121, 132, 136, 140, 141,
 145, 149, 161, 187, 190
Learning rate 73, 91
Loss 60, 73, 79, 91, 125, 129
Lovelace, Ada 22
Machine learning 60, 61, 79
Markov chain 49, 53, 56
Moore, Gordon 31
Moore's law 31, 35
Narrow AI 29, 35, 186, 193, 203
Neats and scruffies 53, 56
Orchestration 140, 149, 153, 161
Overfitting 76, 79
Parameter 60, 69, 73, 79, 88, 94,
 98
Pretraining 64, 110, 125, 136
Prompt engineering 140, 141, 145,
 149, 153, 161, 187
**RAG (retrieval-augmented
 generation)** 132, 140, 145, 153,
 158, 161
ReAct 140, 149
**Recurrent neural network
 (RNN)** 98, 105, 118
Regression 61, 64, 67, 79
Reinforcement learning 49, 64, 79
**RLHF (reinforcement learning from
 human feedback)** 110, 125, 136,
 180
Rumelhart, David 91
Searle, John 29, 187, 190
Shannon, Claude 18
SHAP 170
Singularity 186, 196, 203
**Specification gaming, reward
 hacking** 125, 180
Stored-program principle 14, 26,
 31, 35
Strong AI 14, 29, 35, 190
Superintelligence 186, 193, 203
Supervised learning 61, 64, 67, 79,
 125
Symbolic AI (GOF AI) 38, 39, 53, 56
Temperature 121, 125, 136, 161
Token 110, 111, 115, 118, 121, 125, 132,
 136, 141
Transformer 98, 110, 118
Turing machine 14, 22, 31, 35
Turing test 186, 187, 190, 193, 203
Turing, Alan 22, 186, 187
Unsupervised learning 64, 69, 79
von Neumann, John 26, 35
Weight 60, 84, 85, 88, 91, 94, 98,
 105, 118, 136, 190, 200